

i.MX8M Plus Flight Computer

Knowledge Transfer Documentation

Guided Systems Division

December 6, 2025

Contents

1	Introduction	8
1.1	About This Document	8
1.1.1	My Information	8
1.2	Project Overview	8
1.3	Key Achievements & Limitations	9
1.3.1	What Was Accomplished	9
1.3.2	Pending Tasks (for the next intern) & Known Limitations	10
1.4	Handover Items	11
1.4.1	Physical Hardware	11
1.4.2	Digital Assets	11
1.5	How to Use This Documentation	11
1.5.1	Document Structure	11
1.5.2	Icon Legend	11
2	Quick Start Guide	13
2.1	Hardware Overview	13
2.2	Boot Configuration	14
2.2.1	Boot Source: eMMC	14
2.3	Serial Console Setup	14
2.3.1	UART Port Assignments	14
2.3.2	Recommended Terminal: MobaXterm	15
2.3.3	First Boot and Login	15
2.4	Pre-Built BSP and Resources	15
2.4.1	Locating Pre-Built Images	15
2.4.2	Useful NXP Resources	16
2.5	JTAG Debugger Setup (For M7 Core Development)	16
2.5.1	Hardware Connection	16
2.6	Network Configuration	17
2.6.1	Ethernet Connection	17
2.7	Verifying the Setup	18
2.7.1	Check A53 Cores	18
2.7.2	Check Yocto Build Version	18
2.8	Prerequisites for DSO Laptops	18
2.8.1	Required DSO IT Documentation	18
3	System Architecture	20
3.1	Essential Components for a Bootable Linux System	20
3.1.1	The Four Required Components	20

3.1.2	Where Each Component Lives on eMMC	21
3.2	Complete Boot Sequence	21
3.2.1	Boot Flow: Power-On to Login Prompt	21
3.2.2	Visual Boot Flow	23
3.3	Yocto Build Outputs Explained	23
3.3.1	.wic File - Complete Disk Image	23
3.3.2	Rootfs Archive - Filesystem Only	24
3.3.3	Individual Component Files	24
3.3.4	Update Device Tree Only	24
3.3.5	Update Rootfs Only	26
3.4	M7 Firmware Loading (Brief Overview)	27
3.4.1	M7 Startup Modes	27
3.4.2	Quick M7 Control Commands	27
3.5	Software Stack by Core	28
3.5.1	Cortex-A53 Cores (Application Processors)	28
3.5.2	Cortex-M7 Core (Real-Time Processor)	28
3.5.3	NPU (Neural Processing Unit)	28
3.5.4	DSP (Digital Signal Processor)	28
3.6	Memory Architecture	29
3.6.1	Physical Memory Map	29
3.7	Inter-Processor Communication (Planned)	29
3.8	Resource Domain Controller (RDC)	30
3.8.1	Current RDC Configuration	30
4	Yocto Build Environment	31
4.1	Understanding Yocto Architecture	31
4.1.1	The Yocto Hierarchy	31
4.1.2	Key NXP Layers in Our Build	32
4.2	Option 1: Using Pre-Built Images (Quick Start)	32
4.2.1	Download Pre-Built BSP	33
4.2.2	Install UUU (Universal Update Utility)	33
4.2.3	Flash to eMMC Using UUU	33
4.3	Option 2: Building from Source (Recommended for Development)	34
4.3.1	Prerequisites (follow the documentation)	34
4.3.2	Install repo Utility	34
4.3.3	Download i.MX Yocto BSP Layers	35
4.3.4	Initialize Yocto Build Environment	35
4.3.5	Choose Image Type	36
4.3.6	Build the Image	37
4.3.7	Optimize Build Performance	37
4.3.8	Troubleshooting Build Errors	38
4.3.9	Build Output Location	38
4.4	Adding Custom Packages (Zabbix Example)	38
4.4.1	Step 1: Search OpenEmbedded Layer Index	38
4.4.2	Step 2: Verify Layer is Already Included	39
4.4.3	Step 3: Add Package to Image	40
4.4.4	Step 4: Rebuild Image	40
4.4.5	What If the Package Doesn't Exist?	40

4.5	Creating Your Custom Layer	41
4.5.1	Create meta-chyn Layer	42
4.6	Creating patches for ARM Trusted Firmware	42
4.6.1	Goal	42
4.6.2	Use devtool to Modify ATF	42
4.6.3	Make Your Changes	43
4.6.4	Commit Your Changes	43
4.6.5	Handle Build Artifacts (if present)	44
4.6.6	Finish devtool and Create Patch	44
4.6.7	Add Required Patch Headers	44
4.6.8	Rebuild ATF with Patches	44
4.6.9	Verify Patches Were Applied	45
4.6.10	Extract and Flash Updated Bootloader	45
4.7	Repository Directory Structure (for Reference)	45
4.7.1	Linux Kernel Structure	45
4.7.2	ARM Trusted Firmware Structure	46
5	Device Tree Configuration	47
5.1	What is Device Tree?	47
5.1.1	Purpose and Overview	47
5.1.2	File Types	47
5.2	Checking Your Current Configuration	48
5.2.1	Which DTB is Currently Loaded?	48
5.2.2	Available DTB Files	48
5.3	Device Tree Compiler (dtc)	49
5.3.1	Essential dtc Flags	49
5.3.2	Common dtc Operations	49
5.3.3	Comparing Device Trees	50
5.4	Device Tree Syntax Fundamentals	50
5.4.1	Nodes and Hierarchy	50
5.4.2	Properties	51
5.4.3	Labels and Phandles	51
5.4.4	Interrupts	52
5.4.5	Pin Control (i.MX Specific)	52
5.5	Complete Workflow: Adding an IMU Sensor	53
5.5.1	Step 1: Identify Hardware Connections	53
5.5.2	Step 2: Find and Extract Base Device Tree	54
5.5.3	Step 3: Locate I2C5 Controller Node	54
5.5.4	Step 4: Verify/Add Pin Mux Configuration	54
5.5.5	Step 5: Add MPU-6050 Sensor Node	55
5.5.6	Step 6: Compile Modified Device Tree	55
5.5.7	Step 7: Configure U-Boot to Use New DTB	56
5.5.8	Step 8: Verify Hardware Configuration	56
5.6	Common Device Tree Modifications	57
5.6.1	Disabling Unused Peripherals	57
5.6.2	Changing I2C Bus Speed	58
5.6.3	Adding GPIO-Based Devices	58
5.7	Device Tree Overlays	58

5.8	Reference: i.MX8MP Device Tree Locations	58
5.8.1	Linux Kernel Source Structure	58
5.8.2	Key Files to Reference	59
5.8.3	Common Pitfalls	59
6	Monitoring Infrastructure	60
6.1	Why Monitoring is Essential	60
6.1.1	Purpose of This Setup	60
6.2	Architecture Overview	61
6.2.1	Component Roles	62
6.3	Prerequisites	62
6.3.1	Network Requirements	62
6.3.2	Software Requirements	62
6.4	Docker Setup (Laptop/Host)	63
6.4.1	Prerequisites: Docker Installation	63
6.4.2	Configuration Files Repository	63
6.4.3	Create Project Directory	63
6.4.4	Docker Compose Configuration	63
6.4.5	Start Docker Containers	65
6.4.6	Access Web Interfaces	65
6.4.7	Future Work: External Database	66
6.5	Zabbix Agent Setup (i.MX8MP Board)	66
6.5.1	Verify Zabbix Agent Installation	66
6.5.2	Configure Zabbix Agent	67
6.5.3	Create Custom Metrics Configuration	67
6.5.4	Start Zabbix Agent	68
6.5.5	Test Custom Metrics	68
6.6	Power Domain Monitoring Deep Dive	68
6.6.1	What Are Power Domains?	68
6.6.2	Power State Files	69
6.6.3	Why This Matters	69
6.7	Zabbix Web UI Configuration	70
6.7.1	Add Host (i.MX8MP Board)	70
6.7.2	Create Custom Items	70
6.8	Grafana Dashboard Setup	71
6.8.1	Install Zabbix Plugin	71
6.8.2	Add Zabbix Data Source	71
6.8.3	Create Power Domain Dashboard	71
6.8.4	Add System Metrics Panels	72
6.9	System Validation with Stress Testing	73
6.9.1	What is stress-ng?	73
6.9.2	Installation	74
6.9.3	Stress Testing Commands	74
6.9.4	Stress Test Categories	75
6.9.5	Useful stress-ng Options	76
6.9.6	Thermal Behavior Observations	77
6.9.7	What could be explored next: CPU Frequency Scaling and Governors	77
6.9.8	Other Unexplored Monitoring Topics	81

6.10	Startup Procedure (Quick Reference)	82
6.10.1	Every Time You Start Monitoring	82
7	NPU and Machine Learning	83
7.1	Introduction	83
7.1.1	Why NPU Matters for Embedded Systems	83
7.1.2	Scope of This Investigation	84
7.2	NPU Architecture and Power Management	85
7.2.1	What is a Delegate?	85
7.2.2	NPU Power Efficiency	85
7.3	TensorFlow Lite Setup	86
7.3.1	Pre-installed Runtime	86
7.4	Basic Inference with MobileNet	87
7.4.1	Test Setup	87
7.4.2	Running CPU-Only Inference	87
7.4.3	Running NPU-Accelerated Inference	87
7.5	Performance Profiling	88
7.5.1	NPU Profiling Environment Variables	88
7.5.2	Understanding Profiling Metrics	89
7.5.3	Batch Testing and Logging	89
7.6	Model Quantization	90
7.6.1	What is Quantization?	90
7.6.2	MobileNet V1 Quantized Model	90
7.6.3	Quantization Strategies (Unexplored)	91
7.7	Unexplored Areas and Future Work	92
7.7.1	Computer Vision Requirements for Guided Systems	92
7.8	Summary/tldr	92
7.8.1	What Was Accomplished	92
7.8.2	Key Findings	93
8	DSP Exploration	94
8.1	HiFi 4 DSP Overview	94
8.1.1	What is the DSP?	94
8.1.2	Typical Use Cases	95
8.2	Architecture and Integration	95
8.2.1	Similarity to M7 Core	95
8.2.2	RPMmsg and Remoteproc Framework	96
8.3	Running Zephyr RTOS on DSP	97
8.3.1	Zephyr RTOS Overview	97
8.3.2	Documentation Reviewed	98
8.4	Summary	98
8.4.1	What Was Discovered	98
8.5	What I think could be potential Use Cases	99
8.5.1	Tasks that could be offloaded to dsp	99
8.5.2	Learning Resources	99

9	M7 Core Development	100
9.1	M7 Core Overview	101
9.1.1	What is the M7 Core?	101
9.1.2	Why M7 for Flight Control	101
9.1.3	Heterogeneous Architecture	102
9.2	Development Workflow with MCUXpresso	103
9.2.1	MCUXpresso SDK Builder (Online)	103
9.2.2	SDK Directory Structure	105
9.2.3	MCUXpresso for VS Code Setup	106
9.2.4	Starting a New Project from Example	107
9.2.5	Key Files to Modify	109
9.2.6	Building and Debugging	110
9.2.7	Working with GitHub Examples	111
9.2.8	My Typical Development Cycle	112
9.2.9	Printf and Floating Point	113
9.3	Memory Architecture	113
9.3.1	Complete Memory Map	113
9.3.2	TCM: Tightly Coupled Memory	114
9.3.3	DDR Memory for M7	115
9.3.4	Cacheable vs NonCacheable Memory	115
9.3.5	Linker Script Configuration	116
9.3.6	Linker Script Testing: TCM vs DDR Performance	119
9.4	Device Tree and Boot Methods	123
9.4.1	Device Tree Configuration	123
9.4.2	Reserved Memory Regions	123
9.4.3	Understanding Remoteproc and RPMsg	124
9.4.4	M7 Boot Sequence	124
9.4.5	Initial Testing with NXP SDK Examples	125
9.4.6	Deploying Your Firmware	126
9.5	Peripheral Development	127
9.5.1	Pin Muxing (IOMUXC)	127
9.5.2	I2C: Dual IMU Configuration	128
9.5.3	PWM for LED Control and Signal Generation	130
9.5.4	CAN Bus Communication	131
9.5.5	GPT: Input Capture for RC Receivers	132
9.6	DMA (Direct Memory Access)	133
9.6.1	Why DMA Matters	133
9.6.2	SDMA on i.MX8MP	134
9.6.3	Critical: NonCacheable Memory	134
9.6.4	DMA Examples	134
9.7	Power Management and Suspend Coordination	134
9.7.1	Low-Power Modes	135
9.7.2	M7 Running During A53 Suspend	135
9.7.3	Waking A53 from M7	135
9.7.4	Complete Suspend/Resume Flow	136
9.8	RPMsg Communication	136
9.8.1	RPMsg Overview	136
9.8.2	RPMsg Status During Internship	137

9.9	RDC and Peripheral Ownership	137
9.9.1	Peripheral Conflict Prevention	137
9.9.2	Shared Peripheral Strategies	138
9.10	NuttX RTOS Port	139
9.10.1	NuttX Port Status	139
9.10.2	NuttX Setup Process	139
9.10.3	Board Configuration	140
9.10.4	Loading NuttX to M7 Core	140
9.10.5	UART4 Console Configuration	141
9.10.6	I2C3 Bus Configuration	142
9.10.7	I2C Bus Scanning	143
9.10.8	ADXL345 Accelerometer Verification	143
9.10.9	RPMsg Integration Attempt	144
9.11	PX4 Flight Controller Integration	145
9.11.1	PX4 Port Attempt	145
9.11.2	What Was Attempted	145
9.11.3	Why PX4 Integration Failed	147
9.11.4	Files Modified During PX4 Attempt	147
9.11.5	Key Resources	148
10	Conclusion and Support	149
10.1	Wrapping Up	149
10.2	Documentation	149
10.3	Getting Help	149
10.3.1	Contact me	149
10.3.2	NXP Community Forums - Your Best Friend	150
10.4	Final Words	150

Chapter 1

Introduction

1.1 About This Document

This documentation should serve as a comprehensive knowledge transfer guide for future interns/engineers continuing work on the iMX8MP processor optimization for flight control purposes. Here I will consolidate my findings, configurations, troubleshooting solutions, and pending tasks discovered across internship at DSO's Guided Systems Division.

1.1.1 My Information

- **Name:** Chyn
- **Institution:** NUS Y3 EE
- **Internship end date :** 5th December 2025
- **Email/telegram:** tanchyn2@gmail.com / @yulyuri

1.2 Project Overview

The primary goal of my internship was to explore, configure and optimise the IMX8MP platform as a foundation for a drone flight computer. I am the first intern to start on this project and the work involves acquiring knowledge of the SOM in order to optimise power, performance and thermal characteristics for drone applications. Development is conducted using the nxp imx8mp evaluation board but with plans to migrate to the compulab SOM.

1.3 Key Achievements & Limitations

1.3.1 What Was Accomplished

- **Complete Yocto Build System** - Fully functional meta-layers with custom package integration and M7 core development environment
- **Production-Ready Low-Power Suspend Mode** - Implemented system suspend with Messaging Unit (MU) wake mechanism based on AN13400, achieving significant power savings. Working configuration saved on labeled SD card.
- **ARM Trusted Firmware Modifications** - Successfully patched ATF to disable PLLs during suspend (DRAM PLL, ARM PLL, System PLLs, Audio PLL)
- **Monitoring Infrastructure** - Deployed Docker-based Zabbix + Grafana stack with custom power domain monitoring scripts and stress testing methodology
- **NPU Benchmarking Analysis** - Completed performance profiling across multiple models, delegates, and quantization strategies with documented findings
- **NuttX RTOS on M7 Core** - Successfully ported and running NuttX on Cortex-M7 with UART and I2C peripheral drivers functional
- **M7 Memory Architecture Configuration** - Configured linker scripts for flexible code placement across ITCM, DTCM, and DDR regions
- **Resource Domain Controller (RDC) Configuration** - Configured RDC to properly allocate peripherals between M7 and A53 cores, preventing access conflicts
- **DMA Implementation for SPI/I2C** - Direct Memory Access configured to offload CPU from peripheral data transfers (mostly working)
- **DSP Preliminary Exploration** - Initial investigation into potential DSP workloads and acceleration opportunities
- **RPMsg Framework Setup** - Basic inter-processor communication framework explored, foundation laid for future development

1.3.2 Pending Tasks (for the next intern) & Known Limitations

Important Note

The following items require continuation and I ordered it by priority. More of this will be elaborated in the later sections.

1. **Full RPMsg Implementation** - While basic RPMsg framework has been explored, a complete structured communication protocol between M7 and A53 cores is not implemented. I believe a proper and more intuitive command/response mechanisms and data transfer protocols need to be finalised.
2. **PX4 Flight Controller Port** - NuttX RTOS is successfully running on M7, but PX4 flight control software has NOT been ported yet. This is the most critical remaining task. The Hardware Abstraction Layer (HAL) needs significant work to interface PX4 with i.MX8MP-specific peripherals.
3. **Memory Architecture Optimization** - While linker script configuration is done, the optimal strategy for balancing ITCM/DTCM/DDR usage hasn't been fully determined. Benchmarking needed to understand performance trade-offs for flight-critical code placement.
4. **DMA Completion** - DMA for SPI/I2C is mostly working but needs testing under high-load scenarios and integration with actual sensor data collection pipelines.
5. **DSP Integration** - Only preliminary exploration done. Specific workloads (sensor fusion, signal processing) need to be identified and benchmarked on the DSP core. I don't know if coding on the
6. **Comprehensive Power Profiling** - Individual power measurements exist, but systematic comparison of all PLL configurations with standardized workloads not completed. Should create spreadsheet documenting power vs performance trade-offs.
7. **SOM Migration** - All work done on EVK board. Migration to CompuLab System-on-Module for actual drone integration not started. This will require verifying all configurations work on the smaller form factor.

1.4 Handover Items

1.4.1 Physical Hardware

Hardware Items

The following hardware components are being handed over to the next intern:

- **MPU-6050 IMU Module** - 6-axis gyroscope and accelerometer (details in Chapter)
- **ADXL345 IMU Module** - 3-axis accelerometer (details in Chapter)
- **SD Card** - Contains working configuration image for low suspend mode implementation.

1.4.2 Digital Assets

GitHub Repository

All code, configurations, and scripts are available at:

<https://github.com/yulyuri/firmware-and-dts>

Key repositories:

- **dts files** - different device tree files
- **nuttX-m7-elf files** - NuttX RTOS implementation
- **monitoring-docker** - Zabbix/Grafana setup (json and xml)
- **M7** - Coding examples for low power wake, i2c comms etc

1.5 How to Use This Documentation

1.5.1 Document Structure

This documentation is organized as follows:

- **Chapter 2: Initial Setup Guide** - Start here if you're new to the project
- **Chapter 3: System Architecture** - Hardware and software overview
- **Chapter 4-8: Implementation Details** - Deep dives into specific subsystems

1.5.2 Icon Legend

Throughout this document, you'll encounter the following callout boxes:

 Related Documentation

Documentation Reference: Points to additional PDF guides or external resources for deeper information.

 GitHub Repository

GitHub Reference: Links to relevant code repositories or specific files.

 Important Note

Important Note: Critical information, warnings, or prerequisites you should not skip.

 Hardware Items

Hardware Items: Physical components, tools, or equipment mentioned.

Chapter 2

Quick Start Guide

This chapter will get you up and running with the i.MX8M Plus EVK board. Read this section carefully before powering on the hardware or attempting any development work.

2.1 Hardware Overview

The i.MX8M Plus is a heterogeneous multicore System-on-Chip (SoC) with several processing units, each optimized for different tasks:

- **4x ARM Cortex-A53 Cores (Application Processors)** - Currently running a custom Yocto Linux image that handles high-level tasks like computer vision, machine learning inference, and general application logic.
- **1x ARM Cortex-M7 Core (Real-Time Processor)** - Running at 800 MHz, this is a dedicated real-time core ideal for time-critical tasks like flight control loops. Has its own Tightly-Coupled Memory (TCM) for deterministic performance. Capable of running freeRTOS/nuttx/Zephyr or bare-metal.
- **Neural Processing Unit (NPU)** - AI accelerator for running machine learning models efficiently. Supports TensorFlow Lite delegates for offloading inference from the A53 cores.
- **Digital Signal Processor (DSP)** - Tensilica HiFi4 DSP for signal processing tasks. Can be used for sensor fusion algorithms, audio processing, or other compute-intensive signal processing workloads.

Important Note

Key Architectural Point: The heterogeneous nature of this SoC is what makes it suitable for drone flight computers. The end goal should be where the M7 core handles real-time flight control while the A53 cores run AI/vision algorithms/ML tasks.

Related Documentation

Relevant NXP's official documentation:

i.MX8M Plus EVK Quick Start Guide (8MPLUSEVKQSG)

This document covers:

- Basic board bring-up

imx8mpevk hardware user guide

This document covers:

- what each connectors mean (j19 j20 etc)
- pinouts for connector pins/expansion connector

2.2 Boot Configuration

2.2.1 Boot Source: eMMC

Important Note

NoteL: The custom Yocto build with all configurations resides on the eMMC.

In U-Boot: `mmc dev 1` = SD card, `mmc dev 2` = eMMC (our working image is on `mmc 2:1`)

2.3 Serial Console Setup

2.3.1 UART Port Assignments

The i.MX8M Plus EVK has multiple USB-to-serial ports. Connect a USB cable from the board's debug port to your development machine. This will enumerate as 4 separate serial ports:

- **Port 3: A53 Core Console** - Linux shell access (main interface)
- **Port 4: M7 Core Console** - NuttX RTOS debug output and shell

Important Note

You will primarily use:

- **Port 3** for interacting with the Linux system running on A53 cores
- **Port 4** for debugging the M7 real-time core (NuttX console)

Both ports should be open simultaneously during development.

2.3.2 Recommended Terminal: MobaXterm

I felt that mobaxterm provided the best UI and its able to save your configuration for simplicity sake and it has more features if needed like SFTP

MobaXterm Setup:

1. Launch MobaXterm
2. Click "Session" → "Serial"
3. Select Port 3 (e.g., COM3 or /dev/ttyUSB2) for A53 console
4. Set speed to 115200
5. Save session as "i.MX8MP - A53 Console"
6. Repeat for Port 4, save as "i.MX8MP - M7 Console"

2.3.3 First Boot and Login

Power on the board and open both serial consoles. You should see:

Port 3 (A53):

```

1 [ OK ] Reached target Graphical Interface.
2           Starting Record Runlevel Change in UTMP...
3 [ OK ] Finished Record Runlevel Change in UTMP.
4
5 NXP i.MX Release Distro 6.6-scarthgap imx8mp-lpddr4-evk ttymxc1
6
7 imx8mp-lpddr4-evk login:

```

Login credentials:

- Username: root

2.4 Pre-Built BSP and Resources

2.4.1 Locating Pre-Built Images

If you need to recover the system or flash a new board, pre-built Board Support Package (BSP) images are available on the NXP website under BSP and Device Drivers

Important Note

However, the pre-built BSP from NXP is NOT the same as our custom Yocto build. The pre-built images:

- Do NOT include our patches/DTS/configurations
- But still useful for initial board bring-up/recovery/bench-marking

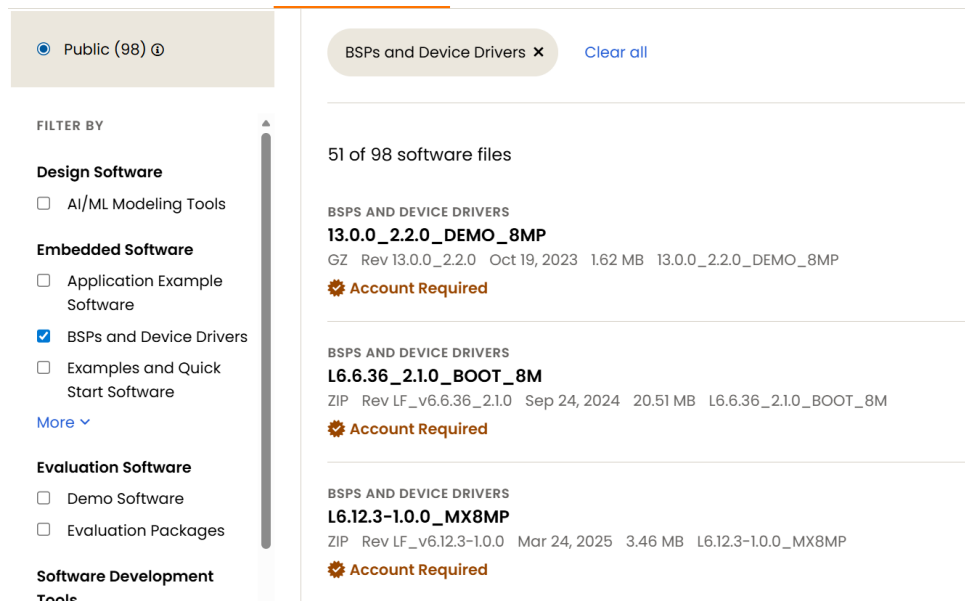


Figure 2.1: Pre-built BSP

2.4.2 Useful NXP Resources

- **i.MX8M Plus Reference Manual** - Comprehensive hardware documentation
- **i.MX Linux User's Guide** - teaches you how to build bsp, how to use u-boot
- **i.MX Linux Developer's Guide** - Kernel and driver development

2.5 JTAG Debugger Setup (For M7 Core Development)

2.5.1 Hardware Connection

If you plan to work on M7 core development or need to debug the Cortex-M7 real-time processor, you'll need a JTAG debugger connected:

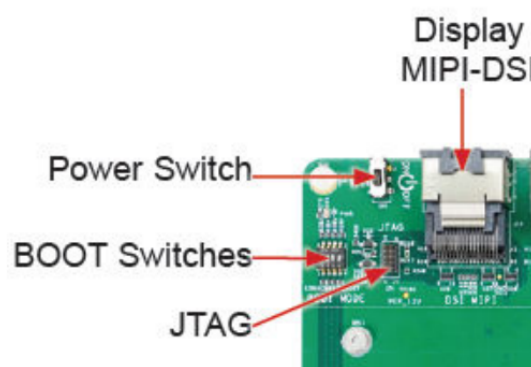


Figure 2.2: JTAG Connection to J24 Connector

Hardware Items

JTAG Hardware:

- **Board Connector:** J24 (20-pin JTAG header)
- **Purpose:** Real-time debugging of Cortex-M7 core (NuttX RTOS/freertos)

2.6 Network Configuration

2.6.1 Ethernet Connection

The easiest way to transfer files (dtb, elf files etc) is via Ethernet:

1. Connect Ethernet cable from board (ETH port) to your network
2. On the A53 console, check IP address:

```

1 root@imx8mpevk:~# ifconfig eth0
2 eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
3         inet 192.168.1.XXX netmask 255.255.255.0 broadcast 192.168.1.255
4

```

or

```

1 root@imx8mpevk:~# ip a
2 lo: .....
3 eth0: ...
4 eth1:.....
5

```

3. From your development machine/DSO laptop, you can now:
 - Use SCP for file transfers: `scp file.txt root@192.168.1.XXX:/home/root/`
 - Access via MobaXterm's built-in SFTP browser

Hardware Items

Ethernet Hardware:

- **Ethernet Cable:** Connect to eth0
- **Router:** Self-explanatory
please ask allan for help otherwise if you need a router etc :)

2.7 Verifying the Setup

Once you're logged in, verify the system is working correctly:

2.7.1 Check A53 Cores

```
1 # Check CPU info
2 root@imx8mpevk:~# cat /proc/cpuinfo
3
4 # Check if all 4 cores are online
5 root@imx8mpevk:~# lscpu | grep "On-line"
6 On-line CPU(s) list: 0-3
```

2.7.2 Check Yocto Build Version

```
1 # Check OS version (should show your custom build)
2 root@imx8mp-lpddr4-evk:~# cat /etc/os-release
3 ID=fsl-imx-xwayland
4 NAME="NXP i.MX Release Distro"
5 VERSION="6.6-scarthgap (scarthgap)"
6 VERSION_ID=6.6-scarthgap
7 VERSION_CODENAME="scarthgap"
8 PRETTY_NAME="NXP i.MX Release Distro 6.6-scarthgap (scarthgap)"
9 CPE_NAME="cpe:/o:openembedded:fsl-imx-xwayland:6.6-scarthgap"
10 root@imx8mp-lpddr4-evk:~#
```

2.8 Prerequisites for DSO Laptops

2.8.1 Required DSO IT Documentation

I think the following guides are pretty useful and I missed them out in the first few months of intern and it could have saved a lot of valuable time :

1. HTTPS Decryption and Certificate Setup

- Guide: "HTTPS Decryption Onboarding Guide"
- **Why you need this:** you need proper certificates, else Python, Anaconda, Git, Docker, and Yocto builds will fail with SSL/certificate errors.
- Follow the guide to install DSO root certificates for all development tools. You should only need 2 certificates the DSO50CA50 and DSOvisibilityCA

2. Software Installation via EPM (Enterprise Package Manager)

- Guide: "EPM Software Installation Guide"
- **Why you need this:** Many development tools require administrator rights to install. EPM allows you to request and install approved software without admin access.

- You'll need to request installation rights for most softwares.

3. WSL2 (Windows Subsystem for Linux) Setup

- Guide: "WSL2 Installation and Configuration"
- **Why you need this:** Yocto builds require a Linux environment. WSL2 is the recommended solution for DSO Windows laptops.
- The guide covers: WSL2 installation via EPM, Ubuntu setup, network configuration, and certificate integration.

Related Documentation

Where to Find These Guides:

DSO Internal Portal ↗ All Knowledge Bases ↗ IT Service ↗ Search for:

- "HTTPS Decryption"
- "EPM Software Installation"
- "WSL2 Setup"

If you cannot find these documents or encounter access issues, contact DSO IT Service Desk.

Chapter 3

System Architecture

This chapter explains the overall system architecture, boot flow, and how different components interact. Understanding this is crucial before diving into specific subsystems.

3.1 Essential Components for a Bootable Linux System

Before understanding the boot sequence, you need to know what pieces are required. To have a fully functional Linux system on the i.MX8M Plus, you need **four essential components**:

3.1.1 The Four Required Components

1. Bootloader Image (imx-boot)

- Combined binary containing U-Boot SPL, U-Boot proper, and ARM Trusted Firmware (ATF)
- First thing that runs after BootROM
- Initializes hardware and loads the kernel
- **Without this:** Board won't boot at all - BootROM won't find anything to execute

2. Linux Kernel Image

- The actual operating system kernel
- Manages hardware, memory, processes, and drivers

3. Device Tree Blob (.dtb)

- Hardware description file telling Linux what hardware exists
- Describes peripherals, memory map, clocks, GPIOs, I2C devices, etc.
- File: `imx8mp-evk.dtb` mainfile `imx8mp-evk-dsp.dtb` etc
- **Without this:** Kernel boots but can't initialize hardware - peripherals might not work immediately

4. Root Filesystem (rootfs)

- All files and directories: /bin, /lib, /etc, /usr, applications, libraries
- Everything that makes the system usable - shell, utilities, your applications
- **Without this:** Kernel boots but panics with "VFS: Unable to mount root fs"

Important Note

ALL FOUR components must be present for a successful boot. Missing any one will cause boot failure at different stages.

3.1.2 Where Each Component Lives on eMMC

Component	eMMC Location	Filesystem Type
Bootloader (imx-boot)	Offset 33KB	Raw binary
Kernel (Image)	Boot partition (mmc 2:1)	FAT32
Device Tree (.dtb)	Boot partition (mmc 2:1)	FAT32
Root Filesystem	Root partition (mmc 2:2)	ext4

3.2 Complete Boot Sequence

Now that you know what components are needed, I think it will be nice to know how they work together during boot. Understanding this sequence is critical for debugging issues and modifying the system.

3.2.1 Boot Flow: Power-On to Login Prompt

1. **Power On** - Board receives power
2. **BootROM** (on-chip, immutable)
 - Executes from internal ROM (cannot be modified)
 - Reads boot mode from DIP switches (SW1101)
 - Looks for bootloader at offset 33KB on eMMC
 - Loads U-Boot SPL into internal OCRAM
 - Validates and jumps to U-Boot SPL
3. **U-Boot SPL** (Secondary Program Loader)
 - Minimal bootloader, runs from OCRAM
 - Initializes DDR RAM (DRAM)
 - Loads full U-Boot and ATF from eMMC into DDR
 - Jumps to ARM Trusted Firmware (ATF)
4. **ARM Trusted Firmware (ATF)**
 - Configures secure/non-secure world split

- Sets up power domains and PLLs
- **Configurable** - disabled PLLs etc
- Loads M7 firmware to TCM memory (if enabled)
- Starts M7 core execution (NuttX RTOS)
- Jumps to U-Boot proper

5. **U-Boot Proper** - Full-featured bootloader

- Provides interactive console (press any key to stop autoboot)
- Reads boot partition from `mmc 2:1`
- Loads Linux kernel (**Image**) into DDR
- Loads device tree blob (`imx8mp-evk.dtb`) into DDR
- Sets kernel command line parameters (e.g., root partition location)
- Jumps to Linux kernel entry point

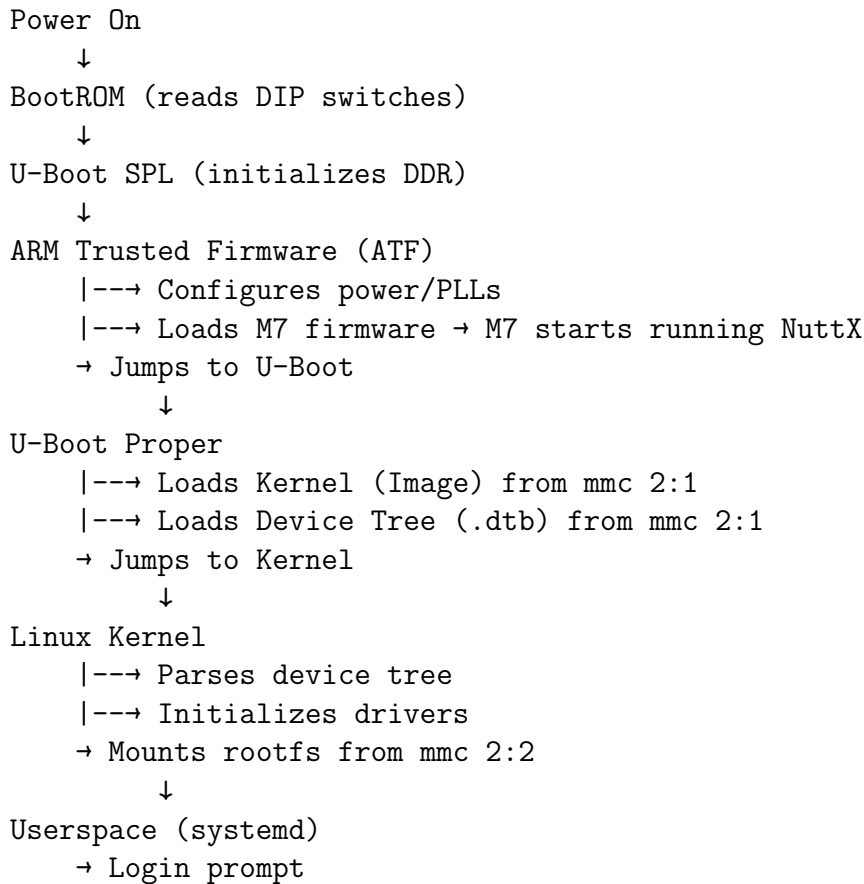
6. **Linux Kernel** - Operating system

- Initializes hardware using device tree information
- Loads drivers for peripherals (UART, I2C, GPIO, etc.)
- Sets up memory management and interrupt handling
- Starts init system (systemd)

7. **Userspace** - Applications and services

- systemd starts all configured services (SSH, networking, monitoring)
- Login prompt appears on serial console
- System ready for use

3.2.2 Visual Boot Flow



3.3 Yocto Build Outputs Explained

When you build a Yocto image, several files are generated. Here's what each one is and when to use it:

3.3.1 .wic File - Complete Disk Image

- **What it is:** A complete disk image containing ALL four components
- **File example:** `imx-image-full-imx8mpevk.wic`
- **When to use:** Initial board setup or complete system recovery
- **How to flash:**

```

1 # Then from your PC:
2 $ uuu -b emmc .wic file
3
  
```

⚠ Important Note

Warning: Flashing a .wic file will OVERWRITES THE ENTIRE eMMC, including all your configurations and data. Only use this for initial setup or complete recovery.

3.3.2 Rootfs Archive - Filesystem Only

- **What it is:** Just the root filesystem, without bootloader/kernel/DTB
- **Format:** tar.gz compressed archive or ext4 image
- **File example:** imx-image-full-imx8mp-evk.rootfs.tar.gz
- **When to use:** Update userspace applications without touching bootloader/kernel
- **Contains:** /bin, /lib, /etc, /usr, your custom applications

3.3.3 Individual Component Files

You can also extract/use individual components:

- imx-boot-imx8mp-evk-sd.bin-flash_evk - Just the bootloader
- Image - Just the kernel
- imx8mp-evk.dtb - Just the device tree

📖 Related Documentation

For detailed Yocto build instructions: See Chapter 4: Yocto Build Environment to learn how to build and customize these images.

For learning how to flash into emmc/sdcard: Recommend checking out Universal Update Utility PDF

3.3.4 Update Device Tree Only

The device tree describes your hardware configuration. Here's the proper workflow for updating it:

Step 1: Modify and Compile Device Tree on Host PC

```

1 # On your development PC:
2 # Edit your device tree source file (e.g., imx8mp-evk-chyn.dts)
3 $ vim imx8mp-evk-chyn.dts
4
5 # Compile .dts to .dtb using device tree compiler
6 $ dtc -I dts -O dtb -o imx8mp-evk-chyn.dtb imx8mp-evk-chyn.dts
7
8 # Or if you're modifying an existing .dtb, decompile first:
9 $ dtc -I dtb -O dts -o imx8mp-evk.dts imx8mp-evk.dtb
10 # All of these can be done on the board as well!
```

Step 2: Transfer to Board

```

1 # SCP the compiled .dtb file to the board
2 $ scp imx8mp-evk-chyn.dtb root@192.168.1.XXX:/tmp/
3
4 #I have a separate folder for dts files at /opt/Dts-files

```

Step 3: Place in Boot Partition

```

1 # On the board (via SSH or serial console):
2 root@imx8mp-lpddr4-evk:~# cp /tmp/imx8mp-evk-chyn.dtb /run/media/boot-mmcb1k2p1
  /

```

! Important Note

Boot Partition Location: The boot partition is automatically mounted at `/run/media/boot-mmcb1k2p1/`. This is where all `.dtb` files must be placed for U-Boot to find them.

```

1 # Reboot and interrupt U-Boot (press any key during countdown)
2 root@imx8mp-lpddr4-evk:~# reboot
3
4 # In U-Boot console:
5 => printenv fdtfile
6 fdtfile=imx8mp-evk.dtb
7
8 # Verify your new .dtb is present on eMMC boot partition:
9 => ls mmc 2:1
10 # Note: Use 'mmc 1:1' if booting from SD card, 'mmc 2:1' for eMMC
11     4567 Image
12     123 boot.scr
13     45678 imx8mp-evk.dtb
14     45680 imx8mp-evk-chyn.dtb <--- Your new device tree
15
16 # Set U-Boot to use your custom device tree:
17 => setenv fdtfile imx8mp-evk-chyn.dtb
18
19 # Save the environment permanently:
20 => saveenv
21 Saving Environment to MMC... Writing to MMC(2)... OK
22
23 # Boot with new device tree:
24 => boot

```

```

Environment size: 6755/16380 bytes
u-boot=> printenv fdtfile
fdtfile=imx8mp-evk-rpmsg.dtb
u-boot=> setenv fdtfile imx8mp-evk-chyn.dtb
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> boot

```

Step 4: Configure U-Boot to Use New Device Tree

Important Note

Key U-Boot Variable: The `fdtfile` environment variable tells U-Boot which device tree blob to load. After using `setenv`, you **MUST** run `saveenv` to make the change persistent across reboots.

Related Documentation

Device Tree Modification Guide: For detailed information on device tree syntax and how to modify specific peripherals, see CHAPTER 5: DEVICE TREE CONFIGURATION .

3.3.5 Update Rootfs Only

```

1 # This is more complex - usually done by:
2 # 1. Extract rootfs.tar.gz to root partition, OR
3 # 2. Rebuild and flash entire .wic image

```

When you need this: Updated applications, added new libraries, changed configuration files

Important Note

TLDR (Please update if you find a better workflow):

- For bootloader/ATF changes: Update bootloader only
- For kernel/driver changes: Update kernel + DTB
- For application changes: Use SCP to copy files directly
- For complete system updates: Flash .wic image

This saves time by avoiding full reflashes for every change.

3.4 M7 Firmware Loading (Brief Overview)

The Cortex-M7 core is a real-time processor that can run independently of the A53 cores.

3.4.1 M7 Startup Modes

The M7 core can be started in different ways:

1. Remoteproc Framework (Linux controlled)

- M7 is OFF by default after boot
- Linux starts M7 via remoteproc subsystem
- Firmware loaded from `/lib/firmware/`

2. JTAG Debugger (Development mode)

- Load firmware directly via JTAG using VS Code + MCUXpresso
- Bypasses remoteproc entirely
- Used for active development and debugging
- Check section 9.2 for more details

3.4.2 Quick M7 Control Commands

```

1 # Check M7 status
2 root@imx8mpevk:~# cat /sys/class/remoteproc/remoteproc0/state
3 offline # M7 is not running
4
5 # Start M7 core
6 root@imx8mpevk:~# echo start > /sys/class/remoteproc/remoteproc0/state
7
8 # Stop M7 core
9 root@imx8mpevk:~# echo stop > /sys/class/remoteproc/remoteproc0/state
10
11 # Load specific elf file from /lib/firmware
12 root@imx8mpevk:~# echo nuttx-m7-uart4.elf > /sys/class/remoteproc/remoteproc0/
   firmware

```

Related Documentation

Detailed M7 Development Guide: For complete information on M7 firmware development, NuttX RTOS setup, JTAG debugging, VS Code configuration, and peripheral drivers, see **Chapter X: M7 Core Development**.

3.5 Software Stack by Core

Here's what runs on each processing unit:

3.5.1 Cortex-A53 Cores (Application Processors)

- **Operating System:** Yocto Linux (custom build)
- **Kernel Version:** Linux 5.15.x (LTS)
- **Init System:** systemd
- **Key Services:**
 - Zabbix agent (monitoring)
 - SSH daemon
 - Network services
 - Future: Computer vision applications, AI inference

3.5.2 Cortex-M7 Core (Real-Time Processor)

- **Status:** Started manually via remoteproc or JTAG debugger
- **Peripherals Configured:**
 - UART4 (console/debugging)
 - I2C3, I2C5 (sensor communication) , GPIO , SPI etc
 -
- **Not implemented:** PX4 flight controller software integration

3.5.3 NPU (Neural Processing Unit)

- **Framework:** TensorFlow Lite
- **Delegate:** VX Delegate (Vivante optimized)
- **Status:** Benchmarked, ready for inference workloads
- **Performance:** 2.3 TOPS for INT8 quantized models

3.5.4 DSP (Digital Signal Processor)

- **Type:** Cadence Tensilica HiFi4 DSP
- **Status:** Preliminary exploration only
- **Potential Use Cases:** Sensor fusion, audio processing, signal filtering
- **Note:** Not currently utilized - future work item

3.6 Memory Architecture

3.6.1 Physical Memory Map

Address Range	Size	Description
0x00000000 - 0x0001FFFF	128 KB	M7 ITCM (Instruction TCM)
0x20000000 - 0x2001FFFF	128 KB	M7 DTCM (Data TCM)
0x40000000 - 0x5FFFFFFF	512 MB	Peripheral registers
0x40000000	–	DDR Controller
0x30400000	–	I2C1
0x30A20000	–	UART4 (M7 console)
0x80000000 - 0xFFFFFFFF	2 GB	DDR RAM (shared by A53 and M7)

⚠ Important Note

Memory Sharing:

- DDR RAM (starting at 0x80000000) is **shared** between A53 and M7 cores
- TCM (ITCM/DTCM) is **exclusive** to M7 core
- Careful memory allocation required to avoid conflicts
- Resource Domain Controller (RDC) manages access permissions

3.7 Inter-Processor Communication (Planned)

⚠ Important Note

Status: RPMsg framework explored but not fully implemented.

The plan for A53 ↔ M7 communication:

- **Protocol:** RPMsg (Remote Processor Messaging)
- **Transport:** Shared memory in DDR + Mailbox interrupts
- **Use Cases:**
 - A53 sends high-level commands to M7 (e.g., "change flight mode")
 - M7 sends telemetry to A53 (e.g., "current altitude, battery level")
 - Offload non-time-critical processing from M7 to A53

3.8 Resource Domain Controller (RDC)

The RDC prevents conflicts when multiple cores access the same peripherals:

3.8.1 Current RDC Configuration

Peripheral	Owner
UART1, UART2, UART3	A53 (Linux)
UART4	M7 (NuttX console)
I2C1	M7 (sensor access)
I2C2	M7 (sensor access)
I2C3, I2C4	A53 (Linux)
SPI1	M7 (future: IMU)
GPIO1	Shared (careful!)

Important Note

RDC Violation = System Crash

If a core tries to access a peripheral it doesn't own, the system will trigger a bus fault and likely crash. Always verify RDC configuration when adding new peripheral access.

Chapter 4

Yocto Build Environment

This chapter covers everything you need to know about building and customizing the Linux BSP for the i.MX8M Plus using the Yocto Project. We'll start with understanding Yocto's structure, then move to practical build instructions.

Important Note

Current Build Configuration (SD-card):

- **Yocto Release:** Walnascar (6.12)
- **BSP Version:** imx-6.12.20-2.0.0
- **Machine:** imx8mp-lpddr4-evk
- **Distro:** fsl-imx-xwayland
- **Custom Layer:** meta-chyn

Emmc build is using Scarthgap (6.6) and works well.

4.1 Understanding Yocto Architecture

Before diving into building, it's crucial to understand how Yocto is organized. Think of it like this:

4.1.1 The Yocto Hierarchy

1. Layers (meta-*)

- Top-level organizational units
- Each layer is a directory containing recipes, configurations, and patches
- Example: `meta-imx`, `meta-freescale`, `meta-chyn`
- Layers stack on top of each other (like transparent overlays)

2. Recipes (.bb files)

- Instructions for building a specific package

- Example: `imx-atf_2.12.bb` tells Yocto how to build ARM Trusted Firmware
- Contains: source location, dependencies, compile steps, install instructions

3. Recipe Appends (.bbappend files)

- Modifies an existing recipe without editing the original
- Example: `imx-atf_%.bbappend` adds your custom patches to ATF
- The `%` is a wildcard matching any version

4. Packages

- The compiled output of a recipe (binary, libraries, config files)
- Gets installed into the final root filesystem
- Example: `zabbix-agent` package from `zabbix` recipe

4.1.2 Key NXP Layers in Our Build

Layer Name	Purpose
meta-imx	NXP's main i.MX layer - BSP recipes, kernel, bootloader, graphics
meta-freescale	Freescale/NXP common layer - machine configs, u-boot, drivers
poky	Yocto reference distribution - base recipes (gcc, glibc, systemd)
meta-openembedded	Community layer - extra packages (networking, utilities)
meta-chyn	YOUR custom layer - your patches, custom configs, added packages

Important Note

Why create your own layer?

- Keeps your customizations separate from NXP's code
- Makes it easy to upgrade to new BSP versions
- Your changes are organized and version-controlled
- Can be shared with future interns without conflicting with base layers

4.2 Option 1: Using Pre-Built Images (Quick Start)

If you just need to get the board running quickly, use NXP's pre-built images.

4.2.1 Download Pre-Built BSP

1. Go to: <https://www.nxp.com>
2. Search for: i.MX Linux BSP
3. Download the latest release for i.MX8M Plus (e.g., LF_v6.6.52-2.2.0_images_IMX8MPEVK.zip)
4. Extract to a working directory:

```
1 ~/Downloads/LF_v6.6.52-2.2.0_images_IMX8MPEVK/
2
```

The extracted folder contains:

- `imx-image-full-*.wic.zst` - Complete disk image (compressed)
- `imx-boot-*` - Bootloader binary
- `uuu.auto` - Flash script for UUU tool (just a one line code to run)

4.2.2 Install UUU (Universal Update Utility)

UUU is NXP's tool for flashing i.MX boards via USB.

```
1 # Download from GitHub releases:
2 # https://github.com/NXPmicro/mfgtools/releases
3 # For Windows:
4 # Download uuu.exe and place in your working folder
```

4.2.3 Flash to eMMC Using UUU

1. Put the board in **Serial Download Mode**:
 - Set DIP switches (SW1101) to: 0001
 - This tells BootROM to wait for USB download instead of booting from eMMC
2. Connect USB cable from board's USB-C port (serial download port J6) to your PC
3. Run UUU:

```
1 cd ~/Downloads/LF_v6.6.52-2.2.0_images_IMX8MPEVK/
2 uuu uuu.auto
3
```

4. You should see output like:

```
C:\Users\TChyn\Downloads\LF_v6.6.52-2.2.0_images_IMX8MPEVK>uuu uuu.auto
uuu (Universal Update Utility) for nxp imx chips -- libuuu_1.5.220-0-g42f13af

Success 1   Failure 0

1:14-150A900 8/ 8 [Done] FB: done
1:4-150A9000 1/ 1 [=====100%=====] SDPS: boot -f imx-boot-imx8mpevk-sd.bin-flash_evk

C:\Users\TChyn\Downloads\LF_v6.6.52-2.2.0_images_IMX8MPEVK>
```

Figure 4.1: UUU output showing progress and success

4.3 Option 2: Building from Source (Recommended for Development)

Building from source gives you full control and allows customization. This is what you'll need for adding custom packages like Zabbix or modifying ATF.

4.3.1 Prerequisites (follow the documentation)

When running WSL2/VM, ensure you have proper setup:

```
1 # Install required packages
2 sudo apt-get update
3 sudo apt-get install build-essential chrpath cpio debianutils diffstat \
4   file gawk gcc git iputils-ping libacl1 liblz4-tool locales \
5   python3 python3-git python3-jinja2 python3-pexpect python3-pip \
6   python3-subunit socat texinfo unzip wget xz-utils zstd efiteols
```

Related Documentation

Documentation Reference: i.MX Yocto Project User's Guide - it has all the information you need to get started with building your custom yocto image. Please use this

Important Note

Disk Space Warning: A full imx-image-full build requires **minimum 550 GB** of free disk space and takes 24-36 hours on my PC with 32 gb of RAM.

4.3.2 Install repo Utility

```
1 # Install curl if not already present
2 sudo apt-get install curl
3
4 # Create bin directory and download repo tool
5 mkdir -p ~/bin
6 curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo
7 chmod a+x ~/bin/repo
8
9 # Add to PATH (add to ~/.bashrc to make permanent)
10 export PATH=~/bin:$PATH
```

4.3.3 Download i.MX Yocto BSP Layers

```
1 # Create workspace
2 mkdir ~/imx-yocto-bsp
3 cd ~/imx-yocto-bsp
4
5 # Initialize repo with NXP manifest
6 repo init -u https://github.com/nxp-imx/imx-manifest \
7   -b imx-linux-walnascar \
8   -m imx-6.12.20-2.0.0.xml
9
10 # When prompted, configure git:
11 # Name: Your Name
12 # Email: your.email@example.com
13
14 # Download all layers (this takes several minutes)
15 repo sync
```

After `repo sync` completes, you'll have multiple layer directories:

```
1 ~/imx-yocto-bsp/
2 sources/
3   meta-imx/           # NXP i.MX BSP layer
4   meta-freescale/     # Freescale layer
5   meta-openembedded/  # Community packages
6   poky/               # Yocto reference distro
7   ...more layers...
8   (build directories will be created here)
9
10 #at this stage, the following steps might differ in the future. Please take
    reference from the imx yocto user guide PDF!
```

4.3.4 Initialize Yocto Build Environment

```
1 cd ~/imx-yocto-bsp
2
3 # Initialize build environment
4 DISTRO=fsl-imx-xwayland MACHINE=imx8mp-lpddr4-evk \
5   source imx-setup-release.sh -b build-xwayland
```

What this does:

- Creates `build-xwayland/` directory with configuration
- Sets up environment variables for BitBake
- Generates `conf/local.conf` and `conf/bblayers.conf`

⚠ Important Note

DISTRO Options Explained:

- `fsl-imx-wayland` - Pure Wayland graphics (no X11 compatibility)
- `fsl-imx-xwayland` - Wayland + XWayland
- `fsl-imx-fb` - Framebuffer only (no Wayland/X11)

What is X11 vs Wayland?

- **X11** - Older Linux display system, supports legacy desktop apps
- **Wayland** - Modern, faster, more secure display protocol
- **XWayland** - Compatibility layer that runs X11 apps on Wayland

For i.MX8MP, use `fsl-imx-xwayland` for best compatibility with both old and new applications. In my experience, its not a lot of new packages downloaded, so leaving it out is also fine

After initialization, you'll see the EULA agreement:

```
Do you accept the EULA you just read? (y/n) y
EULA has been accepted.

Welcome to Freescale Community BSP

The Yocto Project has extensive documentation about OE including a
reference manual which can be found at:
  http://yoctoproject.org/documentation

For more information about OpenEmbedded see their website:
  http://www.openembedded.org/

You can now run 'bitbake <target>'

Common targets are:
  core-image-minimal
  meta-toolchain
  meta-toolchain-sdk
  adt-installer
  meta-ide-support

Your build environment has been configured with:

MACHINE=imx8mp-lpddr4-evk
SDKMACHINE=i686
DISTRO=fsl-imx-xwayland
```

Figure 4.2: EULA agreement, with build instructions

4.3.5 Choose Image Type

Yocto provides several image recipes depending on your needs:

Image Recipe	Description
core-image-minimal	Absolute minimum - boots to shell, nothing else
core-image-base	Console-only with full hardware support
imx-image-core	i.MX test applications for Wayland
imx-image-multimedia	GUI support, multimedia codecs, no Qt
imx-image-full	Everything - Qt6, ML tools, multimedia (what I did)

⚠ Important Note

Build Time:

- imx-image-core - 2-4 hours, 100 GB (unsure)
- imx-image-full - 24-36 hours, 550 GB

4.3.6 Build the Image

```

1 # Make sure you're in the build directory (build-xwayland or whatever you named
   it)
2 cd ~/imx-yocto-bsp/build-xwayland
3
4 # Start the build (this will take hours)
5 bitbake imx-image-full

```

BitBake will:

1. Download all source code (Linux kernel, U-Boot, ATF, packages)
2. Compile everything from scratch
3. Generate root filesystem
4. Package everything into deployable images

4.3.7 Optimize Build Performance

One of the small minor problems i faced was my pc crashing in the middle of builds. Edit `build-xwayland/conf/local.conf` to limit parallel jobs if your machine struggles:

```

1 nano conf/local.conf
2
3 # Add these lines (adjust numbers based on your CPU):
4 BB_NUMBER_THREADS = "5"           # BitBake parallel tasks
5 PARALLEL_MAKE = "-j5"             # Make parallel compilation

```

Guidelines:

- BB_NUMBER_THREADS - Number of recipes BitBake builds in parallel
- PARALLEL_MAKE - Number of compiler threads per recipe

4.3.8 Troubleshooting Build Errors

If a build fails, you can clean and rebuild specific packages:

```
1 # Clean a specific package (removes all build artifacts)
2 bitbake -c cleanall <package-name>
3
4 # Rebuild just that package to see detailed errors
5 bitbake <package-name>
```

Common error scenarios:

- **Fetch errors** - Network issues, check internet connection
- **Compilation errors** - Missing dependencies, check package logs
- **Disk space errors** - Build partition full, free up space

4.3.9 Build Output Location

After successful build, your images are in:

```
1 ~/imx-yocto-bsp/build-xwayland/tmp/deploy/images/imx8mp-lpddr4-evk/
2
3 # Key files:
4 imx-image-full-imx8mp-lpddr4-evk.wic.zst      # Complete disk image (compressed)
5 imx-image-full-imx8mp-lpddr4-evk.rootfs.tar.gz # Root filesystem
6 imx-boot-imx8mp-lpddr4-evk-sd.bin-flash_evk   # Bootloader
```

4.4 Adding Custom Packages (Zabbix Example)

One of the key advantages of building from source is adding custom packages. Before adding any package, you need to verify it exists in your Yocto layers and is compatible with your BSP version.

4.4.1 Step 1: Search OpenEmbedded Layer Index

Before modifying your build configuration, always check if the package exists and which layer provides it:

1. Go to: <https://layers.openembedded.org/layerindex/branch/master/layers/>
2. Select your Yocto branch from the dropdown (e.g., **walnascar** for our build)
3. Search for your package (e.g., "zabbix")

From the search results, you can see:

- **Name:** zabbix
- **Version:** 7.0.9
- **Layer:** meta-oe (walnascar branch)

walnascar → meta-oe → zabbix

zabbix 7.0.9

Name	zabbix
Version	7.0.9
Summary	Open-source monitoring solution for your IT infrastructure
Description	ZABBIX is software that monitors numerous parameters of a network and the health and integrity of servers. ZABBIX uses a flexible notification mechanism that allows users to configure e-mail based alerts for virtually any event. This allows a fast reaction to server problems. ZABBIX offers excellent reporting and data visualisation features based on the stored data. This makes ZABBIX ideal for capacity planning. ZABBIX supports both polling and trapping. All ZABBIX reports and statistics, as well as configuration parameters are accessed through a web-based front end. A web-based front end ensures that the status of your network and the health of your servers can be assessed from any location. Properly configured, ZABBIX can play an important role in monitoring IT infrastructure. This is equally true for small organisations with a few servers and for large companies with a multitude of servers.
Section	Applications/Internet
License	AGPL-3.0-only
Homepage	http://www.zabbix.com/
Recipe file	recipes-connectivity/zabbix/zabbix_7.0.9.bb
Layer	meta-oe (walnascar branch)

Figure 4.3: Zabbix Package in OpenEmbedded Layer Index

- **Recipe file:** `recipes-connectivity/zabbix/zabbix_7.0.9.bb`
- **Summary:** Open-source monitoring solution for IT infrastructure

! Important Note

Key Information to Check:

1. **Layer name** - Which layer provides this package?
2. **Branch compatibility** - Is it available in your Yocto version (walnascar)?
3. **Recipe location** - Helps you find it if you need to inspect or modify

4.4.2 Step 2: Verify Layer is Already Included

Check if the required layer is already part of your build:

```

1 cd ~/imx-yocto-bsp/build-xwayland
2
3 # List all layers in your build
4 bitbake-layers show-layers

```

You should see output like:

```

1 layer                path                                priority
2 =====
3 meta                 /home/user/imx-yocto-bsp/sources/poky/meta  5
4 meta-poky           /home/user/.../poky/meta-poky             5
5 meta-oe              /home/user/.../meta-openembedded/meta-oe   6  <-- HERE!
6 meta-python         /home/user/.../meta-openembedded/meta-python 7
7 meta-networking      /home/user/.../meta-openembedded/meta-networking 5
8 meta-imx             /home/user/.../meta-imx                   8
9 ...

```

- **Layer exists** - Zabbix is in meta-oe, which is already included in the NXP BSP manifest

- **Compatible branch** - walnascar branch has zabbix 7.0.9
- **Ready to use** - Can add directly to IMAGE_INSTALL

4.4.3 Step 3: Add Package to Image

Since meta-oe is already in our build and zabbix exists in the walnascar branch, we can simply add it:

```
1 nano ~/imx-yocto-bsp/build-xwayland/conf/local.conf
2
3 # Add at the end of the file:
4 IMAGE_INSTALL:append = " zabbix zabbix-agent"
```

4.4.4 Step 4: Rebuild Image

```
1 bitbake imx-image-full
```

BitBake will:

1. Fetch zabbix source code (version 7.0.9)
2. Compile zabbix and zabbix-agent
3. Install into root filesystem
4. Repackage the image

4.4.5 What If the Package Doesn't Exist?

If you search for a package and it's **not found** or **not in any layer you have**, you have three options:

Option 1: Add the Required Layer

If the package exists in a layer not included in your build:

```
1 # Example: Package is in meta-networking (usually already included)
2 cd ~/imx-yocto-bsp/build-xwayland
3
4 # Add the layer
5 bitbake-layers add-layer ../sources/meta-openembedded/meta-networking
6
7 # Verify it was added
8 bitbake-layers show-layers | grep networking
```

Option 2: Create Your Own Recipe

If the package doesn't exist in any OpenEmbedded layer, you'll need to create a recipe in your custom layer:

```

1 # Create recipe in meta-chyn
2 mkdir -p ~/imx-yocto-bsp/sources/meta-chyn/recipes-custom/mypackage
3 cd ~/imx-yocto-bsp/sources/meta-chyn/recipes-custom/mypackage
4
5 # Create recipe file
6 vim mypackage_1.0.bb

```

Basic recipe template (refer to other template for better visualisation):

```

1 SUMMARY = "My custom package"
2 DESCRIPTION = "Description of what this does"
3 LICENSE = "MIT"
4 LIC_FILES_CHKSUM = "file://LICENSE;md5=..."
5
6 SRC_URI = "https://example.com/mypackage-1.0.tar.gz"
7 SRC_URI[sha256sum] = "abc123..."
8
9 inherit autotools
10 # or: inherit cmake
11 # or: inherit setuptools (for Python)
12
13 do_install() {
14     install -d ${D}${bindir}
15     install -m 0755 myprogram ${D}${bindir}/
16 }

```

Option 3: Install at Runtime

For quick testing, you can install packages directly on the running system:

```

1
2
3 # Use opkg if available (but opkg needs to be set up)
4 opkg update
5 opkg install <package-name>
6
7 # Or compile from source manually on the board using git/wget

```

Important Note

Best Practice: Always use the Layer Index first. Creating custom recipes should be a last resort - most packages you need already exist in OpenEmbedded layers.

4.5 Creating Your Custom Layer

For more complex customizations (patches, custom configs), create your own layer.

4.5.1 Create meta-chyn Layer

```
1 cd ~/imx-yocto-bsp/build-xwayland
2
3 # Create layer structure
4 bitbake-layers create-layer ../sources/meta-chyn
5
6 # Add layer to build
7 bitbake-layers add-layer ../sources/meta-chyn
8
9 # Verify layer is added
10 bitbake-layers show-layers
```

Your layer structure:

```
1 sources/meta-chyn/
2   conf/
3     layer.conf           # Layer configuration
4   recipes-example/
5     example/
6       example_0.1.bb     # Example recipe
7   README
```

4.6 Creating patches for ARM Trusted Firmware

This section shows how to modify ATF to add custom power management features. This is how the low-power suspend mode optimizations were implemented.

4.6.1 Goal

We want to:

- Keep M7 core alive during Linux suspend (using LPA flags)
- Add A53→M7 notification helpers via GIR (General Interrupt Register)
- Package changes properly using Yocto workflow

4.6.2 Use devtool to Modify ATF

devtool is Yocto's development workflow tool - it sets up a workspace for editing source code.

```
1 cd ~/imx-yocto-bsp/build-xwayland
2
3 # Extract ATF source to workspace
4 devtool modify imx-atf
5
6 # Navigate to ATF source
7 cd workspace/sources/imx-atf
```

Now you have the full ATF source code ready for editing in `workspace/sources/imx-atf/`.

4.6.3 Make Your Changes

Edit the file where power management happens:

```
1 nano plat/imx/imx8m/gpc_common.c
```

Add GIR helper functions:

```
1 #include <platform_def.h>
2 #ifndef IMX_MU_BASE
3 #define IMX_MU_BASE U(0x30AA0000) /* A-side MU base on i.MX8MP */
4 #endif
5
6 /* Helper: Notify M7 via GIRO doorbell */
7 static void imx_notify_m4_set_db(void)
8 {
9     /* Set GIRO bit and enable */
10    mmio_setbits_32(IMX_MU_BASE + 0x24, BIT(19) | BIT(31));
11 }
12
13 /* Helper: Clear GIPO after M7 processes notification */
14 static void imx_notify_m4_clear_db(void)
15 {
16    mmio_setbits_32(IMX_MU_BASE + 0x20, BIT(31));
17 }
```

Hook into suspend/resume path:

Find the `imx_set_sys_wakeup()` function and add:

```
1 /* Enable MU wakeup */
2 if (imx_is_m4_enabled()) {
3     mmio_clrbits_32(IMX_GPC_BASE + gpc_imr_offset[last_core] + 0x8, BIT(24));
4
5     if (imx_m4_lpa_active()) {
6         if (pdn) {
7             /* Entering low power - notify M7 */
8             imx_notify_m4_set_db();
9         } else {
10            /* Exiting low power - clear notification */
11            imx_notify_m4_clear_db();
12        }
13    }
14 }
```

4.6.4 Commit Your Changes

```
1 cd ~/imx-yocto-bsp/build-xwayland/workspace/sources/imx-atf
2
3 # Stage and commit your changes
4 git add plat/imx/imx8m/gpc_common.c
5 git commit -m "imx8mp: add GIR helpers for A53-M7 suspend coordination"
```

4.6.5 Handle Build Artifacts (if present)

Sometimes devtool workspace has build artifacts that shouldn't be committed:

```
1 # Ignore build directories
2 echo "build-optee/" >> .gitignore
3 git add .gitignore
4 git commit -m "Ignore build-optee artifacts"
```

4.6.6 Finish devtool and Create Patch

This command generates a patch and creates a bbappend file in your layer:

```
1 cd ~/imx-yocto-bsp/build-xwayland
2
3 # Finish devtool session - generates patch in meta-chyn
4 devtool finish imx-atf ../sources/meta-chyn
```

This creates:

```
1 sources/meta-chyn/recipes-bsp/imx-atf/
2   imx-atf_%.bbappend           # Recipe append
3   imx-atf/
4     0001-imx8mp-add-GIR-helpers-for-suspend.patch
5     0002-Ignore-build-optee-artifacts.patch
```

4.6.7 Add Required Patch Headers

Yocto QA requires patches to have proper headers. Add Upstream-Status tag:

```
1 cd ~/imx-yocto-bsp/sources/meta-chyn/recipes-bsp/imx-atf/imx-atf/
2
3 # Edit patch file manually or use sed:
4 sed -i '/^Subject:/a Upstream-Status: Inappropriate [i.MX8MP BSP-specific]\
5     nSigned-off-by: Your Name <you@example.com>' \
6     0001-imx8mp-add-GIR-helpers-for-suspend.patch
```

The patch should now have headers like:

```
1 From abc123def456...
2 Subject: [PATCH] imx8mp: add GIR helpers for A53-M7 suspend coordination
3 Upstream-Status: Inappropriate [i.MX8MP BSP-specific low-power behavior]
4 Signed-off-by: Your Name <you@example.com>
5 ---
6 plat/imx/imx8m/gpc_common.c | 20 +++++
7 1 file changed, 20 insertions(+)
```

4.6.8 Rebuild ATF with Patches

```
1 cd ~/imx-yocto-bsp/build-xwayland
2
3 # Clean ATF and imx-boot to force rebuild
```

```

4 bitbake -c cleansstate imx-atf imx-boot
5
6 # Rebuild with patches applied
7 bitbake imx-boot

```

4.6.9 Verify Patches Were Applied

```

1 # Check patch log
2 less tmp/work/*/imx-atf/*/temp/log.do_patch
3 # Look for "Applying patch 0001-..." messages
4
5 # Verify your code is in the source
6 grep -n "imx_notify_m4_set_db" \
7     tmp/work/*/imx-atf/*/git/plat/imx/imx8m/gpc_common.c

```

4.6.10 Extract and Flash Updated Bootloader

```

1 # New imx-boot is in:
2 cd ~/imx-yocto-bsp/build-xwayland/tmp/deploy/images/imx8mp-lpddr4-evk/
3
4 # Flash to SD card (example):
5 sudo dd if=imx-boot-imx8mp-lpddr4-evk-sd.bin-flash_evk \
6         of=/dev/sdX bs=1K seek=33 conv=fsync
7
8 # Or for eMMC via UUU - see Chapter 2 for details

```

4.7 Repository Directory Structure (for Reference)

Understanding where code lives in Linux kernel and ATF repositories:

4.7.1 Linux Kernel Structure

```

linux/
|-- arch/                                # Architecture-specific code
|   |-- arm64/                           # A53 cores live here
|       |-- boot/dts/                   # Device trees
|           freescale/                 # i.MX8MP device trees here!
|       |-- kernel/                     # Low-level ARM64 code
|           mm/                         # Memory management
|       arm/                            # 32-bit ARM (not used on A53)
|
|-- drivers/                             # Hardware drivers
|   |-- i2c/                            # I2C subsystem
|   |-- spi/                            # SPI subsystem
|   |-- gpu/                            # Graphics drivers
|   soc/imx/                            # i.MX-specific drivers

```

```
include/  
    dt-bindings/          # Device tree constants
```

4.7.2 ARM Trusted Firmware Structure

```
arm-trusted-firmware/  
|-- plat/                # Platform-specific code  
    imx/                  # NXP i.MX platforms  
        |-- imx8m/        # i.MX8M family  
            |-- imx8mp/    # i.MX8M Plus specific  
                |-- gpc.c  # Power controller (I modified this!)  
                    imx8mp_bl31_setup.c # Boot setup  
                gpc_common.c # Common GPC code  
            common/        # Common i.MX code  
  
include/  
    plat/imx/             # i.MX headers
```

Related Documentation

For detailed suspend implementation: See AN13400 i.MX 8M Low Power Design
By M Core Running In System

Chapter 5

Device Tree Configuration

Device tree is the foundation of hardware configuration in embedded Linux. This chapter covers everything from basic syntax to practical examples of adding sensors to the i.MX8M Plus.

5.1 What is Device Tree?

5.1.1 Purpose and Overview

Device tree is a data structure that describes hardware to the Linux kernel. Instead of hardcoding hardware information in the kernel, device tree provides a flexible, human-readable way to tell Linux:

- What hardware exists on the board
- Where peripherals are located (memory addresses, I2C addresses, etc.)
- How to configure them (clock frequencies, pin assignments, power supplies)
- How devices are connected (which GPIO pins, which I2C bus, etc.)

Important Note

Key Concept: Device tree does NOT contain drivers. It only describes hardware. The Linux kernel matches device tree nodes to appropriate drivers using the `compatible` property.

5.1.2 File Types

Extension	Type	Description
.dts	Device Tree Source	Human-readable text file you edit
.dtsi	Device Tree Source Include	Reusable chunks included in .dts files
.dtb	Device Tree Blob	Compiled binary that U-Boot loads
.dtbo	Device Tree Blob Overlay	Binary overlay applied at runtime. It only replaces certain portion of a targeted device tree

Typical structure for i.MX8M Plus:

```

imx8mp.dtsi          <- SoC-level definitions (CPU, memory map)
|
+-- imx8mp-evk.dtsi   <- EVK board common features
|
+-- imx8mp-evk.dts    <- Base board configuration
+-- imx8mp-evk-rpmsg.dts <- Board with RPMsg enabled
+-- imx8mp-evk-chyn.dts <- YOUR custom configuration

```

Related Documentation**Additional Resources:**

- **Device Tree Syntax:** NXP Application Note AN5125 "Introduction to Device Trees" - comprehensive syntactical reference
- **Device Tree Overlays:** <https://community.nxp.com/t5/i-MX-Processors-Knowledge-Base/How-to-use-linux-device-tree-overlay-from-u-boot-and-apply-it/ta-p/1896858>

5.2 Checking Your Current Configuration

5.2.1 Which DTB is Currently Loaded?

From U-Boot

```

1 # Stop autoboot and check environment variable
2 u-boot=> printenv fdtfile
3 fdtfile=imx8mp-evk-rpmsg.dtb
4
5 # Check where device tree is loaded in memory
6 u-boot=> printenv fdt_addr
7 fdt_addr=0x43000000

```

5.2.2 Available DTB Files

From U-Boot - List eMMC boot partition:

```

1 u-boot=> ls mmc 2:1
2 35631616 Image
3 84282 imx8mp-evk.dtb
4 65300 imx8mp-evk-rpmsg.dtb
5 84266 imx8mp-evk-flexcan2.dtb
6 # ... more files

```

⚠ Important Note**Boot Partition Location:**

- **From U-Boot:** mmc 2:1 (eMMC) or mmc 1:1 (SD card)
- **From Linux:** /run/media/boot-mmcb1k2p1/ (auto-mounted boot partition)

To check which is which:

```
1 u-boot=> mmc list
2 FSL_SDHC: 1
3 FSL_SDHC: 2 (eMMC)
```

🔗 GitHub Repository

All DTS files can be found in:

<https://github.com/yulyuri/firmware-and-dts/blob/master/SCP%20files>

5.3 Device Tree Compiler (dtc)

The device tree compiler (dtc) converts between human-readable (.dts) and binary (.dtb) formats.

5.3.1 Essential dtc Flags

Flag	Purpose
-I	Input format: dts, dtb, fs (filesystem), yaml
-O	Output format: dts, dtb, yaml
-o	Output file path
-@	Keep/generate symbols (needed for overlays and readable labels)
-s	Sort nodes alphabetically (useful for comparing trees)

5.3.2 Common dtc Operations

Decompile DTB to readable DTS:

```
1 # Decompile a single DTB file
2 dtc -I dtb -O dts -o /tmp/imx8mp-evk.dts \
3     /run/media/boot-mmcb1k2p1/imx8mp-evk.dtb
4
5 # Decompile ALL DTB files at once (very useful!)
6 for f in /run/media/boot-mmcb1k2p1/*.dtb; do
7     dtc -s -I dtb -O dts -o "/tmp/${basename "${f%.dtb}").dts" "$f"
8 done
```

Compile DTS back to DTB:

```

1 # Basic compilation
2 dtc -I dts -O dtb -o /tmp/imx8mp-evk-test.dtb /tmp/imx8mp-evk.dts
3
4 # With symbol preservation (recommended)
5 dtc -@ -I dts -O dtb -o /tmp/imx8mp-evk-chyn.dtb /tmp/imx8mp-evk-chyn.dts

```

5.3.3 Comparing Device Trees

You can use `dtc` with the `-s` (sort) flag to compare two device tree configurations:

```

1 # Decompile both with sorting
2 dtc -s -I dtb -O dts -o /tmp/rpmsg.dts \
3     /run/media/boot-mmcbk2p1/imx8mp-evk-rpmsg.dts
4
5 dtc -s -I dtb -O dts -o /tmp/rpmsg-lpv.dts \
6     /run/media/boot-mmcbk2p1/imx8mp-evk-rpmsg-lpv.dts
7
8 # Compare to see differences
9 diff /tmp/rpmsg.dts /tmp/rpmsg-lpv.dts | less

```

Example finding: Comparing `rpmsg.dtb` vs `rpmsg-lpv.dtb` shows that LPV (Low Power Voice) removes:

- `fsl,enable-lpa` property
- `fsl,rpmsg-in` node

This tells you how LPV is implemented at the hardware level!

5.4 Device Tree Syntax Fundamentals

5.4.1 Nodes and Hierarchy

Device tree is organized as a tree of nodes:

```

1 / {                                     // Root node
2     compatible = "fsl,imx8mp-evk", "fsl,imx8mp";
3
4     soc@0 {                             // SoC container node
5         i2c@30ad0000 {                  // I2C5 controller node
6             compatible = "fsl,imx8mp-i2c";
7             reg = <0x30ad0000 0x10000>;
8
9             sensor@68 {                 // Device on I2C bus
10                compatible = "invensense,mpu6050";
11                reg = <0x68>;           // I2C address
12            };
13        };
14    };
15 };

```

Node naming convention: name@unit-address

- name - Descriptive name (i2c, sensor, gpio, etc.)
- @unit-address - Device address (MMIO base, I2C address, etc.)

5.4.2 Properties

Properties are key-value pairs that describe node characteristics:

```

1 sensor@68 {
2     // String property
3     compatible = "invensense,mpu6050";
4
5     // Integer property (32-bit cells)
6     reg = <0x68>;
7
8     // Multiple strings
9     pinctrl-names = "default", "sleep";
10
11    // Boolean flag (presence = true)
12    regulator-always-on;
13
14    // Reference to another node (phandle)
15    vdd-supply = <&buck2>;
16    interrupt-parent = <&gpio1>;
17
18    // Array of integers
19    clocks = <&clk IMX8MP_CLK_I2C5_ROOT>;
20 };

```

5.4.3 Labels and Phandles

Labels let you reference nodes from anywhere in the device tree:

```

1 // Define a label
2 i2c5: i2c@30ad0000 {
3     compatible = "fsl,imx8mp-i2c";
4     // ... properties
5 };
6
7 // Reference it later
8 &i2c5 {
9     clock-frequency = <100000>; // Override/add properties
10
11    my_sensor@68 {
12        // Add child node
13    };
14 };
15
16 // Or use as phandle in properties
17 sensor {

```

```

18     i2c-bus = <&i2c5>;
19 };

```

5.4.4 Interrupts

Interrupt specification format is defined by the interrupt controller:

```

1 // ARM GIC (most common on i.MX8MP)
2 sensor@68 {
3     interrupt-parent = <&gic>;
4     interrupts = <GIC_SPI 76 IRQ_TYPE_LEVEL_HIGH>;
5     // Which expands to: <0 76 4>
6     // 0 = SPI (Shared Peripheral Interrupt)
7     // 76 = interrupt number
8     // 4 = level-high trigger
9 };
10
11 // GPIO interrupts
12 sensor@68 {
13     interrupt-parent = <&gpio1>;
14     interrupts = <12 IRQ_TYPE_EDGE_FALLING>;
15 };

```

5.4.5 Pin Control (i.MX Specific)

Pin muxing is handled by pinctrl groups:

```

1 // Define pin group in IOMUXC node
2 &iomuxc {
3     pinctrl_i2c5: i2c5grp {
4         fsl,pins = <
5             // MUX_REG PAD_REG SELECT_INPUT ALT DAISY PAD_CTL
6             MX8MP_IOMUXC_SPDIF_TX__I2C5_SCL 0x400001c3
7             MX8MP_IOMUXC_SPDIF_RX__I2C5_SDA 0x400001c3
8         >;
9     };
10 };
11
12 // Use pin group in device node
13 &i2c5 {
14     pinctrl-names = "default";
15     pinctrl-0 = <&pinctrl_i2c5>;
16 };

```

The macros expand to 6 values:

- MUX register offset
- PAD control register offset
- SELECT_INPUT (daisy chain) register offset

- ALT function number (which peripheral uses this pin)
- Daisy chain value
- PAD control value (drive strength, pull-up/down, etc.)

These are defined in `arch/arm64/boot/dts/freescale/imx8mp-pinfunc.h`.

5.5 Complete Workflow: Adding an IMU Sensor

Let's walk through a real example: adding the MPU-6050/6500 IMU to I2C5 on connector J22.

GitHub Repository

Complete Working Example:

File: `imx8mp-evk-chyn-m7.dts`

Lines: 1787-1808 (MPU-6050 sensor configuration)

<https://github.com/yulyuri/firmware-and-dts/blob/master/SCP%20files/imx8mp-evk-chyn-m7.dts>

This file contains the complete I2C5 controller setup with MPU-6050 sensor node, including pin muxing and power supply configuration.

5.5.1 Step 1: Identify Hardware Connections

Table 7. J22 pin definition

Num.	Net name	Description
1/2	VEXT_3V3	Power Output, 3.3 V
3/4	I2C5_SCL_3V3	I2C clock signal
5/6	I2C5_SDA_3V3	I2C data signal
7/8	GND	Ground

Figure 5.1: i2c header pinouts

Check the i.MX8MP EVK Hardware User Guide for J22 pinout:

- J22 is connected to I2C5 lines (SPDIF_TX and SPDIF_RX pins muxed to I2C)
- SoC I2C5 base address: `0x30AD0000`
- Linux typically enumerates this as `i2c-4` (check with `ls /sys/bus/i2c/devices/`)

Verify I2C mapping:

```

1 root@imx8mp-lpddr4-evk:~# cat /sys/bus/i2c/devices/i2c-4/name
2
3 # Cross-reference with i.MX8MP Reference Manual:
4 # 0x30AD0000 = I2C5

```

5.5.2 Step 2: Find and Extract Base Device Tree

```

1 # Decompile the base DTB you want to modify
2 dtc -I dtb -O dts -o /tmp/imx8mp-evk-rpmsg.dts \
3     /run/media/boot-mmcbblk2p1/imx8mp-evk-rpmsg.dtb
4
5 # Copy to working directory for editing or scp to host pc/laptop for editing
6 cp /tmp/imx8mp-evk-rpmsg.dts /tmp/imx8mp-evk-chyn.dts

```

5.5.3 Step 3: Locate I2C5 Controller Node

Search for the I2C5 controller in your DTS file:

```

1 i2c@30ad0000 {                                /* I2C5 controller */
2     compatible = "fsl,imx8mp-i2c", "fsl,imx21-i2c";
3     #address-cells = <0x01>;
4     #size-cells = <0x00>;
5     reg = <0x30ad0000 0x10000>;                /* Base address + size */
6     interrupts = <0x00 0x4c 0x04>;            /* GIC interrupt spec */
7     clocks = <0x02 0xe7>;                     /* Clock: &clk index 0xE7 (231) */
8     status = "okay";
9     clock-frequency = <0x186a0>;              /* 100 kHz */
10    pinctrl-names = "default";
11    pinctrl-0 = <0x46>;                       /* Refers to pinctrl_i2c5 */
12    pinctrl-assert-gpios = <0x31 0x02 0x02>; /* Board mux control */
13 };

```

Understanding the fields:

- `clocks = <0x02 0xe7>`: 0x02 is phandle to CCM, 0xE7 (231) = IMX8MP_CLK_I2C5_ROOT
- `clock-frequency`: Target I2C bus speed (100 kHz here)
- `pinctrl-assert-gpios`: GPIO that must be driven to route pins to I2C5 (board mux)

5.5.4 Step 4: Verify/Add Pin Mux Configuration

Check if I2C5 pin group exists:

```

1 &iomuxc {
2     pinctrl_i2c5: i2c5grp {
3         fsl,pins = <
4             // Maps SPDIF pins to I2C5 function
5             MX8MP_IOMUXC_SPDIF_TX__I2C5_SCL 0x400001c3
6             MX8MP_IOMUXC_SPDIF_RX__I2C5_SDA 0x400001c3
7         >;
8     };
9 };

```

```

635 #define MX8MP_IOMUXC_SPDIF_RX_AUDIOMIX_SPDIF1_IN      0x1D8 0x438 0x544 0x0 0x4
636 #define MX8MP_IOMUXC_SPDIF_RX_PWM2_OUT                0x1D8 0x438 0x000 0x1 0x0
637 #define MX8MP_IOMUXC_SPDIF_RX_I2C5_SDA                0x1D8 0x438 0x5C8 0x2 0x2
638 #define MX8MP_IOMUXC_SPDIF_RX_GPT1_COMPARE2           0x1D8 0x438 0x000 0x3 0x0
639 #define MX8MP_IOMUXC_SPDIF_RX_CAN1_RX                 0x1D8 0x438 0x54C 0x4 0x2
640 #define MX8MP_IOMUXC_SPDIF_RX_GPIO5_IO04              0x1D8 0x438 0x000 0x5 0x0

```

Figure 5.2: mx8mp-pinfunc.h

The 0x400001c3 pad control value sets:

- Open-drain output (required for I2C)
- Pull-up enabled
- Hysteresis enabled
- Moderate drive strength

5.5.5 Step 5: Add MPU-6050 Sensor Node

Add the sensor as a child of the I2C5 controller:

```

1 &i2c5 {
2     status = "okay";
3     clock-frequency = <100000>; // 100 kHz
4
5     mpu6050@68 {
6         compatible = "invensense,mpu6050";
7         reg = <0x68>; // I2C address (ADO = LOW)
8         // Use 0x69 if ADO pin is HIGH
9
10        status = "okay";
11    };
12 };
13

```

! Important Note

Where do you find node schema?

Check the kernel binding documentation:

- Documentation/devicetree/bindings/iio/imu/invensense,mpu6050.yaml

In general, from what I've learnt yaml files lists all required and optional properties with examples. Just simply google [part you are looking for], yaml

5.5.6 Step 6: Compile Modified Device Tree

```

1 # Compile DTS to DTB (with symbol preservation)
2 dtc -@ -I dts -O dtb \
3     -o /tmp/imx8mp-evk-chyn.dtb \
4     /tmp/imx8mp-evk-chyn.dts
5

```

```

6  # Copy to boot partition
7  cp /tmp/imx8mp-evk-chyn.dtb /run/media/boot-mmcbk2p1/
8
9  # Sync to ensure it's written
10 sync
11
12 # Verify it exists
13 ls -lh /run/media/boot-mmcbk2p1/imx8mp-evk-chyn.dtb

```

5.5.7 Step 7: Configure U-Boot to Use New DTB

```

1  # Reboot and interrupt U-Boot
2  reboot
3
4  # In U-Boot console:
5  u-boot=> ls mmc 2:1
6  # Verify your .dtb file is present
7
8  u-boot=> setenv fdtfile imx8mp-evk-chyn.dtb
9  u-boot=> saveenv
10 Saving Environment to MMC... Writing to MMC(2)... OK
11
12 u-boot=> boot

```

Important Note

Critical Step: You MUST run `saveenv` to make the change permanent. Without it, the setting is lost on next reboot.

5.5.8 Step 8: Verify Hardware Configuration

After Linux boots, verify everything is configured correctly:

A) Check device tree is loaded:

```

1  # Check I2C5 status
2  cat /proc/device-tree/soc@0/bus@30800000/i2c@30ad0000/status
3  okay
4
5  # Check sensor node exists
6  cat /proc/device-tree/soc@0/bus@30800000/i2c@30ad0000/mpu6050@68/compatible
7  invensense,mpu6050

```

B) Verify pin muxing:

```

1  grep -E 'SPDIF_(TX|RX)|i2c5' /sys/kernel/debug/pinctrl/*/pinmux-pins
2
3  # Expected output:
4  pin 117 (MX8MP_IOMUXC_SPDIF_TX): 30ad0000.i2c (GPIO UNCLAIMED) function pinctrl
   group i2c5grp
5  pin 118 (MX8MP_IOMUXC_SPDIF_RX): 30ad0000.i2c (GPIO UNCLAIMED) function pinctrl
   group i2c5grp

```

C) Check clocks are enabled:

```

1 cat /sys/kernel/debug/clk/clk_summary | grep -i i2c5
2
3 # Expected output:
4 i2c5          0      0      0      24000000    0
5   i2c5_root_clk 0      0      0      24000000    0

```

D) Scan I2C bus for device:

```

1 i2cdetect -y -r 4
2
3 # Expected output showing device at 0x68:
4   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
5 00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
6 10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
7 20:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
8 30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
9 40:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10 50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
11 60:  --  --  --  --  --  --  --  68  --  --  --  --  --  --  --
12 70:  --  --  --  --  --  --  --

```

E) Check driver binding:

```

1 # Load driver if built as module
2 modprobe inv_mpu6050_i2c
3
4 # Check driver loaded
5 ls -l /sys/bus/i2c/drivers/inv-mpu6050-i2c/
6
7 # Verify IIO device created
8 ls /sys/bus/iio/devices/
9
10 # Read sensor data
11 cat /sys/bus/iio/devices/iio:device0/in_accel_x_raw
12 cat /sys/bus/iio/devices/iio:device0/in_gyro_x_raw

```

If IIO device exists, your sensor is working!

5.6 Common Device Tree Modifications

5.6.1 Disabling Unused Peripherals

I think this is also a good start to learn how the device tree works, making small changes and seeing their corresponding results! To save power or free up pins, disable unused peripherals:

```

1 // Example: Disable FlexCAN (not using CAN bus)
2 &flexcan1 {
3     status = "disabled";
4 };
5

```

```

6 &flexcan2 {
7     status = "disabled";
8 };

```

After rebuilding and booting:

```

1 # CAN0 should not appear
2 ip link show | grep can
3 # No output means CAN interfaces are disabled

```

5.6.2 Changing I2C Bus Speed

```

1 &i2c5 {
2     clock-frequency = <400000>; // 400 kHz (Fast Mode)
3     // or <100000> for Standard Mode
4     // or <1000000> for Fast Mode Plus
5 };

```

5.6.3 Adding GPIO-Based Devices

```

1 // Example: Add a GPIO LED
2 leds {
3     compatible = "gpio-leds";
4
5     status_led {
6         label = "status:green";
7         gpios = <&gpio5 3 GPIO_ACTIVE_HIGH>;
8         default-state = "on";
9     };
10 };

```

5.7 Device Tree Overlays

Overlays allow runtime modification of device tree without reflashing the entire DTB. However, this is advanced and not covered in this guide. For this internship, we modified the base DTS and recompiled.

Related Documentation

For device tree overlay usage: See Linux kernel documentation: [Documentation/devicetree/overlay-notes.rst](#)

5.8 Reference: i.MX8MP Device Tree Locations

5.8.1 Linux Kernel Source Structure

linux/

```

|-- arch/
|   +-- arm64/
|       +-- boot/dts/
|           +-- freescale/
|               |-- imx8mp.dtsi          (SoC definitions)
|               |-- imx8mp-evk.dtsi      (EVK common)
|               |-- imx8mp-evk.dts       (Base board)
|               |-- imx8mp-evk-rpmsg.dts (RPMsg variant)
|               +-- imx8mp-pinctrl.h     (Pin mux macros)
|
+-- include/
    +-- dt-bindings/
        |-- clock/
        |   +-- imx8mp-clock.h          (Clock indices)
        |-- interrupt-controller/
        |   +-- irq.h                  (IRQ flags)
        +-- gpio/
            +-- gpio.h                 (GPIO flags)

```

5.8.2 Key Files to Reference

- **Pin functions:** `arch/arm64/boot/dts/freescale/imx8mp-pinctrl.h`
- **Clock indices:** `include/dt-bindings/clock/imx8mp-clock.h`
- **Binding docs:** `Documentation/devicetree/bindings/`
- **i.MX8MP Reference Manual:** Complete register map and peripheral descriptions

5.8.3 Common Pitfalls

Important Note

Don't:

- Edit `.dtb` files directly - always work with `.dts`
- Forget to run `saveenv` in U-Boot
- Assume Linux bus numbers match hardware numbers (i2c-4 might be I2C5!)
- Skip verification steps - always check pinmux, clocks, and bus scan
- Modify SoC `.dtsi` files - override in board `.dts` instead

Chapter 6

Monitoring Infrastructure

This chapter covers the complete monitoring setup used to track system performance, power consumption, and hardware state during development and testing. The monitoring stack consists of Zabbix for data collection and Grafana for visualization, both running in Docker containers on your development machine.

Chapter organization:

- **Sections 6.1-6.9:** Setting up the monitoring infrastructure
- **Section 6.10:** Using the monitoring with stress testing (validation)
- **Section 6.11:** Summary and quick reference

6.1 Why Monitoring is Essential

6.1.1 Purpose of This Setup

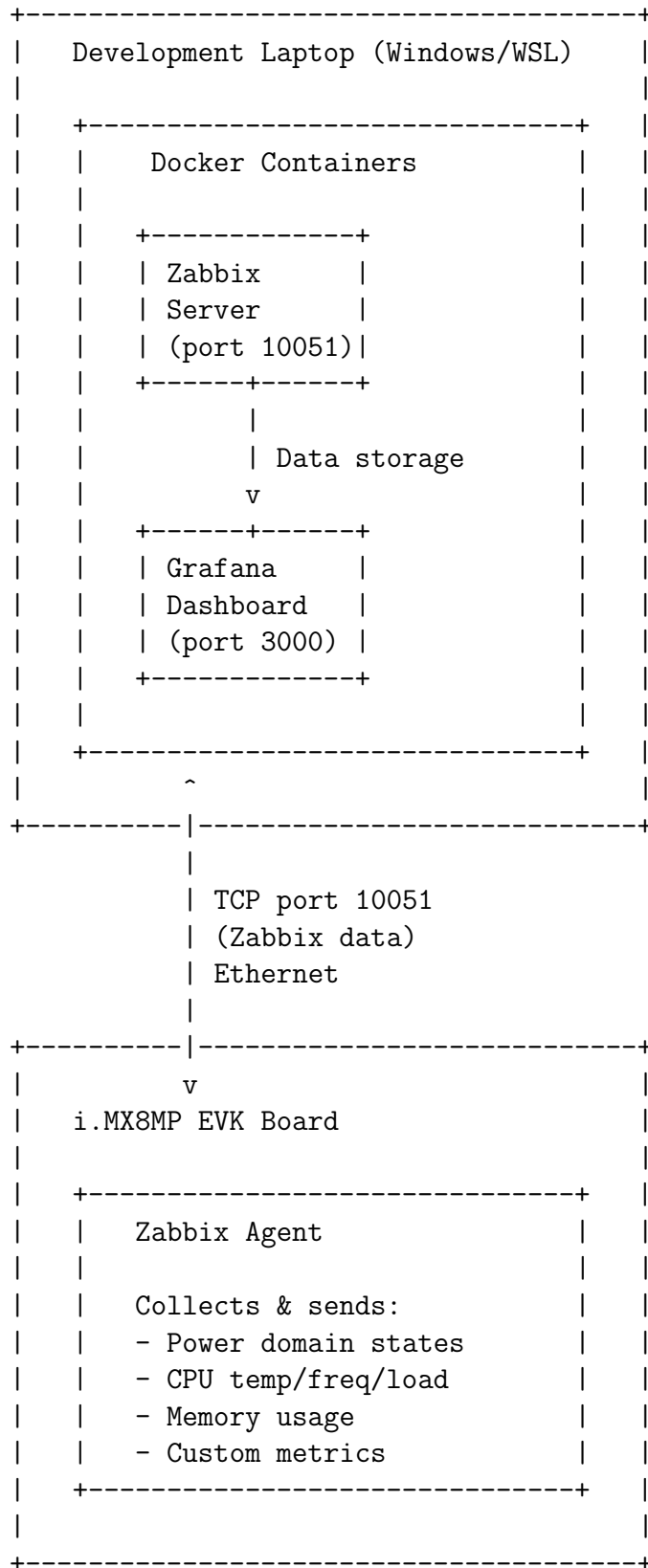
During embedded system development, especially when working on power optimization, it will be nice to have real-time visibility into:

- **Power domain states** - Which parts of the SoC are on/off (GPU, VPU, NPU, audio)
- **CPU metrics** - Temperature, frequency, load per core
- **System resources** - Memory usage, I/O activity
- **Thermal behavior** - How the system heats up under load
- **Performance trends** - Historical data for benchmarking

Use cases during my internship:

- Verify power domains turn off during idle (power optimization)
- Observe which components activate during specific workloads
- Measure impact of ATF modifications on power consumption
- Benchmark NPU/GPU performance under load
- Identify thermal throttling or anomalies

6.2 Architecture Overview



Data Flow:

1. Zabbix Agent collects metrics from board

2. Sends to Zabbix Server (port 10051)
3. Zabbix stores data in internal database
4. Grafana queries Zabbix for visualization
5. You view dashboards at localhost:3000

6.2.1 Component Roles

Component	Purpose
Zabbix Server	Collects and stores metrics, triggers alerts
Zabbix Agent	Runs on i.MX8MP, gathers system metrics
Grafana	Visualizes data with customizable dashboards
PostgreSQL	Database backend for Zabbix (in Docker)
Docker Compose	Orchestrates all containers on laptop

6.3 Prerequisites

6.3.1 Network Requirements

Important Note

Critical: Both laptop and i.MX8MP board must be on the same network. At DSO, I used Ethernet connection via router .

Check IP addresses:

On i.MX8MP board:

```
1 root@imx8mp-lpddr4-evk:~# ip a
2 # Look for inet address on eth0, e.g., 192.168.1.100
```

On laptop (Windows):

```
1 ipconfig
2 # Look for Ethernet adapter IP, e.g., 192.168.1.50
```

Both should be in the same subnet (e.g., 192.168.1.x).

6.3.2 Software Requirements

On development laptop:

- Docker Desktop installed and running
- Docker Compose (included with Docker Desktop)

On i.MX8MP board:

- Zabbix agent package (added via Yocto - see Chapter 4)
- Root or sudo access for power domain monitoring

6.4 Docker Setup (Laptop/Host)

6.4.1 Prerequisites: Docker Installation

Related Documentation

Docker Desktop Setup:

Before proceeding, ensure Docker Desktop is properly installed and running on your laptop. For DSO-specific Docker installation procedures, refer to:

github Bases IT Service "Docker Desktop Installation and Configuration"
Verify Docker is working:

```
1 docker --version
2 docker-compose --version
3 docker ps # Should run without errors
```

6.4.2 Configuration Files Repository

GitHub Repository

All Monitoring Configuration Files:

The complete docker-compose.yml and all configuration files are available at:

<https://github.com/yulyuri/firmware-and-dts/tree/master/zabbixgrafanastuff/zabbix-dso>

Directory structure:

```
monitoring/
|-- docker-compose.yml      <- Start here
|-- zabbix_agentd.conf      <- Board agent config
|-- custom_metrics.conf     <- Power domain metrics
+-- grafana-dashboard.json   <- Pre-built dashboard
```

Quick start: Clone the folder and use the provided files instead of creating from scratch. The Grafana dashboard JSON can be imported directly to get the complete power monitoring setup instantly.

6.4.3 Create Project Directory

```
1 # On Windows with WSL2 or Linux:
2 mkdir -p ~/Desktop/zabbix-dso
3 cd ~/Desktop/zabbix-dso
```

6.4.4 Docker Compose Configuration

Create docker-compose.yml in the project directory or just download from github:

```
1 version: "3.8"
2 services:
```

```

3  zabbix:
4      image: zabbix/zabbix-appliance:alpine-latest
5      container_name: zabbix-1
6      ports:
7          - "8080:80"          # Zabbix frontend
8          - "10051:10051"     # Zabbix server
9      networks:
10         - zbxnet
11
12  zabbix-agent:
13      image: zabbix/zabbix-agent:alpine-latest
14      container_name: zabbix-agent-1
15      environment:
16          - ZBX_SERVER_HOST=zabbix-1
17          - ZBX_HOSTNAME=zabbix-agent-1
18      networks:
19         - zbxnet
20
21  grafana:
22      image: grafana/grafana:latest
23      container_name: grafana-1
24      ports:
25          - "3000:3000"
26      networks:
27         - zbxnet
28
29  networks:
30      zbxnet:
31          driver: bridge

```

⚠ Important Note

About This Setup:

zabbix-appliance: This is an all-in-one image that includes:

- Zabbix Server
- Zabbix Frontend (web UI)
- MySQL database (internal) - stores all metrics and configuration
- All bundled together for easy deployment

This simplifies setup significantly compared to running separate database, server, and frontend containers.

zabbix-agent container: This container is included in the compose file but not actually used. The real Zabbix agent runs directly on the i.MX8MP board, not in Docker.

grafana: Visualization and dashboard platform that queries Zabbix for data.

6.4.5 Start Docker Containers

```
1 cd ~/Desktop/zabbix-dso
2
3 # Start all containers in background
4 docker-compose up -d
5
6 # Verify all containers are running
7 docker ps
8
9 # Expected output: 3 containers running
10 # - zabbix-1
11 # - zabbix-agent-1 (not used, but running)
12 # - grafana-1
```

Check container logs if issues:

```
1 docker-compose logs zabbix
2 docker-compose logs grafana
```

6.4.6 Access Web Interfaces

Zabbix Web UI:

- URL: <http://localhost:8080>
- Default username: Admin
- Default password: zabbix

Grafana:

- URL: <http://localhost:3000>
- Default username: admin
- Default password: admin

Important Note

First time setup: Docker will download images (1-2 GB). This takes a few minutes. Subsequent starts are instant.

6.4.7 Future Work: External Database

⚠ Important Note

Current Limitation:

The zabbix-appliance uses an internal MySQL database for storing all metrics and configuration. This is sufficient for development and testing but has limitations:

Limitations:

- Data is stored inside the container (will be lost if container is gone)
- Limited scalability for long-term data retention
- No easy backup/restore mechanism
- Can't share database with other services

Possible Future Improvement:

Set up an external database (I was exploring InfluxDB) to:

- Persist data outside containers (process/view data after)
- Enable long-term historical analysis (days of data)
- Implement proper backup strategies
- Scale storage independently from Zabbix server
- Share data with other analysis tools

This was not completed during my internship but I think would be useful down the road when everything is set up (PX4 code etc) and can be used for more comprehensive platform for development, testing, and monitoring.

6.5 Zabbix Agent Setup (i.MX8MP Board)

6.5.1 Verify Zabbix Agent Installation

Zabbix agent should already be installed if you added it to your Yocto build (Chapter 4).

```

1 # Check if zabbix agent is installed
2 root@imx8mp-lpddr4-evk:~# which zabbix_agentd
3 /usr/sbin/zabbix_agentd
4
5 # Check version
6 root@imx8mp-lpddr4-evk:~# zabbix_agentd --version
7 zabbix_agentd (daemon) (Zabbix) 7.0.9

```

If not installed, you need to rebuild your Yocto image with zabbix package (see Chapter 4).

6.5.2 Configure Zabbix Agent

Edit the main configuration file:

```
1 nano /etc/zabbix/zabbix_agentd.conf
```

Key settings to change:

```
1 # Server IP (your laptop's IP address)
2 Server=192.168.1.50
3
4 # Active checks (send data to server)
5 ServerActive=192.168.1.50:10051
6
7 # Hostname (must match in Zabbix web UI later)
8 Hostname=imx8mp-board
9
10 # Allow root to run commands (needed for power domain access)
11 AllowRoot=1
12
13 # Include custom metrics directory
14 Include=/etc/zabbix/zabbix_agentd.conf.d/*.conf
```

Important Note

Replace 192.168.1.50 with your actual laptop IP address!

6.5.3 Create Custom Metrics Configuration

Create directory and configuration file for custom metrics:

```
1 mkdir -p /etc/zabbix/zabbix_agentd.conf.d
2 nano /etc/zabbix/zabbix_agentd.conf.d/custom_metrics.conf
3
4 this conf file might be in /etc/zabbix_agentd.conf as well. please check
   properly.
```

Add the following UserParameter definitions:

```
1 # CPU Metrics
2 UserParameter=cpu.temp,awk '{printf "%.1f", $1/1000}' /sys/class/thermal/
   thermal_zone0/temp
3 UserParameter=cpu.freq,awk '{printf "%.0f", $1/1000}' /sys/devices/system/cpu/
   cpu0/cpufreq/scaling_cur_freq
4 UserParameter=cpu0.usage,awk '/^cpu0 / {u=$2+$3+$4; t=$2+$3+$4+$5+$6+$7+$8;
   print int((u/t)*100)}' /proc/stat
5
6 # System Metrics
7 UserParameter=load.1min,cut -d' ' -f1 /proc/loadavg
8 UserParameter=mem.free,free -m | awk '/Mem:/ {print $4}'
9
10 # Power Domain States (on=1, off=0)
11 UserParameter=power.gpu3d,grep -q on /sys/kernel/debug/pm_genpd/gpu3d/
   current_state && echo 1 || echo 0
```

```

12 UserParameter=power.gpu2d,grep -q on /sys/kernel/debug/pm_genpd/gpu2d/
    current_state && echo 1 || echo 0
13 UserParameter=power.vpumix,grep -q on /sys/kernel/debug/pm_genpd/vpumix/
    current_state && echo 1 || echo 0
14 UserParameter=power.vpu-g1,grep -q on /sys/kernel/debug/pm_genpd/vpu-g1/
    current_state && echo 1 || echo 0
15 UserParameter=power.vpu-g2,grep -q on /sys/kernel/debug/pm_genpd/vpu-g2/
    current_state && echo 1 || echo 0
16 UserParameter=power.mlmix,grep -q on /sys/kernel/debug/pm_genpd/mlmix/
    current_state && echo 1 || echo 0
17 UserParameter=power.audiomix,grep -q on /sys/kernel/debug/pm_genpd/audiomix/
    current_state && echo 1 || echo 0

```

6.5.4 Start Zabbix Agent

```

1 # Start agent manually (for testing)
2 zabbix_agentd -c /etc/zabbix/zabbix_agentd.conf
3
4 # Or use systemd (if configured)
5 systemctl restart zabbix-agent
6 systemctl status zabbix-agent

```

6.5.5 Test Custom Metrics

Verify that custom metrics work:

```

1 # Test individual metrics
2 zabbix_agentd -t cpu.temp
3 # Expected: cpu.temp [t/45.2]
4
5 zabbix_agentd -t power.gpu3d
6 # Expected: power.gpu3d [t/0] or [t/1]
7
8 zabbix_agentd -t mem.free
9 # Expected: mem.free [t/1234]

```

Common errors:

- [m|ZBX_NOTSUPPORTED] [Unsupported item key.] - Metric not defined in config
- [m|ZBX_NOTSUPPORTED] [Cannot obtain system information.] - Permission denied or path wrong
- No output - Agent not running or config file not loaded

6.6 Power Domain Monitoring Deep Dive

6.6.1 What Are Power Domains?

The i.MX8M Plus has multiple power domains that can be independently controlled to save power:

Domain	Controls
gpu3d	3D graphics accelerator
gpu2d	2D graphics and compositor
vpumix	Video processing unit parent domain
vpu-g1	VPU decoder (H.264, etc.)
vpu-g2	VPU encoder/decoder (HEVC, VP9)
mlmix	Machine learning accelerator (NPU)
audiomix	Audio processing and codecs

6.6.2 Power State Files

Linux exposes power domain states via debugfs:

```

1 # Check all power domain states
2 root@imx8mp-lpddr4-evk:~# ls /sys/kernel/debug/pm_genpd/
3 audiomix gpu2d gpu3d mlmix vpu-g1 vpu-g2 vpumix
4
5 # Check individual state
6 root@imx8mp-lpddr4-evk:~# cat /sys/kernel/debug/pm_genpd/gpu3d/current_state
7 off
8
9 # When GPU is active:
10 root@imx8mp-lpddr4-evk:~# cat /sys/kernel/debug/pm_genpd/gpu3d/current_state
11 on

```

6.6.3 Why This Matters

For power optimization:

- Idle domains should be OFF to save power
- Monitoring shows if optimizations actually work
- Can identify "zombie" domains that stay on unnecessarily

For debugging:

- See which workload activates which domain
- Verify driver power management is working
- Understand system behavior under different loads

Important Note

Example Discovery: During testing, you might notice `audiomix` stays on even when not playing audio. This indicates a potential driver bug or missing power management hook - something you'd never discover without monitoring!

6.7 Zabbix Web UI Configuration

6.7.1 Add Host (i.MX8MP Board)

1. Log into Zabbix: <http://localhost:8080>
2. Go to: **Configuration** → **Hosts**
3. Click **Create host**
4. Fill in:
 - **Host name:** `imx8mp-board` (must match agent config!)
 - **Groups:** Select "Linux servers" or create "Embedded Devices"
 - **Agent interface:** Add IP address of i.MX8MP (e.g., 192.168.1.100)
 - **Port:** 10050
5. Click **Add**

Verify connection:

- Go to **Monitoring** → **Latest data**
- Select host: `imx8mp-board`
- Should see "ZBX" indicator turn green within 60 seconds

6.7.2 Create Custom Items

For each custom metric, create an item:

1. Go to: **Configuration** → **Hosts** → `imx8mp-board` → **Items**
2. Click **Create item**
3. For GPU3D power state:
 - **Name:** GPU3D Power State
 - **Key:** `power.gpu3d`
 - **Type of information:** Numeric (unsigned)
 - **Update interval:** 5s
4. Repeat for all power domains and metrics

6.8 Grafana Dashboard Setup

6.8.1 Install Zabbix Plugin

1. Access Grafana: <http://localhost:3000>
2. Go to: **Administration** → **Plugins**
3. Search for: **Zabbix**
4. Install: **Zabbix plugin by Alexander Zobnin**

Note: The Docker Compose file already installs this plugin, so it should be available.

6.8.2 Add Zabbix Data Source

1. Go to: **Connections** → **Data sources**
2. Click **Add data source**
3. Select **Zabbix**
4. Configure:
 - **Name:** Zabbix Server
 - **URL:** `http://zabbix-server:8080/api_jsonrpc.php`
 - **Username:** Admin
 - **Password:** zabbix
5. Click **Save & Test**
6. Should show: "Data source is working"

6.8.3 Create Power Domain Dashboard

1. Go to: **Dashboards** → **New** → **New Dashboard**
2. Click **Add visualization**
3. Select **Polystat** panel
4. Configure panel:
 - **Data source:** Zabbix Server
 - **Group:** Embedded Devices (or your host group)
 - **Host:** imx8mp-board
 - **Item:** power.gpu3d
5. Under **Polystat options**:
 - **Layout:** Honeycomb or AutoSize

- **Color mode:** Threshold-based
- **Thresholds:**
 - 0 = Red (domain OFF)
 - 1 = Green (domain ON)

6. Click **Apply**

7. Repeat for other power domains

Organize panels by rows:

- Row 1: GPU domains (gpu3d, gpu2d)
- Row 2: VPU domains (vpumix, vpu-g1, vpu-g2)
- Row 3: Other domains (mlmix, audiomix)

6.8.4 Add System Metrics Panels

CPU Temperature (Gauge):

- Panel type: Gauge
- Item: cpu.temp
- Min: 20, Max: 90
- Thresholds: Green ≤ 60 , Yellow 60-75, Red ≥ 75

Memory Free (Stat):

- Panel type: Stat
- Item: mem.free
- Unit: MB

Load Average (Graph):

- Panel type: Time series
- Item: load.1min
- Shows system load over time

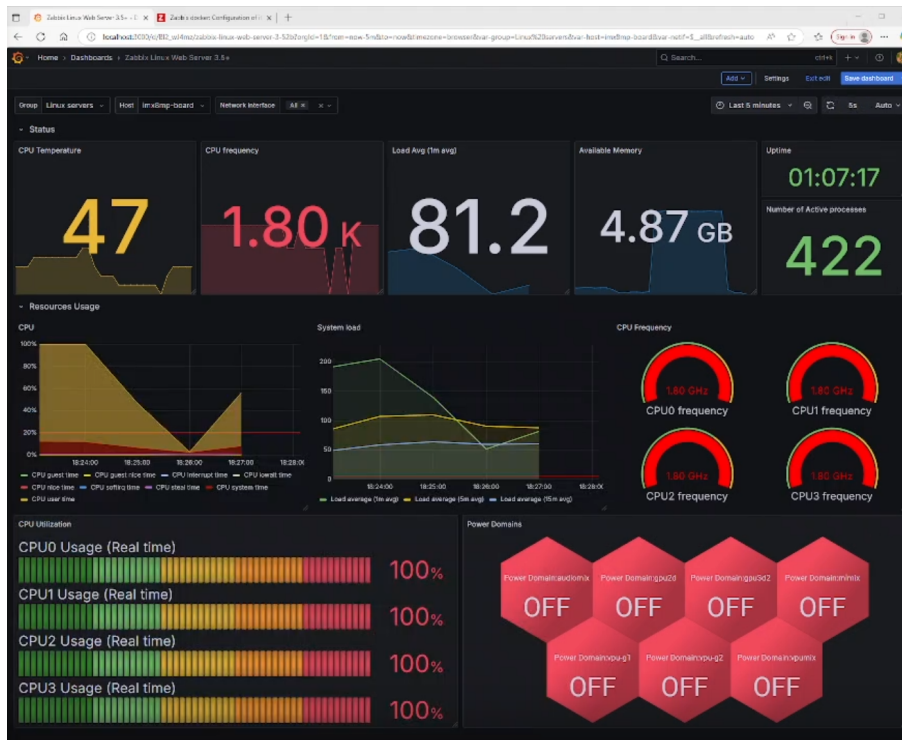


Figure 6.1: Grafana Dashboard

6.9 System Validation with Stress Testing

⚠ Important Note

Important Distinction:

The following sections describe how to **USE** the monitoring infrastructure you just set up. stress-ng is **NOT part of the monitoring stack** - it's a separate testing tool that generates controlled load so you can observe system behavior through Zabbix and Grafana.

Workflow:

1. Open Grafana dashboard (monitoring is already running)
2. Run stress-ng on the i.MX8MP board (generates load)
3. Watch power domains, temperature, CPU frequency change in real-time on Grafana
4. Document findings and validate optimizations

6.9.1 What is stress-ng?

stress-ng (stress next generation) is a comprehensive Linux system stress testing tool that can exercise various subsystems including CPU, memory, I/O, filesystem, and more.

Key capabilities:

- 280+ different stress test methods

- Parallel execution of multiple stressors
- Configurable duration and intensity
- Built-in metrics and verification
- Minimal dependencies (works on embedded systems)

Related Documentation

Complete stress-ng Documentation:

- GitHub repository: <https://github.com/ColinIanKing/stress-ng>
- Manual page: <https://manpages.ubuntu.com/manpages/jammy/man1/stress-ng.1.html>
- Stress methods reference: <https://wiki.ubuntu.com/Kernel/Reference/stress-ng>

These resources document all available stress tests, options, and usage examples.

6.9.2 Installation

Recommended: Add to Yocto build

```

1 # In build-xywayland/conf/local.conf
2 IMAGE_INSTALL:append = " stress-ng"
3
4 # Rebuild image
5 bitbake imx-image-full

```

Verify installation:

```

1 root@imx8mp-lpddr4-evk:~# stress-ng --version
2 stress-ng, version 0.XX.XX

```

6.9.3 Stress Testing Commands

CPU stress (all cores):

```

1 stress-ng --cpu 4 --timeout 60s

```

Memory stress:

```

1 stress-ng --vm 2 --vm-bytes 512M --timeout 60s

```

I/O stress:

```

1 stress-ng --io 2 --hdd 1 --timeout 60s

```

Comprehensive stress (all-in-one):

```

1 stress-ng --cpu 4 --io 2 --vm 2 --hdd 1 --timeout 120s

```

CPU method variations:

```

1 # Test all CPU stress methods
2 stress-ng --cpu 4 --cpu-method all --timeout 60s
3
4 # Specific methods
5 stress-ng --cpu 4 --cpu-method fft --timeout 60s
6 stress-ng --cpu 4 --cpu-method matrixprod --timeout 60s

```

6.9.4 Stress Test Categories

stress-ng organizes tests into categories based on what subsystem they stress:

CPU stressors:

```

1 # Integer arithmetic
2 stress-ng --cpu 4 --cpu-method ackermann --timeout 60s
3
4 # Floating point operations
5 stress-ng --cpu 4 --cpu-method fft --timeout 60s
6 stress-ng --cpu 4 --cpu-method omega --timeout 60s
7
8 # Matrix operations
9 stress-ng --cpu 4 --cpu-method matrixprod --timeout 60s
10
11 # All CPU methods sequentially
12 stress-ng --cpu 4 --cpu-method all --timeout 300s

```

Memory stressors:

```

1 # Virtual memory allocation/deallocation
2 stress-ng --vm 2 --vm-bytes 512M --timeout 60s
3
4 # Memory copy operations
5 stress-ng --memcpy 4 --timeout 60s
6
7 # Cache thrashing
8 stress-ng --cache 4 --timeout 60s
9
10 # Memory bandwidth stress
11 stress-ng --stream 4 --timeout 60s

```

I/O and filesystem stressors:

```

1 # Generic I/O operations
2 stress-ng --io 4 --timeout 60s
3
4 # Disk write/read
5 stress-ng --hdd 2 --hdd-bytes 256M --timeout 60s
6
7 # File allocation
8 stress-ng --fallocate 4 --timeout 60s
9
10 # Directory operations
11 stress-ng --dentry 4 --timeout 60s

```

6.9.5 Useful stress-ng Options

Option	Purpose
-timeout Ns	Run for N seconds
-metrics	Show bogo ops (operations/sec) at end
-metrics-brief	Condensed metrics output
-verify	Verify computations are correct
-log-file FILE	Save output to file for later analysis
-aggressive	More intense stress (higher resource usage)
-taskset N	Pin workers to specific CPU cores
-cpu-load N%	Target N% CPU load (vs 100%)

Example with metrics:

```

1 stress-ng --cpu 4 --timeout 60s --metrics-brief
2 # Shows: bogo ops/s for each worker (performance metric)

```

Pin to specific cores:

```

1 # Stress only cores 0 and 1
2 stress-ng --cpu 2 --taskset 0,1 --timeout 60s
3
4 # Test single core performance
5 stress-ng --cpu 1 --taskset 0 --timeout 60s

```

6.9.6 Thermal Behavior Observations

Important Note

Key Finding from Extended Stress Testing:

During comprehensive stress testing over extended periods (60+ seconds with maximum load), the following thermal behavior was consistently observed:

Temperature Profile:

- Idle: 35-40°C
- Light load (1-2 cores): 45-50°C
- Heavy load (all cores): 55-60°C
- Peak temperature: Never exceeded 60°C
- Thermal throttling: Never observed

Implications:

- The EVK board has adequate cooling (heatsink + natural airflow)
- The i.MX8MP's thermal design is conservative
- No thermal throttling interferes with benchmarking results
- The board can sustain full CPU load indefinitely
- Temperature is not a limiting factor for performance testing

6.9.7 What could be explored next: CPU Frequency Scaling and Governors

During this internship, CPU frequency scaling was observed but not thoroughly investigated. This is an important area for future work, especially for real-time flight control applications.

Current Status

The i.MX8MP uses the Linux CPUfreq framework to dynamically scale CPU frequency based on load. The system was tested with the default governor, but detailed characterization of scaling behavior was not completed.

Check current configuration:

```

1 # Current governor
2 cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
3 # ondemand , performance etc
4
5 # Current frequency for each core
6 cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cur_freq
7 # Shows frequency in kHz (e.g., 1800000 = 1.8 GHz)
8
```

```
9 # Min/Max frequency limits
10 cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_min_freq
11 cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq
12
13 # Available frequency steps
14 cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
```

Monitor frequency in real-time:

```
1 # Watch all cores
2 watch -n 0.5 'cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cur_freq'
3
4 # Or use Grafana (if cpu.freq metric configured)
5 # More convenient for observing during stress tests
```

Unexplored Questions

 Important Note**Critical Questions for Future Investigation:
Governor Behavior (Thresholds and Timing):**

- At what CPU utilization percentage does the governor trigger frequency up-scaling?
- How long does the CPU stay at high frequency before scaling down?
- What is the time window for calculating "load"? (1ms? 10ms? 100ms?)
- Is load calculated per-core or system-wide?
- Does the governor consider thermal feedback when scaling?
- What happens when a high-priority real-time task wakes up? Immediate scaling or delayed?

Governor Comparison (Performance vs Power):

- **Performance impact:** How much faster is "performance" vs "schedutil" for typical workloads?
- **Power consumption:** Measure actual power draw differences between governors
- **Latency:** Time to scale from minimum to maximum frequency
- **Response time:** How quickly does governor react to load changes?
- **Best for flight control:** Which governor provides lowest latency for real-time tasks?

Scaling Parameters (Tuning Potential):

- Can up-scaling thresholds be adjusted? (e.g., scale up at 70% load instead of 80%)
- Can down-scaling delay be configured? (stay at high freq longer)
- Are there governor-specific tunables in `/sys/devices/system/cpu/cpufreq/`?
- What's the overhead of frequency transitions themselves?

Quick Governor Switching for Testing

```

1 # Switch to performance governor (fixed maximum frequency)
2 echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/
   scaling_governor
3
4 # Switch to powersave governor (fixed minimum frequency)

```

```

5 echo powersave | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
6
7 # Switch to schedutil (dynamic, scheduler-driven - recommended)
8 echo schedutil | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
9
10 # Switch to ondemand (dynamic, legacy but common)
11 echo ondemand | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
12
13 # Verify change took effect
14 cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor

```

Suggested testing procedure:

1. Switch to performance governor
2. Run stress-ng benchmark, record metrics
3. Switch to powersave governor
4. Run same benchmark, record metrics (will be much slower!)
5. Switch to schedutil governor
6. Run same benchmark, record metrics
7. Compare: performance, power consumption (if measurable), temperature
8. Observe frequency changes in Grafana during each test

Related Documentation

Recommended Tools for Deep Governor Investigation:

- **trace-cmd / ftrace:** Kernel tracing to capture frequency transitions with timestamps
- **perf:** CPU performance counter analysis
- **powertop:** Power consumption analysis tool
- **schedviz:** Visualize scheduler behavior and frequency changes

Example trace command:

```

1 # Trace frequency changes
2 trace-cmd record -e power:cpu_frequency -e sched:sched_switch
3 trace-cmd report

```

Implications for Flight Control

For PX4 flight controller software (Chapter 9: M7 Development), CPU frequency scaling behavior is critical:

- **Latency sensitivity:** Flight control requires deterministic response times

- **Real-time performance:** Frequency scaling can introduce jitter
- **Power efficiency:** Lower frequencies save power during hover/cruise
- **Recommendation:** May need to pin flight control threads to specific cores at fixed frequency

This investigation should be completed before deploying any flight control software.

6.9.8 Other Unexplored Monitoring Topics

Database Integration

As mentioned earlier, migrating to an external database would enable:

- InfluxDB for time-series data (better than MySQL for metrics)
- Long-term trend analysis (weeks/months of data)
- Automated anomaly detection
- Machine learning on historical data

Additional Power Domains

Other power domains exist but were not monitored:

- HDMI/MIPI display power domains
- PCIe power management states
- USB subsystem power states
- SATA/storage power management

Check availability: `ls /sys/kernel/debug/pm_genpd/`

Per-Core Metrics

More granular CPU monitoring:

- Individual core frequency tracking (each core independently)
- some sort of more accurate way to measure temperature? other than thermal zones?
- Core isolation testing (disable cores, test performance)
- CPU affinity impact on performance

Advanced Grafana Dashboards/graphs/visualisation

Potential dashboard improvements:

- Correlation panels (show CPU load vs frequency vs temperature)
- Comparison views (before/after optimization side-by-side)
- Annotations for test events (mark when tests started/ended)

6.10 Startup Procedure (Quick Reference)

6.10.1 Every Time You Start Monitoring

Step 1: Start Docker containers (Laptop)

```
1 cd ~/Desktop/zabbix-dso
2 docker-compose up -d
3 docker ps # Verify all running
```

Step 2: Get board IP address

```
1 # On i.MX8MP
2 ip a
3 # Note the IP, e.g., 192.168.1.100
```

Step 3: Update Zabbix agent config (if IP changed)

```
1 # Only if laptop IP changed
2 nano /etc/zabbix/zabbix_agentd.conf
3 # Update Server= and ServerActive= lines
```

Step 4: Start Zabbix agent (Board)

```
1 zabbix_agentd -c /etc/zabbix/zabbix_agentd.conf
2 # Or: systemctl restart zabbix-agent
```

Step 5: Access dashboards

- Zabbix: <http://localhost:8080>
- Grafana: <http://localhost:3000>

Chapter 7

NPU and Machine Learning

Important Note

Exploration Status:

This chapter documents preliminary NPU exploration conducted over approximately **one day** of testing. The work focused on:

- Verifying NPU functionality with basic inference
- Understanding TensorFlow Lite with VX Delegate
- Running benchmark models (MobileNet V1)
- Basic performance profiling

This is NOT comprehensive ML optimization work. There is significant room for exploration in quantization strategies, model optimization, and understanding what computer vision tasks are actually required for guided systems/drone applications.

7.1 Introduction

7.1.1 Why NPU Matters for Embedded Systems

The i.MX8M Plus includes a dedicated Neural Processing Unit (NPU) capable of 2.3 TOPS (Tera Operations Per Second) for AI inference. This hardware accelerator enables:

- **Real-time computer vision** - Object detection, tracking
- **Low power inference** - More efficient than CPU/GPU
- **Parallel processing** - Optimized for neural network operations
- **Edge AI** - Run models locally without cloud dependency

Potential applications for drone/guided systems:

- Object detection and avoidance

- Target tracking and recognition
- Autonomous navigation
- Visual odometry
- Situational awareness

Hardware Items

i.MX8MP NPU Specifications:

- **Architecture:** Vivante VIP8000NanoSi-UltraLite
- **Performance:** 2.3 TOPS INT8
- **Power:** Integrated with automatic power management
- **Frameworks:** TensorFlow Lite, ONNX Runtime (via delegates)
- **Quantization:** Optimized for INT8, supports mixed precision

7.1.2 Scope of This Investigation

During this internship, NPU work was limited to:

1. Basic setting up of TensorFlow Lite runtime
2. Understanding VX Delegate (NPU acceleration layer)
3. Running pre-trained quantized models
4. Basic performance benchmarking
5. Exploring profiling tools

What was NOT explored:

- Custom model training and optimization
- Quantization strategy comparison (INT8 vs mixed precision)
- Real-world application development (object detection pipeline)
- Power consumption measurement during inference
- Integration with flight control software
- What CV tasks are actually needed for DSO's guided systems
- How to use it concurrently with other tasks? What kind of ML tasks should be offloaded to NPU etc.

7.2 NPU Architecture and Power Management

7.2.1 What is a Delegate?

A **delegate** is a plugin or runtime module that tells the inference engine **how** and **where** to execute a model.

Common delegates:

- **Default (no delegate):** Runs on CPU using optimized kernels
- **GPU Delegate:** Offloads operations to GPU
- **VX Delegate (libvx_delegate.so):** Offloads to NPU on i.MX8MP
- **XNNPACK:** Highly optimized CPU execution

How delegates work:

1. TensorFlow Lite loads your model (.tflite file)
2. Delegate analyzes which operations it can accelerate
3. Supported operations get offloaded to NPU/GPU
4. Unsupported operations fall back to CPU
5. Results are combined seamlessly

7.2.2 NPU Power Efficiency

Important Note

Key Finding:

The i.MX8MP NPU is tightly integrated into the SoC with automatic power management:

- **Idle state:** NPU is powered down (mlmix power domain OFF)
- **Active inference:** NPU powers up automatically when delegate loads
- **Post-inference:** NPU returns to low-power standby
- **No manual control needed:** Linux kernel and runtime handle power states

This makes the NPU excellent for battery-powered edge applications - it only consumes power when actively processing.

Verify NPU power state with monitoring:

```

1 # Check mlmix (NPU) power domain
2 cat /sys/kernel/debug/pm_genpd/mlmix/current_state
3
4 # During inference: shows "on"
5 # When idle: shows "off"
```

See Chapter 6 (Monitoring Infrastructure) for real-time power domain visualization in Grafana.

7.3 TensorFlow Lite Setup

7.3.1 Pre-installed Runtime

The Yocto image used during this internship came with TensorFlow Lite pre-installed:

```
1 root@imx8mp-lpddr4-evk:~# ls /usr/bin/tensorflow-lite-2.16.2/  
2 benchmark  examples  label_image  python  
3  
4 root@imx8mp-lpddr4-evk:~# cd /usr/bin/tensorflow-lite-2.16.2/examples
```

Included components:

- TensorFlow Lite runtime (version 2.16.2)
- Python bindings for TFLite
- VX Delegate library (`libvx_delegate.so`)
- Example scripts and test images
- Pre-trained models (MobileNet V1 quantized)

Related Documentation

Official NXP Documentation:

i.MX Machine Learning User's Guide (UG10166): <https://www.nxp.com/docs/en/user-guide/IMX-MACHINE-LEARNING-UG.pdf>

This guide covers:

- TensorFlow Lite installation and configuration
- VX Delegate usage and optimization
- Model conversion and quantization
- Performance benchmarking tools

NPU Performance Tuning (Chinese documentation): https://community.nxp.com/pwmx87654/attachments/pwmx87654/imx-processors@tkb/5692/1/i.MX8MPNPUdebugandperformancefinetuning_v1_released.pdf I found this much more beginner friendly It explained:

- concept of quantization and how it affects runtime
- Step by step guide. Choosing the appropriate model... etc

Detailed profiling guide with bandwidth analysis and optimization techniques.

7.4 Basic Inference with MobileNet

7.4.1 Test Setup

The TensorFlow Lite examples directory contains everything needed for basic testing:

```

1 root@imx8mp-lpddr4-evk:~# cd /usr/bin/tensorflow-lite-2.16.2/examples
2 root@imx8mp-lpddr4-evk:/usr/bin/tensorflow-lite-2.16.2/examples# ls
3 grace_hopper.bmp           # Test image
4 labels.txt                 # ImageNet class labels
5 label_image.py             # Inference script
6 mobilenet_v1_1.0_224_quant.tflite # Quantized model

```

7.4.2 Running CPU-Only Inference

Without NPU acceleration (CPU baseline):

```

1 python3 label_image.py \
2     --model_file mobilenet_v1_1.0_224_quant.tflite \
3     --image grace_hopper.bmp \
4     --label_file labels.txt
5
6 # Output (approximate):
7 # Inference time: ~45-60 ms (CPU only)

```

7.4.3 Running NPU-Accelerated Inference

With VX Delegate (NPU):

```

1 python3 label_image.py \
2     --model_file mobilenet_v1_1.0_224_quant.tflite \
3     --image grace_hopper.bmp \
4     --label_file labels.txt \
5     --ext_delegate libvx_delegate.so

```

Output:

```

1 Loading external delegate from libvx_delegate.so with args: {}
2 INFO: Vx delegate: allowed_cache_mode set to 0.
3 INFO: Vx delegate: device num set to 0.
4 INFO: Vx delegate: allowed_builtin_code set to 0.
5 INFO: Vx delegate: error_during_init set to 0.
6 INFO: Vx delegate: error_during_prepare set to 0.
7 INFO: Vx delegate: error_during_invoke set to 0.
8 W [HandleLayoutInfer:332]Op 162: default layout inference pass.
9
10 Warm-up time: 3006.9 ms
11
12 Inference time: 3.2 ms
13
14 0.870588: military uniform
15 0.031373: Windsor tie

```

```

16 0.011765: mortarboard
17 0.007843: bow tie
18 0.007843: bulletproof vest

```

⚠ Important Note

Performance Analysis:

Warm-up time: 3006.9 ms (3 seconds)

- First-time NPU initialization
- Model compilation and optimization
- Memory allocation on NPU
- Operator graph transformation

Inference time: 3.2 ms

- Actual inference on NPU after warm-up
- **15x faster than CPU** (45ms vs 3.2ms)
- Subsequent inferences maintain this speed

Implications:

- First inference has latency (warm-up overhead)
- Suitable for applications with sustained inference (video processing)
- Pre-warming NPU recommended for latency-critical applications

7.5 Performance Profiling

7.5.1 NPU Profiling Environment Variables

Enable detailed NPU performance metrics:

```

1  # Set profiling environment variables
2  export VSI_NN_LOG_LEVEL=0      # Clean logs (0 = minimal)
3  export CNN_PERF=1             # Enable CNN performance stats
4  export NN_EXT_SHOW_PERF=1     # Show extended performance info
5  export VIV_VX_DEBUG_LEVEL=0   # Debug level (0 = clean, 1 = verbose)
6  export VIV_VX_PROFILE=1       # Enable VX profiling
7
8  # Run inference
9  python3 label_image.py \
10     --model_file mobilenet_v1_1.0_224_quant.tflite \
11     --image grace_hopper.bmp \
12     --label_file labels.txt \
13     --ext_delegate libvx_delegate.so

```

Environment variable effects:

- `VSI_NN_LOG_LEVEL=0`: Keeps output clean (set to 5 for debug)
- `CNN_PERF=1`: Prints layer-by-layer performance
- `NN_EXT_SHOW_PERF=1`: Shows memory bandwidth usage
- `VIV_VX_PROFILE=1`: Enables cycle counting and GPU stats

7.5.2 Understanding Profiling Metrics

When profiling is enabled, you'll see additional metrics:

Bandwidth Metrics:

- **TOTAL_READ_BANDWIDTH**: Total bandwidth when reading data
- **TOTAL_WRITE_BANDWIDTH**: Total bandwidth when writing data
- **AXI_READ_BANDWIDTH**: AXI bus read from SRAM
- **AXI_WRITE_BANDWIDTH**: AXI bus write to SRAM
- **DDR_READ_BANDWIDTH**: DDR memory read operations
- **DDR_WRITE_BANDWIDTH**: DDR memory write operations

Cycle Metrics:

- **GPUTOTALCYCLES**: Total number of NPU cycles
- **GPUIDLECYCLES**: Number of cycles NPU was idle

NPU Utilization Calculation:

```
1 NPU_Usage = (GPUTOTALCYCLES - GPUIDLECYCLES) / GPUTOTALCYCLES * 100%
```

Related Documentation

Bandwidth and Performance Analysis:

NXP Community discussion on profiling metrics: <https://community.nxp.com/t5/i-MX-Processors/NPU-Profiler-How-should-I-understand-TOTAL-READ-BANDWIDTH-and-td-p/1859922>

This thread explains each profiling metric and how to interpret them for optimization.

7.5.3 Batch Testing and Logging

For comprehensive benchmarking, redirect profiling output to a file:

```
1 # Run inference with profiling, save to log
2 python3 label_image.py \
3     --model_file mobilenet_v1_1.0_224_quant.tflite \
4     --image grace_hopper.bmp \
5     --label_file labels.txt \
```

```

6  --ext_delegate libvx_delegate.so \
7  > myprofile.log 2>&1
8
9  # Or create a batch script
10 ./batch_detect.sh > myprofile.log 2>&1

```

Analyzing logs:

```

1  # Extract inference times
2  grep "Inference time" myprofile.log
3
4  # Extract bandwidth metrics
5  grep "BANDWIDTH" myprofile.log
6
7  # Calculate average NPU usage
8  grep "GPUTOTALCYCLES\|GPUIDLECYCLES" myprofile.log

```

7.6 Model Quantization

7.6.1 What is Quantization?

Quantization reduces model precision from 32-bit floating point (FP32) to lower bit representations (INT8, INT16) to:

- Reduce model size (4x smaller for INT8)
- Increase inference speed (INT8 operations faster)
- Lower memory bandwidth requirements
- Enable NPU acceleration (optimized for INT8)

Trade-off: Slightly reduced accuracy (1-2% typically) for significant speed gains.

7.6.2 MobileNet V1 Quantized Model

The test model used was `mobilenet_v1_1.0_224_quant.tflite`:

- **Architecture:** MobileNet V1 (lightweight CNN)
- **Input:** 224x224 RGB images
- **Quantization:** INT8 (8-bit integer operations)
- **Classes:** 1000 ImageNet categories
- **Size:** 4.3 MB (vs 17 MB for FP32 version)

7.6.3 Quantization Strategies (Unexplored)

Important Note

Critical Unexplored Area:

Different quantization strategies have different impacts on accuracy and performance:

Full INT8 Quantization:

- All layers quantized to 8-bit
- Maximum NPU acceleration
- Highest speed, lowest accuracy

Mixed Precision:

- Critical layers remain FP32/FP16
- Less critical layers quantized to INT8
- Balanced speed and accuracy
- May reduce NPU utilization

Dynamic Range Quantization:

- Weights quantized, activations remain FP32
- Smaller model size, moderate speed gain
- Better accuracy preservation

Questions for Future Investigation:

- What accuracy loss is acceptable for drone vision tasks?
- Which layers are most sensitive to quantization?
- Can quantization-aware training reduce accuracy loss?
- What's the optimal bit-width for different CV tasks (detection vs classification)?
- How does quantization affect object detection models (YOLO, SSD, Efficient-Det)?

7.7 Unexplored Areas and Future Work

7.7.1 Computer Vision Requirements for Guided Systems

Important Note

Critical Knowledge Gap:

During this internship, there was **no information available** about what computer vision tasks are actually required for DSO's guided systems/drone architecture:

Unknown requirements:

- What objects need to be detected? (targets, obstacles, landmarks?)
- What detection accuracy is required? (mAP threshold?)
- What frame rate is needed? (real-time 30fps? or 5fps is sufficient?)
- What resolution are the cameras? (affects model input size)
- Indoor or outdoor operation? (lighting conditions matter)
- Static or moving targets?
- Detection only, or also tracking/recognition?

Impact on ML development:

Without knowing the actual requirements, it's impossible to:

- Select appropriate model architectures
- Determine optimal quantization strategy
- Set performance benchmarks
- Validate if NPU acceleration is sufficient
- Design end-to-end inference pipeline

7.8 Summary/tldr

7.8.1 What Was Accomplished

During one day of NPU exploration:

- Verified NPU functionality with TensorFlow Lite
- Ran basic inference with MobileNet V1 quantized
- Measured inference performance: 3.2ms (vs 45-60ms CPU)
- Explored profiling tools and environment variables

7.8.2 Key Findings

- **NPU is fast:** 15x faster than CPU for INT8 inference
- **Warm-up overhead:** First inference has 3-second delay
- **Power efficient:** NPU only active during inference
- **Easy to use:** Single parameter (`-ext_delegate`) enables NPU
- **Well supported:** Good documentation from NXP but not fully explored of course!

Chapter 8

DSP Exploration

Important Note

Exploration Status:

The DSP (Digital Signal Processor) on the i.MX8M Plus was **not actively explored** during this internship. This chapter documents:

- What the DSP is and its intended purpose
- Available documentation reviewed
- Architectural similarities to M7 core (RPMmsg, remoteproc)
- Why it might be relevant for future work

I did not write any actual DSP code This is purely a reference for future investigation.

8.1 HiFi 4 DSP Overview

8.1.1 What is the DSP?

The i.MX8M Plus includes a Cadence Tensilica HiFi 4 DSP core, designed specifically for audio and signal processing tasks.

Hardware Items

HiFi 4 DSP Specifications:

- **Architecture:** Cadence Tensilica HiFi 4
- **Clock:** Up to 800 MHz
- **Purpose:** Audio processing, voice recognition, signal processing
- **Key features:**
 - SIMD (Single Instruction Multiple Data) for parallel audio processing
 - Optimized for low-power audio codecs
 - Hardware accelerators for common audio algorithms
 - Separate instruction and data memory (Harvard architecture)
- **Memory:** Dedicated SRAM for fast access
- **Communication:** Shares RPSmsg/remoteproc framework with M7

8.1.2 Typical Use Cases

The DSP is optimized for:

- **Audio codecs:** AAC, MP3, Opus encoding/decoding
- **Voice processing:** Noise cancellation, echo suppression, beamforming
- **Voice recognition:** Wake word detection, speech-to-text preprocessing
- **Signal processing:** FFT, filtering, convolution
- **Audio effects:** Equalization, spatial audio, reverb

Why use DSP instead of CPU?

- More power-efficient for audio tasks (specialized instructions)
- Frees up A53 cores for other processing
- Lower latency for real-time audio processing
- Dedicated hardware accelerators

8.2 Architecture and Integration

8.2.1 Similarity to M7 Core

The DSP shares the same communication infrastructure as the M7 core (Chapter 9):

Feature	M7 Core	DSP
OS Support	NuttX/freeRTOS	Zephyr RTOS
Communication	RPMMsg	RPMMsg
Linux Driver	remoteproc	remoteproc
Firmware Loading	ATF/U-Boot	ATF/U-Boot
Use Case	Real-time control	Audio processing

Important Note

Key Insight:

If you understand M7 core development (Chapter 9), the DSP follows the same pattern:

1. Develop firmware in Zephyr RTOS
2. Compile DSP binary
3. Load via Linux remoteproc driver
4. Communicate via RPMMsg

The main difference is the DSP runs Zephyr (not NuttX) and has audio-specific libraries.

8.2.2 RPMMsg and Remoteproc Framework

Just like the M7 core, the DSP uses:

RPMMsg (Remote Processor Messaging):

- Message passing between A53 cores (Linux) and DSP
- Virtual serial ports or custom protocols
- Shared memory for data transfer

Remoteproc (Remote Processor Control):

- Linux kernel framework for managing remote processors
- Handles DSP firmware loading
- Controls DSP power state (start/stop)
- Manages shared resources

Check DSP status:

```

1 # Check if DSP remoteproc is available
2 ls /sys/class/remoteproc/
3
4 # Check DSP state
5 cat /sys/class/remoteproc/remoteproc1/state
6 # Typical: "offline" or "running"
7
```

```
8 # Check DSP firmware path
9 cat /sys/class/remoteproc/remoteproc1/firmware
10 Take note that there is a specific dts file meant for running the dsp please
    use that
```

8.3 Running Zephyr RTOS on DSP

8.3.1 Zephyr RTOS Overview

The DSP can run Zephyr RTOS which is interesting as this would mean that there could be potentially 3 OS found on the imx8mp: linux on A core, NuttX on M7 and possibly zephyr on the DSP

Zephyr for i.MX8MP DSP:

- NXP provides Zephyr BSP for HiFi 4 DSP
- Pre-built audio processing samples
- RPMsg communication examples
- Integration with Linux audio subsystem (ALSA)

8.3.2 Documentation Reviewed

Related Documentation

Key Documentation:**NXP i.MX DSP User's Guide (UG10167):**

Primary reference for DSP development on i.MX8M Plus. Covers:

- DSP architecture and memory map
- Zephyr RTOS setup and configuration
- Building and loading DSP firmware
- Audio codec integration
- RPMsg communication examples

Application Note AN13970:

"Running Zephyr RTOS on Cadence Tensilica HiFi 4 DSP"

Step-by-step guide for:

- Setting up Zephyr development environment
- Compiling DSP firmware
- Loading firmware via remoteproc
- Testing RPMsg communication
- Audio processing examples

Both documents are available on NXP's website or through DSO's internal documentation system.

8.4 Summary

8.4.1 What Was Discovered

From reviewing documentation:

- DSP can run its own OS (Zephyr RTOS) independently
- RPMsg/remoteproc framework mirrors M7 implementation
- NXP provides good documentation and examples
- Audio processing libraries are readily available
- Power management is automatic (similar to NPU, M7)

8.5 What I think could be potential Use Cases

8.5.1 Tasks that could be offloaded to dsp

While not explored, potential applications include:

Communication systems:

- Radio signal processing

Sensor fusion:

- Acoustic localization (sound source detection)
- Ultrasonic obstacle detection processing
- Vibration analysis for health monitoring (Hao Karh's work)

Payload applications:

- Audio surveillance
- Environmental sound monitoring
- Acoustic signature analysis

Offloading from A53:

- Even if not using audio, DSP can handle signal processing tasks
- Frees up A53 cores for computer vision or flight control
- More power-efficient than CPU for repetitive DSP operations

8.5.2 Learning Resources

Related Documentation

Essential Reading:

- **UG10167:** i.MX DSP User's Guide (comprehensive reference)
- **AN13970:** Zephyr RTOS on HiFi 4 DSP (step-by-step tutorial)
- **Zephyr Docs:** <https://docs.zephyrproject.org/>
- **Cadence HiFi 4:** https://www.cadence.com/en_US/home/tools/ip/tensilica-ip/hifi-dsps.html

NXP Community:

- i.MX Processors forum (search for "DSP" or "HiFi 4")
- Example projects and troubleshooting threads

Chapter 9

M7 Core Development

Important Note

Chapter Scope:

This chapter documents the complete M7 (Cortex-M7) core development work conducted during this internship. The M7 core is essential for:

- Real-time flight control operations
- Low-latency sensor processing
- Running independently while A53 cores are suspended
- Deterministic interrupt handling

What was accomplished:

- Complete NuttX RTOS port to M7 core
- Working low-power suspend mode with A53-M7 coordination
- Peripheral drivers (I2C, UART, PWM, DMA, CAN, GPT)
- Memory architecture optimization (TCM vs DDR)
- Inter-processor communication via RPMsg

What remains incomplete:

- PX4 flight controller software port (HIGH PRIORITY for next intern)
- Full RPMsg implementation for sensor data streaming
- Performance benchmarking under flight control loads

9.1 M7 Core Overview

9.1.1 What is the M7 Core?

The i.MX8M Plus contains a Cortex-M7 core alongside four Cortex-A53 cores, forming a heterogeneous multiprocessing system.

Hardware Items

Cortex-M7 Specifications:

- **Clock:** 800 MHz maximum
- **Architecture:** ARMv7E-M with DSP extensions
- **Instruction TCM:** 128 KB (ultra-fast code execution)
- **Data TCM:** 128 KB (zero-wait-state data access)
- **Cache:** 32 KB I-cache, 32 KB D-cache
- **FPU:** Double-precision floating-point unit
- **MPU:** Memory Protection Unit for safety

9.1.2 Why M7 for Flight Control

Advantages over A53 for real-time tasks:

- **Deterministic timing:** No MMU, no context switching overhead
- **Low latency:** Direct interrupt handling, no OS scheduler delays
- **Low power:** Can run from TCM while DDR is off (suspend mode)
- **Independent operation:** Continues running when A53 cores sleep
- **Real-time OS:** NuttX RTOS provides deterministic task scheduling

Flight control requirements that demand M7:

- Attitude control loops at 400-1000 Hz
- Sensor fusion (IMU, GPS, barometer) at 100-400 Hz
- Motor mixing and ESC updates at 400 Hz minimum
- Emergency handling (< 1 ms response to critical events)

9.1.3 Heterogeneous Architecture

i.MX8M Plus Processing Elements:

+-----+	
4x ARM Cortex-A53 (1.8 GHz)	
- Linux (Yocto)	
- Application processing	
- Computer vision (NPU delegate)	
- Non-real-time tasks	
+-----+	
RPMsg / Shared Memory / MU (Messaging Unit)	
+-----+	
1x ARM Cortex-M7 (800 MHz)	
- NuttX RTOS	
- Flight control	
- Real-time sensor processing	
- Runs from TCM (DDR-independent)	
+-----+	

Communication mechanisms:

- **RPMsg:** Message passing protocol over shared memory
- **MU (Messaging Unit):** Hardware mailbox for interrupts
- **Shared DDR:** Large data buffers for sensor streams
- **GIR (General Interrupt Request):** M7 can wake A53 from suspend

9.2 Development Workflow with MCUXpresso

Important Note

Recommended Development Method:

This section describes the **easiest and fastest** way to develop M7 firmware using MCUXpresso for VS Code with JTAG debugging.

Why I fell this method is better:

- Real-time debugging with breakpoints
- Instant code reload (zero copying of ELF files)
- Step-through debugging
- Variable inspection
- No Linux remoteproc complexity during development

Alternative method (Linux remoteproc):

- Useful for production deployment
- Required when J-Link is not available
- Slower iteration (copy ELF, reload, no debugging)
- Covered later in Section [9.4](#)

Start with MCUXpresso + JTAG for development, then deploy via remoteproc for production.

9.2.1 MCUXpresso SDK Builder (Online)

The MCUXpresso SDK must be built online with your specific board and middleware selections.

Step 1: Navigate to SDK Builder

- Go to: <https://mcuxpresso.nxp.com/en/select>
- Sign in with NXP account (free registration)

Step 2: Select Development Board

1. Search for processor: **MIMX8ML8xxxKZ**
2. Select: "MIMX8ML8xxxKZ - Cortex-M7 core"
3. Board: i.MX 8M Plus EVK (or custom board)
4. Click "Build MCUXpresso SDK"

Step 3: Select SDK Components

On the SDK Builder configuration page, select components:

Middleware (choose based on needs):

- **Multicore:** RPMsg, Remote Processor Messaging (essential for A53-M7 communication)
- **FreeRTOS/NuttX:** RTOS support (if using RTOS)
- **USB:** USB device/host stacks (if needed)
- **lwIP:** Lightweight IP stack (if networking needed)
- **FatFS:** File system (if SD card access)

Examples to include (or just simply select all that is included):

- Driver examples: I2C, UART, SPI, PWM, CAN, GPT, SDMA
- RTOS examples: Task scheduling, semaphores, queues
- Multicore examples: RPMsg string echo, ping-pong
- Demo apps: Complete application templates

Step 4: Generate and Download

1. Review selections
2. Click "Download SDK Archive"
3. Save `SDK_2.x.x_MIMX8ML8xxxxx.zip`
4. Extract to workspace (e.g., `~/mcuxpresso-sdk/`)

 GitHub Repository**Pre-Built SDK Archive (Used During Internship):**

<https://mcuxpresso.nxp.com/download/96dab09fa2d71336c91f91c0d1a6ee32>

This archive contains the exact SDK configuration used for all examples in this chapter. Includes:

- All peripheral driver examples (I2C, UART, PWM, CAN, DMA)
- RPMsg multicore examples
- Pin mux configurations for i.MX8MP EVK
- Linker scripts for TCM and DDR

You can use this as reference or build your own with different middleware selections.

9.2.2 SDK Directory Structure

After extraction, the SDK has this structure:

```

SDK_2.x.x_MIMX8ML8xxxxx/
|
|-- boards/evkmimx8mp/
|   |-- driver_examples/      # Peripheral examples
|   |   |-- i2c/
|   |   |-- uart/
|   |   |-- pwm/
|   |   |-- flexcan/
|   |   |-- sdma/
|   |   +-- gpt/
|   |
|   |-- rtos_examples/        # FreeRTOS/NuttX examples
|   +-- demo_apps/           # Complete applications
|
|-- devices/MIMX8ML8/
|   |-- drivers/              # HAL drivers
|   |   |-- fsl_i2c.c/h
|   |   |-- fsl_uart.c/h
|   |   |-- fsl_pwm.c/h
|   |   +-- fsl_sdma.c/h
|   |
|   |-- cmsis/                # ARM CMSIS headers
|   +-- system_MIMX8ML8_cm7.c # System initialization
|
+-- middleware/
|   |-- multicore/            # RPMsg library
|   |   +-- rpmsg_lite/
|   +-- freertos/            # FreeRTOS kernel (if selected)

```

Key driver files:

- `fsl_i2c.h/.c` - I2C master/slave
- `fsl_uart.h/.c` - UART communication
- `fsl_sdma.h/.c` - DMA transfers
- `fsl_pwm.h/.c` - PWM generation
- `fsl_flexcan.h/.c` - CAN bus
- `fsl_gpt.h/.c` - General Purpose Timers
- `fsl_iomuxc.h` - Pin muxing

9.2.3 MCUXpresso for VS Code Setup

Step 1: Install VS Code Extension

1. Open Visual Studio Code
2. Go to Extensions (Ctrl+Shift+X)
3. Search: "MCUXpresso for VS Code"
4. Install the official NXP extension
5. Restart VS Code

Step 2: Import SDK Repository

1. Open MCUXpresso extension panel (left sidebar)
2. Click "Installed Repositories"
3. Click "+" to add repository
4. Navigate to extracted SDK folder: `SDK_2.x.x_MIMX8ML8xxxxx/`
5. Select folder and import
6. SDK should appear in "Repositories" list

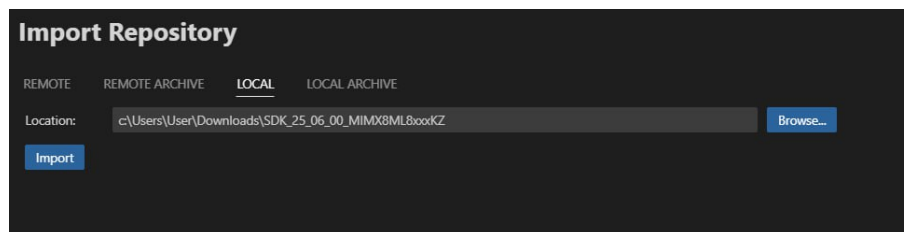


Figure 9.1: Import Repo

Step 3: Configure Debug Probe

During this internship, **Segger J-Link** was used for JTAG debugging.

1. At the bottom below projects, you are able to see the different debug probes available
2. Debug Probe: Select "SEGGER J-Link"
3. Vscode should prompt you about mcuxpresso installer. Use it to download all the necessary tools needed such as MCUXpresso sdk developer and the j link debug probes software pack
4. Verify connection: "Debug Adapter" → "Test Connection"

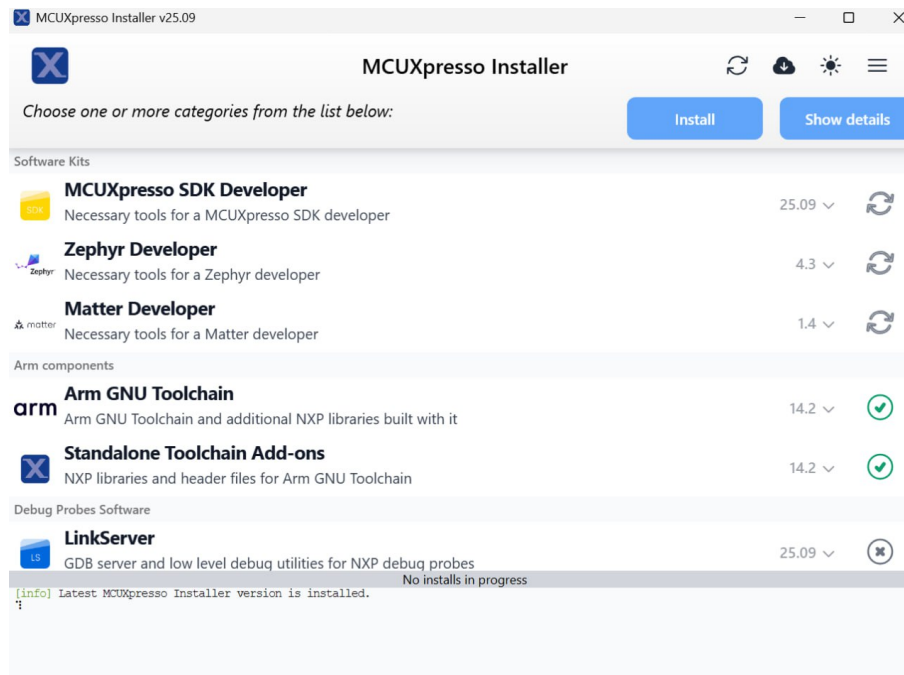


Figure 9.2: mcuxpresso installer

Supported debug probes:

- Segger J-Link (used during internship)
- NXP LPC-Link2
- PEmicro OpenSDA
- CMSIS-DAP

9.2.4 Starting a New Project from Example**Example: Starting a PWM project**

1. MCUXpresso extension → "Import Example Project"
2. Repository: Select your imported SDK
3. Board: evkmimx8mp
4. Navigate: `driver_examples > pwm > pwm`
5. Click "Import"
6. Project appears in workspace

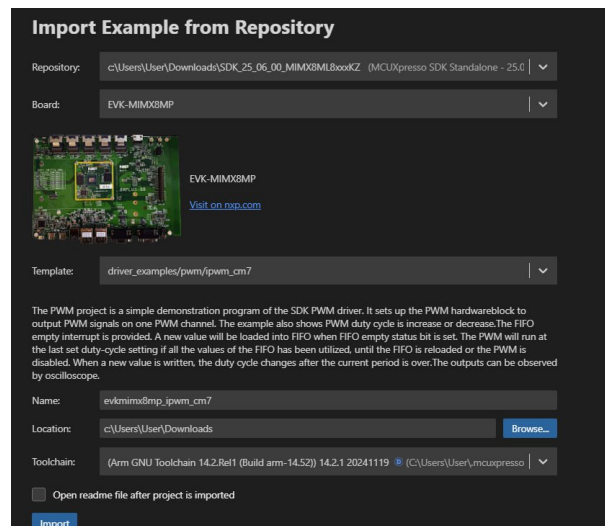


Figure 9.3: PWM example

The imported project contains:

```
pwm_example/
|-- source/
|   |-- main.c           # Main application code
|   |-- pin_mux.c        # Pin configuration (IOMUXC)
|   |-- pin_mux.h
|   |-- board.c          # Board initialization
|   |-- board.h
|   +-- clock_config.c   # Clock tree configuration
|
|-- armgcc/
|   |-- CMakeLists.txt   # Build configuration
|   +-- MIMX8ML8xxxxx_cm7_ram.ld # Linker script
|
+-- .vscode/
    |-- launch.json      # Debug configuration
    +-- tasks.json       # Build tasks
```

 Important Note**Finding the Linker Script:**

The linker script is located at:

```
armgcc/MIMX8ML8xxxxx_cm7_ram.ld
```

You can view and edit it directly in VS Code. This file defines:

- Memory regions (ITCM, DTCM, DDR)
- Section placement (.text, .data, .bss)
- Stack and heap sizes

See Section [9.3.5](#) for detailed linker script configuration and testing.

9.2.5 Key Files to Modify

When working with examples, these are the main files you'll edit:

1. `main.c/pwm.c (examplename.c)` - Application Logic

Contains:

- `main()` function
- Peripheral initialization
- Application loop
- Interrupt handlers

2. `pin_mux.c` - Pin Configuration

Contains:

- `BOARD_InitPins()` - IOMUXC configuration
- Pin mux settings (ALT functions)
- Pad electrical properties (pull-ups, drive strength)

Example from I2C:

```
1 void BOARD_InitPins(void) {  
2     IOMUXC_SetPinMux(IOMUXC_I2C3_SCL_I2C3_SCL, 1U);  
3     IOMUXC_SetPinConfig(IOMUXC_I2C3_SCL_I2C3_SCL,  
4         IOMUXC_SW_PAD_CTL_PAD_ODE_MASK | /* ... */);  
5 }
```

3. `board.c` - Board Initialization

Contains:

- `BOARD_InitHardware()` - Calls all init functions
- `BOARD_InitDebugConsole()` - UART console setup

- Clock initialization
- System configuration

4. MIMX8ML8xxxxx_cm7_ram.ld - Linker Script

Located in `armgcc/` folder. Contains:

- Memory region definitions (ITCM, DTCM, DDR)
- Section placement (`.text`, `.data`, `.bss`, `.ncache`)
- Stack and heap sizes

9.2.6 Building and Debugging

Build Process:

1. Open project in VS Code
2. Terminal → Run Build Task (Ctrl+Shift+B)
3. Or: MCUXpresso extension → "Build"
4. Output: `armgcc/release/pwm_example.elf`

Check build output for memory usage:

1	Memory region	Used Size	Region Size	%age Used
2	m_interrupts:	680 B	1 KB	66.41%
3	m_text:	18536 B	127 KB	14.25%
4	m_data:	2184 B	128 KB	1.67%
5	m_data2:	0 B	16 MB	0.00%

Debug Process:

1. Connect J-Link to EVK board JTAG header
2. Power on EVK board
3. VS Code → Run and Debug (Ctrl+Shift+D)
4. Select "Debug (J-Link)"
5. Press F5 or click "Start Debugging"

What happens when you press Debug/F5:

1. J-Link connects to M7 core via JTAG
2. Firmware is loaded directly to TCM/DDR
3. M7 is reset and halted at `main()`
4. Debugger attaches
5. You can now:

- Set breakpoints (click left of line number)
- Step through code (F10=step over, F11=step into)
- Inspect variables (hover or watch window)
- View registers
- Monitor UART output (using mobaxterm)
- Modify variables in real-time

Common debugging workflow:

```
1 int main(void) {  
2     BOARD_InitHardware(); // <- Set breakpoint here (click line number)  
3     PRINTF("Starting...\r\n");  
4  
5     // Initialize peripheral  
6     I2C_MasterInit(DEMO_I2C, &config, srcClkHz);  
7  
8     // Your code  
9     while (1) {  
10        // Step through with F10 to debug logic  
11        read_sensor();  
12        process_data();  
13    }  
14 }
```

Advantages over remoteproc method:

- **Instant reload:** Press F5, code reloads in 2 seconds
- **No file copying:** No need to SCP ELF to board
- **Real debugging:** Breakpoints, step-through, variable inspection
- **Fast iteration:** Edit → Build → Debug cycle is ~10 seconds

9.2.7 Working with GitHub Examples

All examples from this internship are available on GitHub:

GitHub Repository

GitHub Repository:

<https://github.com/yulyuri/firmware-and-dts>

Contains:

- Complete working examples (I2C, UART, PWM, CAN, DMA)
- Device tree files
- Linker scripts
- Pin mux configurations

To use an example:

1. Try to download the example code it was based from
2. Build project (Ctrl+Shift+B)
3. Examine the key files and simply copy and paste over them !:
 - `source/main.c` - Application logic
 - `source/pin_mux.c` - Pin configuration
 - `source/board.c` - Hardware initialization
 - `armgcc/MIMX8ML8xxxxx_cm7_ram.ld` - Linker script (only relevant to the linker script example)
4. Debug with J-Link (F5)

9.2.8 My Typical Development Cycle

1. **Start from example:** Import closest matching example from SDK
2. **Modify pins:** Edit `pin_mux.c` for your hardware
3. **Implement logic:** Edit `main.c` for your application
4. **Build:** Ctrl+Shift+B (watch for memory usage)
5. **Debug:** F5 with J-Link connected
6. **Test:** Verify functionality with PRINTF functions
7. **Iterate:** Fix bugs, optimize, repeat steps 4-6
8. **Deploy:** When stable, deploy via remoteproc (Section 9.4)

Pro tips:

- Always start from working SDK example
- Make incremental changes and test frequently

- Use PRINTF liberally for debugging
- Check memory usage after every build
- Keep backup of working versions
- Document pin assignments and configurations
- Use Git for version control

9.2.9 Printf and Floating Point

By default, printf doesn't support floating point to save code size.

To enable float printf:

Add to CMakeLists.txt:

```
1 target_compile_definitions(${MCUX_SDK_PROJECT_NAME}  
2     PRIVATE PRINTF_FLOAT_ENABLE=1)
```

Include stdio explicitly in main.c:

```
1 #include <stdio.h>  
2  
3 // Now works:  
4 float temp = 25.6;  
5 printf("Temperature: %.2f C\n", temp);
```

Related Documentation

Printf Float Support:

<https://community.nxp.com/t5/MCUXpresso-IDE/To-complement-solution-how-to-printf-float-p/1431699>

Detailed guide on enabling float formatting in fsl_debug_console.

9.3 Memory Architecture

9.3.1 Complete Memory Map

Understanding the memory map is critical for M7 development:

Address Range	Size	Purpose
M7 Fast Memory (TCM)		
0x00000000 - 0x0001FFFF	128 KB	ITCM (code execution)
0x20000000 - 0x2001FFFF	128 KB	DTCM (data, stack, heap)
Shared On-Chip RAM		
0x00900000 - 0x0096FFFF	448 KB	OCRAM (shared with A53)
DDR Memory Regions		
0x40000000 - 0x54FFFFFF	336 MB	Linux reserved (A53 only)
0x55000000 - 0x550FFFFFF	1 MB	RPMsg communication
0x80000000 - 0x80FFFFFF	16 MB	M7 firmware & data (Non-Cacheable)
0x90000000+	3.5 GB	Linux system memory
Peripherals		
0x30000000 - 0x3FFFFFFF	—	Hardware registers (UART, I2C, etc.)

9.3.2 TCM: Tightly Coupled Memory

What is TCM?

TCM (Tightly Coupled Memory) is ultra-fast, deterministic memory directly connected to the M7 core:

- **Zero wait states:** Single-cycle access (no cache misses)
- **Deterministic timing:** Critical for real-time control loops
- **Power efficient:** Stays powered in suspend mode
- **DDR-independent:** M7 runs even when DDR is in self-refresh

TCM is split into two regions:

Region	Address	Size	Contents
ITCM	0x00000000	128 KB	Code (.text, .rodata)
DTCM	0x20000000	128 KB	Data (.data, .bss, stack, heap)

Important Note

Why TCM is Critical for Suspend Mode:

If M7 firmware runs from DDR and the system enters suspend:

- DDR enters self-refresh mode (powered down)
- M7 cannot fetch instructions → crash
- M7 cannot access data → undefined behavior

Solution: Flight control firmware MUST run from TCM to support low-power operation while A53 is suspended.

9.3.3 DDR Memory for M7

Only 16 MB of DDR is reserved for M7 use (configured in device tree):

DDR allocation:

- Total DDR: 4 GB
- Linux (A53): 3.9 GB
- M7 reserved: 16 MB at 0x80000000
- RPMMsg buffers: 4 MB at 0x55000000

When to use DDR:

- Large buffers (> 128 KB) that don't fit in TCM
- DMA buffers (must be NonCacheable)
- Shared data with A53 cores
- Sensor data logs
- Camera frame buffers

Important: DDR is NOT used automatically

- Normal variables do NOT overflow to DDR
- Only explicit NonCacheable sections go to DDR
- If DTCM fills up (>128 KB), build fails

9.3.4 Cacheable vs NonCacheable Memory

What is Cache?

Cache is super-fast memory between CPU and slower DDR:

CPU → Cache Hit (fast, 0-1 cycle)

CPU → Cache Miss → DDR (slow, 100+ cycles)

Cacheable Data (Default):

```
1 int sensor_value = 123; // Cacheable by default
```

- CPU reads from DDR → copies to cache
- Next read is super fast from cache
- Good for: Normal variables only M7 uses

NonCacheable Data (Special Cases):

```
1 __attribute__((section("NonCacheable")))
2 uint8_t shared_buffer[1024]; // NonCacheable
```

- CPU reads directly from DDR (no cache)
- Slower (100+ cycles per access)
- Good for: Shared data or DMA buffers

When to use NonCacheable:

1. RPMsg buffers: Shared between M7 and A53

- Both cores see changes immediately
- No cache coherency issues

2. DMA buffers: Hardware writes directly

- DMA writes to DDR
- M7 sees DMA updates immediately

3. Large buffers: Too big for DTCM

- Camera frames
- Large sensor logs
- Audio/video buffers

Important Note

Cache Coherency Problems:

Without NonCacheable attribute:

- CPU writes data → stays in cache → DMA reads old RAM value
- DMA writes data → in RAM only → CPU reads old cached value

Solution: Always use NonCacheable for DMA and shared buffers!

9.3.5 Linker Script Configuration

The linker script (MIMX8ML8xxxxx_cm7_ram.ld) defines memory regions and section placement. Please take note that the default linker script differs from example to example

Location: armgcc/MIMX8ML8xxxxx_cm7_ram.ld

Memory region definitions:

```

1 MEMORY
2 {
3     m_interrupts (RX) : ORIGIN = 0x00000000, LENGTH = 0x00000400 // 1 KB
4     m_text       (RX) : ORIGIN = 0x00000400, LENGTH = 0x0001FC00 // 127 KB
5     m_data       (RW) : ORIGIN = 0x20000000, LENGTH = 0x00020000 // 128 KB
6     m_data2      (RW) : ORIGIN = 0x80000000, LENGTH = 0x01000000 // 16 MB
7 }
```

Region	Address	Size	Purpose
m_interrupts	0x00000000	1 KB	Interrupt Vector Table (ITCM)
m_text	0x00000400	127 KB	Program Code & Constants (ITCM)
m_data	0x20000000	128 KB	Variables, Stack, Heap (DTCM)
m_data2	0x80000000	16 MB	NonCacheable / DMA Data (DDR)

Standard section placement:

```

1 SECTIONS
2 {
3     /* Code ITCM */
4     .text :
5     {
6         KEEP(*(.isr_vector)) /* Interrupt vectors */
7         *(.text*)           /* All functions */
8         *(.rodata*)         /* Constants */
9     } > m_text
10
11     /* Initialized variables DTCM */
12     .data :
13     {
14         *(.data*)           /* e.g., int x = 42; */
15     } > m_data
16
17     /* Uninitialized variables DTCM */
18     .bss :
19     {
20         *(.bss*)           /* e.g., int y; */
21     } > m_data
22
23     /* NonCacheable/DMA DDR */
24     .ncache :
25     {
26         *(NonCacheable)
27     } > m_data2
28
29     /* Stack DTCM */
30     .stack :
31     {
32         . = ALIGN(8);
33         . += STACK_SIZE;
34     } > m_data
35
36     /* Heap DTCM */
37     .heap :
38     {
39         . = ALIGN(8);
40         . += HEAP_SIZE;

```

```

41 } > m_data
42 }

```

Custom sections for DDR code/data (Added during testing):

To enable selective placement of code in DDR, these sections were added:

```

1 SECTIONS
2 {
3     /* ... standard sections above ... */
4
5     /* Code placed in DDR (for large, non-critical functions) */
6     .text_dds :
7     {
8         . = ALIGN(4);
9         *(.text_dds)
10        *(.text_dds*)
11        . = ALIGN(4);
12    } > m_data2
13
14    /* Read-only data in DDR (for large lookup tables, constants) */
15    .rodata_dds :
16    {
17        . = ALIGN(4);
18        *(.rodata_dds)
19        *(.rodata_dds*)
20        . = ALIGN(4);
21    } > m_data2
22
23    /* ... rest of sections ... */
24 }

```

Stack and Heap Configuration:

Default sizes:

```

1 HEAP_SIZE = 0x0400; // 1 KB
2 STACK_SIZE = 0x0400; // 1 KB

```

Recommended for flight control:

```

1 HEAP_SIZE = 0x2000; // 8 KB
2 STACK_SIZE = 0x2000; // 8 KB

```

Checking Memory Usage After Build:

After compilation, terminal shows:

Memory region	Used Size	Region Size	%age Used	
m_interrupts:	680 B	1 KB	66.41%	
m_text:	18536 B	127 KB	14.25%	<- Code in TCM
m_data:	2184 B	128 KB	1.67%	<- Variables in TCM
m_data2:	2048 B	16 MB	0.01%	<- DDR (if .text_dds used)

⚠ Important Note**Memory Overflow:**

If you exceed DTCM (128 KB):

- Build will fail with "region overflow" error
- Move large buffers to DDR using custom sections
- Or reduce buffer sizes
- Cannot automatically overflow to DDR

9.3.6 Linker Script Testing: TCM vs DDR Performance

During development, comprehensive testing was conducted to quantify the performance difference between TCM and DDR execution.

🔗 GitHub Repository**Complete Test Code:**

https://github.com/yulyuri/firmware-and-dts/tree/master/M7/linker_script

Contains full source code for all three benchmark tests with detailed comments.

Test Setup and Macros

To enable easy comparison, the following macros were defined:

```
1  /* Placement control macros */
2  #define FAST_CODE    /* Nothing - default is TCM */
3  #define SLOW_CODE    __attribute__((section(".text_dds")))
4  #define SLOW_DATA    __attribute__((section(".rodata_dds")))
```

How these macros work:

- FAST_CODE: Empty macro, functions default to .text section → ITCM
- SLOW_CODE: Tells linker to place function in .text_dds section → DDR
- SLOW_DATA: Tells linker to place data in .rodata_dds section → DDR

Timing measurement:

```
1  // Enable DWT cycle counter
2  static inline void start_timer(void) {
3      CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
4      DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
5      DWT->CYCCNT = 0;
6  }
7
8  // Read cycle count
9  static inline uint32_t get_cycles(void) {
10     return DWT->CYCCNT; // Returns CPU cycles elapsed
```

11 }

At 800 MHz, each cycle = 1.25 ns. This provides precise microsecond-level timing.

Test 1: Simple Loop (Integer Arithmetic)

Test code:

```

1 FAST_CODE
2 uint32_t simple_loop_fast(uint32_t count) {
3     uint32_t sum = 0;
4     for (uint32_t i = 0; i < count; i++) {
5         sum += i;
6     }
7     return sum;
8 }
9
10 SLOW_CODE
11 uint32_t simple_loop_slow(uint32_t count) {
12     uint32_t sum = 0;
13     for (uint32_t i = 0; i < count; i++) {
14         sum += i;
15     }
16     return sum;
17 }
18
19 // Test with 5000 iterations

```

Results:

Code in TCM was 22x faster

Test 2: FFT Computation (Floating Point)

Test code:

256-point Fast Fourier Transform with bit-reversal and butterfly operations:

```

1 FAST_CODE
2 void compute_fft_fast(complex_t *data, uint32_t n) {
3     // Bit reversal reordering
4     // Butterfly computation with twiddle factors
5     // Multiple stages with sine/cosine calculations
6 }
7
8 SLOW_CODE
9 void compute_fft_slow(complex_t *data, uint32_t n) {
10     // Identical algorithm, different memory location
11 }

```

Results:

Code in TCM was 7.8x faster

Test 3: Array Access (Sequential Read)**Test code:**

```

1 float array_fast[500]; // Default DTCM
2
3 SLOW_DATA const float array_slow[500] = {[0 ... 499] = 1.0f}; // DDR
4
5 // Read all elements sequentially
6 volatile float sum = 0;
7 for (int i = 0; i < 500; i++) {
8     sum += array[i];
9 }

```

Results:

Code in TCM was 7x faster

Performance Analysis**The possible reasons why I think TCM is faster:****1. Zero wait states:**

- TCM: Single-cycle access (1 cycle = 1.25 ns)
- DDR: 100-200+ cycles per cache miss

2. No cache misses:

- TCM: Always hit (deterministic)
- DDR: Cache misses cause stalls

3. Instruction fetch:

- TCM: Fetch and execute simultaneously
- DDR: Pipeline stalls on cache misses

4. DDR refresh cycles:

- DDR must periodically refresh (unavailable during refresh)
- TCM: Always available

Test-specific observations (might be wrong, please verify):

- **Test 1 (22x speedup):** Small loop fits entirely in instruction cache initially, but frequent jumps cause cache thrashing in DDR version. TCM has no such penalty.
- **Test 2 (7.8x speedup):** FFT has complex control flow (nested loops, bit reversal). Many instruction cache misses in DDR. Large code size exceeds cache capacity.
- **Test 3 (7x speedup):** Sequential array access benefits from DDR burst mode and prefetching, which partially mitigates latency. Still slower than TCM's zero-wait access.

! Important Note**Remaining Investigation Needed:**

Several aspects were not fully explored during this internship:

1. Cache behavior under different workloads:

- Why does Test 1 show 22x difference but Test 2 only 7.8x?
- How does cache size (32 KB I-cache, 32 KB D-cache) affect each test?
- What is the exact cache miss rate for each test?

2. Mixed TCM/DDR strategies, when exactly we use what?:

- Critical loops in TCM, everything else in DDR?
- Can interrupt handlers in TCM call DDR functions safely?
- What is the overhead of TCMDDR function calls?

Possible design for flight control in the future based on results:

Based on these results:

• Always in TCM:

- Interrupt handlers
- Control loops (PID, motor mixing)
- Sensor reading functions
- Safety-critical code

• Can be in DDR:

- Telemetry formatting
- Logging functions
- Configuration parsing
- Non-real-time diagnostics

• Measured decision needed:

- State estimation (Kalman filters) - depends on complexity
- Path planning - depends on update rate required
- Sensor fusion - measure actual performance

Summary: TCM provides 7-22x performance improvement depending on workload characteristics. For deterministic real-time performance, keep critical code in TCM. The exact reasons for variation between tests require further investigation with cache profiling tools.

9.4 Device Tree and Boot Methods

9.4.1 Device Tree Configuration

The M7 core must be defined in the Linux device tree for proper resource allocation. You can refer to the default imx8mp rpmsg dts:

```

1  imx8mp-cm7 {
2      compatible = "fsl,imx8mn-cm7"; // Used for both SMN and SMP
3      rsc-da = <0x55000000>; // Resource table address
4      clocks = <&clk IMX8MP_CLK_M7_DIV>;
5      mbox-names = "tx", "rx", "rxdb";
6      mbox-es = <&mu 0 1>, <&mu 1 1>, <&mu 3 1>; // MU channels
7      memory-region = <&vdev0vring0>, <&vdev0vring1>, <&vdevbuffer>,
8                      <&m4_reserved>, <&m7_ddr_alias>,
9                      <&m7_itcm>, <&m7_dtcn>;
10     status = "okay";
11 };

```

9.4.2 Reserved Memory Regions

Critical memory regions must be reserved for M7 use:

```

1  reserved-memory {
2      // Main DDR region for M7 firmware and data
3      m4_reserved: m4@0x80000000 {
4          no-map;
5          reg = <0 0x80000000 0 0x1000000>; // 16 MB in DDR
6      };
7
8      // Instruction Tightly Coupled Memory
9      m7_itcm: m4@0x7E0000 {
10         no-map;
11         reg = <0 0x7E0000 0 0x20000>; // 128 KB ITCM
12     };
13
14     // Data Tightly Coupled Memory
15     m7_dtcn: m4@0x800000 {
16         no-map;
17         reg = <0 0x800000 0 0x20000>; // 128 KB DTCM
18     };
19
20     // RPMsg communication buffers
21     vdev0vring0: vdev0vring0@55000000 {
22         reg = <0 0x55000000 0 0x8000>; // 32 KB TX ring
23     };
24
25     vdev0vring1: vdev0vring1@55008000 {
26         reg = <0 0x55008000 0 0x8000>; // 32 KB RX ring
27     };
28 };

```

```

29     vdevbuffer: vdevbuffer@55400000 {
30         reg = <0 0x55400000 0 0x100000>; // 1 MB shared buffer
31     };
32 };

```

GitHub Repository

Device Tree Files:

Complete default device tree with M7 configuration:

<https://github.com/yulyuri/firmware-and-dts/blob/master/SCP%20files/imx8mp-evk-rpmsg.dts>

9.4.3 Understanding Remoteproc and RPMsg

Remoteproc (Remote Processor):

- Linux kernel framework for managing remote processors
- Handles M7 firmware loading from `/lib/firmware/`
- Controls M7 power state (start, stop, restart)
- Manages resource table and memory allocation
- Exposed via sysfs: `/sys/class/remoteproc/`

RPMsg (Remote Processor Messaging):

- Communication protocol between A53 (Linux) and M7 (RTOS)
- Built on top of VirtIO shared memory buffers
- Provides virtual TTY devices: `/dev/ttyRPMMSG*`
- Bidirectional message passing
- Examples: string echo, command/response, sensor data streaming

9.4.4 M7 Boot Sequence

The M7 can be started at different points in the boot process:

Option 1: Boot from U-Boot (Early Start)

```

1 # In U-Boot console, before booting Linux:
2 u-boot=> run prepare_mcore
3 u-boot=> boot

```

What `prepare_mcore` does:

- Loads M7 firmware from SD card or eMMC
- Sets shared memory flag indicating M7 is pre-started
- Enables M7 clock

- Starts M7 execution
- Linux remoteproc detects M7 is already running

Option 2: Boot from Linux (Runtime Start)

This is the method you use when NOT debugging with J-Link.

```

1 # Check available remoteproc instances
2 ls /sys/class/remoteproc/
3 # Typically: remoteproc0 (M7), remoteproc1 (DSP)
4
5 # Load firmware and start M7
6 echo your_firmware.elf > /sys/class/remoteproc/remoteproc0/firmware
7 echo start > /sys/class/remoteproc/remoteproc0/state
8
9 # Check status
10 cat /sys/class/remoteproc/remoteproc0/state
11 # Should show: running

```

Important Note

Clock Dependency Issue:

From NXP AN5317: "For i.MX 8M platforms, the root clock for M7 must be kept always enabled by Linux to load firmware and start Cortex-M7."

If starting M7 from Linux remoteproc:

- Linux clock driver may gate M7 clock by default
- You must modify kernel driver: `drivers/clk/imx/clk-composite-8m.c`
- Skip gate registration for M7's root clock
- Otherwise, M7 will fail to start or crash unexpectedly

If starting M7 from U-Boot:

- Clock is enabled during `prepare_mcore`
- Linux inherits the clock state
- No kernel driver modification needed
- **Recommended approach for flight control applications**

9.4.5 Initial Testing with NXP SDK Examples

NXP provides pre-built example firmware:

```

1 # Check available firmware
2 ls /lib/firmware/
3 # Look for: imx8mp_m7_TCM_rpmsg_lite_str_echo_rtos.elf
4
5 # Boot from U-Boot:

```

```

6 u-boot=> run prepare_mcore
7 u-boot=> boot
8
9 # After Linux boots, load RPMsg TTY driver
10 modprobe imx_rpmsg_tty
11
12 # Check for RPMsg devices
13 ls /dev/ttyRPMMSG*
14 # Should see: /dev/ttyRPMMSG0
15
16 # Test string echo
17 echo "hello" > /dev/ttyRPMMSG0
18 cat /dev/ttyRPMMSG0
19 # Should echo back: hello

```

Verify M7 status:

```

1 # Check remoteproc state
2 cat /sys/class/remoteproc/remoteproc0/state
3 # If U-Boot started M7: shows "detached" or "running"
4 # If not started: shows "offline"
5
6 # Check kernel messages
7 dmesg | grep rpmsg
8 # Should see messages about RPMsg channel creation

```

9.4.6 Deploying Your Firmware

After developing with J-Link, deploy to production:

```

1 # Copy ELF from development machine to board
2 scp elf_file_path root@192.168.1.100:/lib/firmware/
3
4 # On target board (via SSH or serial console)
5 echo my_flight_control.elf > /sys/class/remoteproc/remoteproc0/firmware
6 echo start > /sys/class/remoteproc/remoteproc0/state
7
8 # Verify running
9 cat /sys/class/remoteproc/remoteproc0/state
10 # Should show: running
11
12 # Check M7 console output (if using RPMsg TTY)
13 cat /dev/ttyRPMMSG0
14
15 # Stop M7 (if needed)
16 echo stop > /sys/class/remoteproc/remoteproc0/state

```

Comparison: Development vs Production

Aspect	J-Link (Dev)	Remoteproc (Prod)
Speed	Fast (2 sec reload)	Slow (copy + reload)
Debugging	Full breakpoints	None (use PRINTF)
Iteration	Instant	Manual file copy
Use case	Development	Production/Field
Hardware	J-Link required	No extra hardware

9.5 Peripheral Development

This section covers practical peripheral usage with actual code examples tested during the internship. All working examples are available on GitHub.

Important Note

Security Note - External Device Whitelisting:

Before connecting ANY external devices (oscilloscopes, logic analyzers, USB drives, SD cards) to your development machine:

- Register device with IT Concierge FIRST
- Get device whitelisted before connecting
- Failure to do so triggers security alerts
- Personal SD cards with work files will be flagged by cybersecurity

Lesson learned: My SD card containing Linux images for the M7 was connected without whitelisting and triggered a security incident. Please don't make this stupid mistake!

9.5.1 Pin Muxing (IOMUXC)

Before using any peripheral, pins must be muxed to the correct function.

Pin mux example for I2C3:

```

1 void BOARD_InitPins(void)
2 {
3     // Route pads to I2C3 function (ALT4)
4     IOMUXC_SetPinMux(IOMUXC_I2C3_SCL_I2C3_SCL, 1U); // SION=1
5     IOMUXC_SetPinMux(IOMUXC_I2C3_SDA_I2C3_SDA, 1U);
6
7     // Configure electrical properties for I2C
8     const uint32_t i2c_pad =
9         IOMUXC_SW_PAD_CTL_PAD_ODE_MASK | // Open-drain (required)
10        IOMUXC_SW_PAD_CTL_PAD_FSEL_MASK | // Fast slew
11        IOMUXC_SW_PAD_CTL_PAD_PE_MASK | // Pull enable
12        IOMUXC_SW_PAD_CTL_PAD_PUE_MASK | // Pull-up (not down)
13        IOMUXC_SW_PAD_CTL_PAD_PUS(2U) | // Weak pull-up
14        IOMUXC_SW_PAD_CTL_PAD_DSE(2U) | // Medium drive
15        IOMUXC_SW_PAD_CTL_PAD_HYS_MASK; // Schmitt trigger
16

```

```

17     IOMUXC_SetPinConfig(IOMUXC_I2C3_SCL_I2C3_SCL, i2c_pad);
18     IOMUXC_SetPinConfig(IOMUXC_I2C3_SDA_I2C3_SDA, i2c_pad);
19 }

```

Understanding pad control bits:

- **ODE** (bit 5): Open drain enable - MUST be set for I2C
- **PE** (bit 8): Pull/keeper enable
- **PUE** (bit 6): Pull-up (1) vs keeper (0)
- **HYS** (bit 7): Schmitt trigger - recommended for all I/O
- **FSEL** (bit 4): Fast (1) vs slow (0) slew rate
- **DSE** (bits 2-1): Drive strength (X1/X2/X4/X6)

9.5.2 I2C: Dual IMU Configuration

During testing, two IMUs were connected simultaneously:

- **MPU-6050**: Connected to I2C3
- **ADXL345**: Connected to I2C5

This configuration demonstrates multi-sensor setups typical in flight control systems.

GitHub Repository

Complete I2C + Dual IMU Code:

https://github.com/yulyuri/firmware-and-dts/tree/master/M7/i2c_for_mpu_and_adxl

Contains:

- MPU-6050 driver (I2C3)
- ADXL345 driver (I2C5)
- Simultaneous sensor reading
- Pin mux configuration for both I2C instances

Register definitions:

```

1 // MPU-6050 (I2C3)
2 #define MPU_ADDR_7BIT      0x68u
3 #define MPU_REG_PWR1      0x6B
4 #define MPU_REG_WHOAMI    0x75
5 #define MPU_REG_ACCEL_XOUT 0x3B
6
7 // ADXL345 (I2C5)
8 #define ADXL_ADDR_7BIT     0x53u
9 #define ADXL_REG_DEVID     0x00
10 #define ADXL_REG_POWER_CTL 0x2D
11 #define ADXL_REG_DATAX0    0x32

```

Initialize I2C controller (MPU-6050 example):

```

1 i2c_master_config_t cfg;
2 uint32_t srcClkHz;
3
4 I2C_MasterGetDefaultConfig(&cfg);
5 cfg.baudRate_Bps = 100000u;  // 100 kHz
6
7 srcClkHz = I2C_MASTER_CLK_FREQ;
8 I2C_MasterInit(I2C3, &cfg, srcClkHz);

```

Wake IMU (write to register):

```

1 i2c_master_transfer_t xfer;
2 uint8_t zero = 0x00;
3
4 memset(&xfer, 0, sizeof(xfer));
5 xfer.slaveAddress = MPU_ADDR_7BIT;
6 xfer.direction    = kI2C_Write;
7 xfer.subaddress   = MPU_REG_PWR1;
8 xfer.subaddressSize = 1;
9 xfer.data          = &zero;
10 xfer.dataSize      = 1;
11 xfer.flags         = kI2C_TransferDefaultFlag;
12
13 status_t st = I2C_MasterTransferBlocking(I2C3, &xfer);

```

Read WHO_AM_I:

```

1 uint8_t who = 0xFF;
2
3 memset(&xfer, 0, sizeof(xfer));
4 xfer.slaveAddress = MPU_ADDR_7BIT;
5 xfer.direction    = kI2C_Read;
6 xfer.subaddress   = MPU_REG_WHOAMI;
7 xfer.subaddressSize = 1;
8 xfer.data          = &who;
9 xfer.dataSize      = 1;
10
11 st = I2C_MasterTransferBlocking(I2C3, &xfer);
12 PRINTF("WHO_AM_I = 0x%02X\r\n", who);
13 // MPU-6050: 0x68, MPU-6500: 0x70, etc.

```

Burst read sensor data (14 bytes):

```

1 uint8_t sensor_data[14];
2
3 xfer.subaddress = MPU_REG_ACCEL_XOUT;
4 xfer.data       = sensor_data;
5 xfer.dataSize   = 14;  // AX,AY,AZ,TEMP,GX,GY,GZ
6
7 st = I2C_MasterTransferBlocking(I2C3, &xfer);
8
9 // Parse 16-bit values (big-endian)

```

```

10 int16_t accel_x = (sensor_data[0] << 8) | sensor_data[1];
11 int16_t accel_y = (sensor_data[2] << 8) | sensor_data[3];
12 int16_t accel_z = (sensor_data[4] << 8) | sensor_data[5];
13
14 PRINTF("Accel: X=%d Y=%d Z=%d\r\n", accel_x, accel_y, accel_z);

```

Important notes:

- Check device tree: I2C3/I2C5 nodes must be disabled for Linux
- Verify addresses: MPU-6050 (0x68/0x69), ADXL345 (0x53/0x1D)

9.5.3 PWM for LED Control and Signal Generation

PWM was tested with LED fade effects and signal generation measured with an oscilloscope.

GitHub Repository

Complete PWM Code:

https://github.com/yulyuri/firmware-and-dts/blob/master/M7/PWM_4/pwm.c

This implementation demonstrates:

- Smooth LED breathing effect
- Configurable fade timing
- FIFO interrupt-driven updates
- 1 kHz PWM frequency

Hardware used for testing:

- **Digilent Analog Discovery 2 (AD2):** Used as oscilloscope to measure PWM signals
- Remember to whitelist AD2 with IT Concierge before connecting!

PWM configuration:

```

1 pwm_config_t pwmConfig;
2
3 PWM_GetDefaultConfig(&pwmConfig);
4 pwmConfig.clockSource = kPWM_PeripheralClock; // 24 MHz
5 pwmConfig.prescale = 0U; // No prescaler
6
7 PWM_Init(DEMO_PWM_BASEADDR, &pwmConfig);
8
9 // Set period for ~1 kHz: f = 24MHz / (period + 2)
10 #define PWM_PERIOD_VALUE 24000
11 PWM_SetPeriodValue(DEMO_PWM_BASEADDR, PWM_PERIOD_VALUE);

```

LED breathing effect with interrupt:

```

1 void DEMO_PWM_IRQHandler(void)
2 {
3     if (PWM_GetStatusFlags(DEMO_PWM_BASEADDR) & kPWM_FIFOEmptyFlag)
4     {
5         // Configurable fade parameters
6         enum { DUTY_MIN_PCT = 10, DUTY_MAX_PCT = 90 };
7         enum { FADE_TIME_MS = 250 }; // Time for minmax transition
8         enum { HOLD_MS = 80 }; // Pause at peaks
9
10        const uint32_t dutyMin = (DUTY_MIN_PCT * PWM_PERIOD_VALUE) / 100U;
11        const uint32_t dutyMax = (DUTY_MAX_PCT * PWM_PERIOD_VALUE) / 100U;
12
13        // Calculate step size for smooth fade
14        const uint32_t totalSpan = dutyMax - dutyMin;
15        const uint32_t step = totalSpan / FADE_TIME_MS;
16
17        static bool goingUp = true;
18        static uint32_t hold = 0;
19
20        if (goingUp) {
21            if (pwmDutycycle + step >= dutyMax) {
22                pwmDutycycle = dutyMax;
23                goingUp = false;
24                hold = HOLD_MS;
25            } else {
26                pwmDutycycle += step;
27            }
28        } else {
29            if (pwmDutycycle <= dutyMin + step) {
30                pwmDutycycle = dutyMin;
31                goingUp = true;
32                hold = HOLD_MS;
33            } else {
34                pwmDutycycle -= step;
35            }
36        }
37
38        if (hold) --hold;
39
40        PWM_SetSampleValue(DEMO_PWM_BASEADDR, pwmDutycycle);
41        PWM_clearStatusFlags(DEMO_PWM_BASEADDR, kPWM_FIFOEmptyFlag);
42    }
43 }

```

9.5.4 CAN Bus Communication

CAN bus code was tested using SDK examples with minimal modifications. See SDK documentation for detailed CAN implementation.

Basic CAN configuration:

```

1 flexcan_config_t flexcanConfig;
2
3 FLEXCAN_GetDefaultConfig(&flexcanConfig);
4 flexcanConfig.bitRate = 500000U; // 500 kbit/s
5
6 FLEXCAN_Init(EXAMPLE_CAN, &flexcanConfig, CAN_CLK_FREQ);

```

CAN electrical notes:

- Requires CAN transceiver (e.g., TJA1050, MCP2551)
- Termination resistors (120) at both ends of bus
- All nodes must use same bitrate

9.5.5 GPT: Input Capture for RC Receivers

GPT input capture was tested for reading RC receiver signals (PPM and pulse width timing).

GitHub Repository

Complete GPT Input Capture Code:

https://github.com/yulyuri/firmware-and-dts/tree/master/M7/gpt_capture2

Implements:

- Input capture on rising/falling edges
- Pulse width measurement
- PPM decoding framework
- Microsecond-accurate timing

GPT Input Capture setup:

```

1 gpt_config_t gptConfig;
2
3 GPT_GetDefaultConfig(&gptConfig);
4 gptConfig.clockSource = kGPT_ClockSource_Periph; // 24 MHz
5 gptConfig.divider = 24; // 1 s tick resolution
6
7 GPT_Init(DEMO_GPT, &gptConfig);
8
9 // Configure input capture
10 GPT_SetupInputCapture(DEMO_GPT, kGPT_InputCapture_Channel1,
11                       kGPT_InputCapture_RiseEdge);
12
13 GPT_EnableInterrupts(DEMO_GPT, kGPT_InputCapture1InterruptEnable);
14 EnableIRQ(DEMO_GPT_IRQn);
15
16 GPT_StartTimer(DEMO_GPT);

```

Pulse width measurement:

```

1  #define PPM_SYNC_GAP_US 3000  // Sync pulse threshold
2
3  uint32_t last_capture = 0;
4  uint8_t channel_index = 0;
5  uint16_t rc_channels[8];
6
7  void DEMO_GPT_IRQHandler(void) {
8      if (GPT_GetStatusFlags(DEMO_GPT) & kGPT_InputCapture1Flag) {
9          uint32_t capture = GPT_GetInputCaptureValue(DEMO_GPT,
10                                                       kGPT_InputCapture_Channel1
11          );
12          uint32_t pulse_width = capture - last_capture;
13          last_capture = capture;
14
15          if (pulse_width > PPM_SYNC_GAP_US) {
16              channel_index = 0;  // Sync pulse
17          } else if (channel_index < 8) {
18              rc_channels[channel_index] = pulse_width;
19              channel_index++;
20          }
21
22          GPT_ClearStatusFlags(DEMO_GPT, kGPT_InputCapture1Flag);
23      }
24  }

```

Modern RC protocols:

- **SBUS (UART, 100k baud):** 16 channels, inverted, 14 ms latency
- **CRSF/ELRS (UART, 420k baud):** 16 channels, low latency (5 ms)
- **DSM2/DSMX (UART):** Spektrum ecosystem

9.6 DMA (Direct Memory Access)

DMA allows peripherals to transfer data without CPU intervention. During this internship, DMA was tested using NXP SDK examples with minimal modifications.

9.6.1 Why DMA Matters

Key benefits:

- CPU-free data transfers
- Essential for high-bandwidth peripherals (audio, video)
- Reduces interrupt overhead
- Enables true background operations

9.6.2 SDMA on i.MX8MP

Controller	Typical Use
SDMA-1	UART, SPI, low-speed
SDMA-2	High-bandwidth audio
SDMA-3	Standard audio

9.6.3 Critical: NonCacheable Memory

Important Note

Always use NonCacheable memory for DMA buffers:

```

1 // All DMA buffers MUST be NonCacheable
2 AT_NONCACHEABLE_SECTION_ALIGN(sdma_context_data_t context, 4);
3 AT_NONCACHEABLE_SECTION_ALIGN(uint8_t dma_buffer[1024], 4);

```

Without NonCacheable attribute:

- CPU cache and DMA see different data
- Leads to data corruption
- Very difficult to debug

9.6.4 DMA Examples

Detailed DMA examples (memory-to-memory, UART DMA) are available in the NXP MCUXpresso SDK:

- boards/evkmimx8mp/driver_examples/sdma/memory_to_memory
- boards/evkmimx8mp/driver_examples/uart/sdma_transfer

These examples demonstrate:

- DMA handle creation and callbacks
- Transfer configuration
- Channel priority settings (1-7, never 0!)
- Interrupt-driven completion

The SDK examples work well as-is and provide good starting points for custom DMA implementations.

9.7 Power Management and Suspend Coordination

Low-power operation with M7 running while A53 suspended was successfully implemented and tested.

GitHub Repository

Complete Suspend/Wake Implementation:

<https://github.com/yulyuri/firmware-and-dts/tree/master/M7/Acorewakeup>

Contains:

- M7 firmware with LPA flag setting
- GIR interrupt triggering
- A53 wakeup handling
- Complete power state management

9.7.1 Low-Power Modes

Mode	Cores	Description
Wait	A53, M7	Light sleep, clocks running
Stop	A53, M7	Deep sleep, clocks/PLLs off
DSM	Whole SoC	Deepest, only SNVS_LP powered

9.7.2 M7 Running During A53 Suspend

Requirements for M7 to continue during A53 suspend:

1. M7 firmware MUST run from TCM (not DDR)
2. DDR enters self-refresh during suspend
3. M7 clock source remains active
4. LPA flags properly set in firmware
5. ATF configured to handle M7 active state

Set LPA flag in M7 firmware:

```

1 // Signal ATF that M7 must stay active
2 SRC->GPR10 = 0x5555; // M4_LPA_ACTIVE flag
3
4 // This tells ATF:
5 // - Keep M7 clock enabled
6 // - Don't disable MU (Messaging Unit)
7 // - Keep essential PLLs running
8 // - Unmask GIR for wakeup

```

9.7.3 Waking A53 from M7

M7 can wake suspended A53 using GIR (General Interrupt Request):

From M7 (trigger wake):

```

1 // Trigger GIRO to wake A53
2 MU->CR |= (1 << 16); // Set GIRO bit

```

From A53 (after wake, clear flag):

```

1 // Clear GIR pending status
2 MU->SR |= (1 << 28); // Clear GIPO bit

```

ATF modification (plat/imx/imx8m/gpc_common.c):

```

1 // Unmask MU interrupt only if M7 LPA is active
2 mmio_clrbits_32(IMX_GPC_BASE + gpc_imr_offset[last_core] + 0x8,
3                 BIT(24));

```

9.7.4 Complete Suspend/Resume Flow

1. A53 runs Linux, M7 runs flight control in parallel
2. User triggers: `echo mem > /sys/power/state`
3. Linux prepares for suspend
4. ATF detects M7 LPA flag (`SRC->GPR10 = 0x5555`)
5. ATF keeps M7 clock + PLLs enabled
6. ATF unmaskes GIR interrupt
7. A53 enters Stop mode
8. DDR enters self-refresh (~50 mW)
9. M7 continues from TCM (<10 mW)
10. M7 detects critical event
11. M7 triggers GIRO (`MU->CR |= (1<<16)`)
12. A53 wakes from Stop mode
13. ATF restores clocks/PLLs
14. Linux resumes
15. Both cores running again

Power savings:

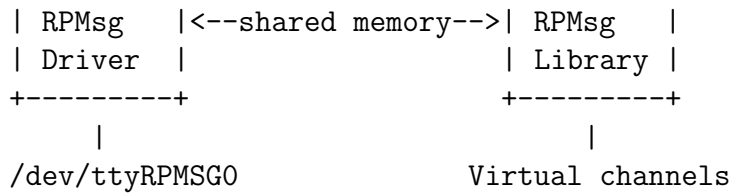
- A53 Stop: 90% reduction from active
- M7 from TCM: ~10 mW
- DDR self-refresh: 50 mW
- Total: 100 mW (vs 2W active)

9.8 RPMsg Communication

9.8.1 RPMsg Overview

RPMsg provides message-passing between A53 (Linux) and M7 (Nuttx/bare-metal):

Linux (A53)	M7 (Nuttx/Bare)
+-----+	+-----+



9.8.2 RPMsg Status During Internship

⚠ Important Note

RPMsg Implementation - Limited Success:

What worked:

- Basic string echo with NXP pre-built examples
- Channel creation and MU interrupts
- Simple command/response protocols
- /dev/ttyRPMMSG0 device appears correctly

What did NOT work:

- High-bandwidth sensor data streaming
- Integration with custom NuttX port
- Multi-channel concurrent operation
- Zero-copy buffer optimization

The RPMsg infrastructure is present but requires significant additional work for production use.

Basic RPMsg usage from Linux:

```

1 # Load driver
2 modprobe imx_rpmsg_tty
3
4 # Check devices
5 ls /dev/ttyRPMMSG* # Should see /dev/ttyRPMMSG0
6
7 # Send/receive
8 echo "hello" > /dev/ttyRPMMSG0
9 cat /dev/ttyRPMMSG0 # M7 echoes back

```

9.9 RDC and Peripheral Ownership

9.9.1 Peripheral Conflict Prevention

Both A53 and M7 can access peripherals - conflicts must be prevented.

Solution 1: Device Tree Exclusion (Used during internship)

```
1 // In device tree
2 &uart2 { status = "disabled"; }; // M7 uses UART2
3 &i2c3 { status = "disabled"; }; // M7 uses I2C3
4 &i2c5 { status = "disabled"; }; // M7 uses I2C5
```

Solution 2: RDC Configuration (Not implemented)

RDC provides hardware-enforced isolation but was not configured during this internship. For production, RDC should be set up in ATF/U-Boot.

9.9.2 Shared Peripheral Strategies

For peripherals that must be shared:

- Use RPMsg virtual driver (Linux requests, M7 executes)
- Time-division (A53 active, M7 suspend and vice versa)
- Never disable clocks from M7 (will crash Linux)

9.10 NuttX RTOS Port

9.10.1 NuttX Port Status

Important Note

NuttX RTOS: FUNCTIONAL WITH LIMITATIONS

NuttX RTOS was successfully ported to the M7 core during this internship with significant functionality:

What works :

- NuttX kernel boots reliably on M7 core
- UART4 console (115200 baud, 8N1) fully functional
- NShell (NSH) command line interface
- I2C3 bus with device detection
- Sensor communication (ADXL345 accelerometer verified)
- Task scheduling and basic RTOS features
- I2C tools for debugging and testing

What does NOT work :

- RPMsg integration with Linux (critical gap!)
- PX4 Autopilot integration (very complex, incomplete)
- High-bandwidth inter-processor communication
- Advanced DMA with NuttX drivers
- Multi-core synchronization with A53

NuttX provides a solid foundation but requires significant work for production flight control.

9.10.2 NuttX Setup Process

Initial setup:

```

1 # Create working directory
2 # I did this on wsl ubuntu 22.04 for context
3 mkdir ~/nuttx-work
4 cd ~/nuttx-work
5
6 # Clone NuttX (mainline Apache)
7 git clone https://github.com/apache/nuttx.git nuttx
8 git clone https://github.com/apache/nuttx-apps.git apps
9
```

```

10 # Install dependencies
11 sudo apt update
12 sudo apt install -y gcc-arm-none-eabi build-essential \
13     kconfig-frontends gperf xxd u-boot-tools

```

9.10.3 Board Configuration

Key decision: Used Toradex Verdin-MX8MP board configuration as base, as it's the closest existing board to the i.MX8MP EVK.

```

1 cd ~/nuttx-work/nuttx
2
3 # Configure for Verdin-MX8MP with NSH (NuttShell)
4 ./tools/configure.sh verdin-mx8mp:nsh
5
6 # Build
7 make clean
8 make -j$(nproc)
9
10 # Output: nuttx.bin - ready to load to M7 core

```

Related Documentation

NuttX i.MX8MP Support:

Critical GitHub Pull Request that added Verdin-MX8MP support:

<https://github.com/apache/nuttx/pull/10486>

This PR contains:

- Device tree examples
- Board initialization code
- Peripheral configurations
- Memory layout

The first forum discussion i saw about i.MX8MP M7 core with NuttX:

<https://lists.apache.org/thread/yj7jj7rz0svrnz8zn1nv1gzygz5gorkp>

9.10.4 Loading NuttX to M7 Core

```

1 # Copy binary to Linux firmware directory
2 sudo cp nuttx.bin /lib/firmware/
3
4 # Load and start M7 core
5 echo nuttx.bin | sudo tee /sys/class/remoteproc/remoteproc0/firmware
6 echo start | sudo tee /sys/class/remoteproc/remoteproc0/state
7
8 # Check status
9 cat /sys/class/remoteproc/remoteproc0/state

```

```

10 # Should show: running
11
12 # Stop M7 core (when needed)
13 echo stop | sudo tee /sys/class/remoteproc/remoteproc0/state

```

9.10.5 UART4 Console Configuration

Initial problem: No console output after boot - couldn't see NuttX messages.

Solution: UART4 setup

NuttX configuration (menuconfig):

```

1 cd ~/nuttx-work/nuttx
2 make menuconfig
3
4 # Navigate to:
5 Device Drivers -> Serial Driver Support
6 [*] UART4 serial port
7     Default baudrate = 115200
8     Bits = 8
9     Parity = None
10    Stop bits = 1

```

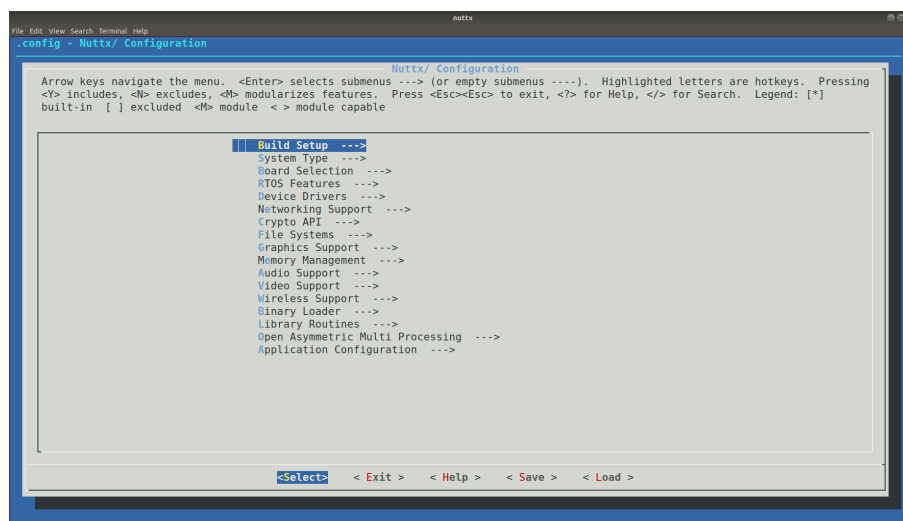


Figure 9.4: Enter Caption

Testing console:

```

1 # Connect via serial (UART4 on EVK board)
2 picocom -b 115200 /dev/ttyUSB0
3
4 # Expected output after M7 boots:
5 NShell (NSH) NuttX-12.x.x
6 nsh>
7
8 # Test commands:
9 nsh> help

```

```

10 nsh> free
11 nsh> ps

```

Success! Console working reliably.

9.10.6 I2C3 Bus Configuration

Goal: Enable I2C3 bus for sensor communication (ADXL345 accelerometer).

I2C3 hardware details:

- Pins: GPIO_SAI5_RXD0 (SDA), GPIO_SAI5_RXD1 (SCL)
- Base Address: 0x30A40000
- Clock: 24 MHz (default)

Enable I2C driver in NuttX:

```

1 cd ~/nuttx-work/nuttx
2 make menuconfig
3
4 # Navigate to:
5 System Type -> iMX8M Peripheral Selection
6 [*] I2C3
7
8 Device Drivers -> I2C Driver Support
9 [*] I2C Driver Support
10 [*] I2C character driver

```

Enable I2C tools for testing:

```

1 # In menuconfig:
2 Application Configuration -> NSH Library
3 Disable Built-in Commands
4 [ ] Disable i2c # Make sure NOT checked
5
6 Application Configuration -> System Libraries and NSH Add-Ons
7 [*] I2C tool

```

Rebuild and test:

```

1 make clean
2 make -j$(nproc)
3
4 # Load to M7
5 sudo cp nuttx.bin /lib/firmware/
6 echo stop | sudo tee /sys/class/remoteproc/remoteproc0/state
7 echo start | sudo tee /sys/class/remoteproc/remoteproc0/state
8
9 # Connect to console
10 picocom -b 115200 /dev/ttyUSB0

```

9.10.7 I2C Bus Scanning

Scan I2C3 bus for devices:

```

1 nsh> i2c dev 3 0x00
2      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
3 00:      -- -- -- -- -- -- -- -- -- -- -- -- -- --
4 10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
5 20: 20 -- -- -- -- -- -- -- -- -- -- -- -- -- --
6 30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
7 40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
8 50: 50 -- -- 53 -- -- -- -- -- -- -- -- -- --
9 60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
10 70: -- -- -- -- -- -- -- -- --

```

Devices detected:

- **0x20:** Unknown (possibly GPIO expander)
- **0x50:** Unknown (possibly EEPROM)
- **0x53:** ADXL345 Accelerometer

9.10.8 ADXL345 Accelerometer Verification

Read Device ID (should be 0xE5):

```

1 nsh> i2c get -a 53 -r 0 3 1
2 READ Bus: 3 Addr: 53 Subaddr: 00 Value: e5

```

Device ID correct! ADXL345 responding.

9.10.9 RPMsg Integration Attempt

Important Note

RPMsg with NuttX: DID NOT WORK

RPMsg communication between Linux (A53) and NuttX (M7) was attempted but unsuccessful.

What was tried:

- NuttX RPMsg library configuration
- Linux kernel RPMsg TTY driver
- Shared memory buffer setup
- Resource table configuration

What failed:

- NuttX could not establish RPMsg channel with Linux
- `/dev/ttyRPMMSG0` did not appear
- MU interrupts not properly handled in NuttX
- VirtIO ring buffer initialization issues

Related Documentation

Potential Solution for Next Intern:

NuttX RPMMsg pull request with Linux kernel patches:

<https://github.com/apache/nuttx/pull/12235>

This PR includes:

- Updated NuttX RPMMsg implementation
- Linux kernel driver patches needed for compatibility
- Example configurations
- Documentation on integration

Important: The Linux kernel patches from this PR were NOT applied during this internship. They may be required to get RPMMsg working properly.

Next intern should:

1. Apply Linux kernel patches from PR #12235
2. Update NuttX to version with RPMMsg fixes
3. Test basic RPMMsg channel creation
4. Validate bidirectional communication
5. Implement high-bandwidth streaming

9.11 PX4 Flight Controller Integration

9.11.1 PX4 Port Attempt

Important Note

PX4 Status: INCOMPLETE Since PX4 is not compatible with imx8mp M7 by default. I tried to rewrite all the necessary files needed by copying similar builds PX4 Autopilot integration was attempted but not even close to being successful.

9.11.2 What Was Attempted

1. Initial PX4 setup:

```
1 cd ~
2 git clone https://github.com/PX4/PX4-Autopilot.git --recursive
3 cd PX4-Autopilot
```

2. Board configuration created:

```
1 # Created new board based on fmu-v6xrt template
2 mkdir -p boards/nxp/imx8mp-evk
3 cp -r boards/px4/fmu-v6xrt/* boards/nxp/imx8mp-evk/
```

```

4
5 # Copied working NuttX config
6 cp ~/nuttx-work/nuttx/boards/arm/mx8mp/verdin-mx8mp/configs/nsh/defconfig \
7   boards/nxp/imx8mp-evk/nuttx-config/nsh/defconfig

```

3. PX4 platform code additions:

Added mx8mp chip detection in `platforms/nuttx/cmake/px4_impl_os.cmake`:

```

1 elseif(CONFIG_ARCH_CHIP_MX8MP)
2     set(CHIP_MANUFACTURER "nxp")
3     set(CHIP "mx8mp")

```

Created platform code:

```

1 cd platforms/nuttx/src/px4/nxp/
2 cp -r rt117x mx8mp # Used RT117x as template

```

4. Critical discovery: PX4's embedded NuttX lacks mx8mp support!

```

1 # Copied mx8mp architecture files to PX4's embedded NuttX
2 cp -r ~/nuttx-work/nuttx/arch/arm/src/mx8mp \
3   platforms/nuttx/NuttX/nuttx/arch/arm/src/
4
5 cp -r ~/nuttx-work/nuttx/arch/arm/include/mx8mp \
6   platforms/nuttx/NuttX/nuttx/arch/arm/include/
7
8 # Fixed chip symlink
9 cd platforms/nuttx/NuttX/nuttx/arch/arm/src/
10 rm chip
11 ln -s mx8mp chip

```

5. Minimal board configuration:

Created `board_config.h`:

```

1 #pragma once
2
3 #include <nuttx/config.h>
4 #include <px4_platform_common/px4_config.h>
5 #include <nuttx/compiler.h>
6 #include <stdint.h>
7 #include <arch/board/board.h>
8
9 #define BOARD_NAME "IMX8MP_EVK"
10 #define BOARD_HAS_CONTROL_STATUS_LEDS 0
11
12 __BEGIN_DECLS
13 __EXPORT const char *board_get_hw_type_name(void);
14 __EXPORT int board_get_hw_version(void);
15 __EXPORT int board_get_hw_revision(void);
16 __END_DECLS

```

6. Disabled unsupported drivers:

In `boards/nxp/imx8mp-evk/default.px4board`:

```

1 # Disabled drivers requiring chip-specific implementations:

```

```

2 # CONFIG_DRIVERS_UAVCAN=y          # CAN bus
3 # CONFIG_DRIVERS_ADC_BOARD_ADC=y   # ADC
4 # CONFIG_DRIVERS_DSHOT=y           # Motor control
5 # CONFIG_DRIVERS_PX4IO=y           # I/O coprocessor
6 # CONFIG_DRIVERS_TONE_ALARM=y       # Audio

```

7. Build attempt:

```

1 cd ~/PX4-Autopilot
2 make clean
3 make nxp_imx8mp-evk_default

```

9.11.3 Why PX4 Integration Failed

Multiple complex issues encountered:

1. Board path configuration:

- PX4's NuttX uses custom board directory structure
- CONFIG_ARCH_BOARD_CUSTOM_DIR path resolution issues
- Build system couldn't locate board files consistently

2. Embedded NuttX vs standalone:

- PX4 has its own NuttX fork with different structure
- Many mx8mp-specific files missing or incompatible
- Version mismatches between standalone NuttX and PX4's embedded version

3. Driver dependencies:

- PX4 expects many chip-specific drivers not ported to mx8mp
- Build system tightly coupled to supported platforms
- Missing: ADC, DShot, CAN, I/O coprocessor drivers

9.11.4 Files Modified During PX4 Attempt

```

PX4-Autopilot/
|-- boards/nxp/imx8mp-evk/
|   |-- default.px4board          # Board configuration
|   |-- src/
|       |-- board_config.h        # Board definitions
|       |-- init.c                # Initialization
|       CMakeLists.txt            # Build config
|       nuttx-config/nsh/defconfig # NuttX config
|-- platforms/nuttx/cmake/
|   px4_impl_os.cmake             # Added mx8mp detection
|-- platforms/nuttx/src/px4/nxp/
|   mx8mp/                        # Platform code (from rt117x)
platforms/nuttx/NuttX/nuttx/

```

```

arch/arm/
|-- src/mx8mp/           # Architecture code
include/mx8mp/          # Headers

```

9.11.5 Key Resources

Related Documentation

Essential Documentation: NXP Official:

- i.MX8MP Reference Manual
- AN5317: Using Cortex-M with Linux
- MCUXpresso SDK: <https://mcuxpresso.nxp.com>

NuttX Resources:

- NuttX Documentation: <https://nuttx.apache.org/docs/>
- i.MX8MP Support PR: <https://github.com/apache/nuttx/pull/10486>
- RPiMsg with kernel patches: <https://github.com/apache/nuttx/pull/12235>
- Community forum discussion: <https://lists.apache.org/thread/yj7jj7rz0svrnz8zn1nv1gzygz5gorkp>
- NuttX Google Groups (search: "imx8mp", "verdin-mx8mp")

PX4 Resources (if attempting integration):

- PX4 Developer Guide: <https://docs.px4.io/main/en/>
- PX4 NXP Platform Code: RT117x implementation as reference
- MAVLink Protocol: <https://mavlink.io/>

Working nuttx elf:

- GitHub (bare-metal): <https://github.com/yulyuri/firmware-and-dts/blob/master/nuttx-m7-uart4-i2c.elf>
- NuttX configuration (in home directory backup)

Chapter 10

Conclusion and Support

10.1 Wrapping Up

You've reached the end of this documentation! Yay!

10.2 Documentation

GitHub Repository

LaTeX Source Code Available:

The complete LaTeX source for this documentation will be included in the GitHub repository:

<https://github.com/yulyuri/firmware-and-dts>

Why this matters:

- You can update it as you make progress
- Fix any possible errors I made or unclear sections
- Add your own discoveries and of course help future interns/ anyone working on this

10.3 Getting Help

10.3.1 Contact me

If you get stuck or have some questions:

My contact information is at the beginning of this document. Just send an email, I'm happy to help clarify things. I was struggling very hard

10.3.2 NXP Community Forums - Your Best Friend

Important Note

The NXP Community Forums Are very helpful:

<https://community.nxp.com/>

The NXP forums saved me countless times during this internship. I asked **god knows how many questions** there, and the community is incredibly helpful.

10.4 Final Words

This has been an incredibly rewarding internship. The i.MX8M Plus is a powerful platform, and you now have the foundation to build amazing things with it. I Good luck with the project. I hope you are able to learn and have as much fun as I did.

*Chyn
December 2025*